Table of Contents

1.0 Introduction **3**

2.0 Problem Solving and Design **5**

2.1 Functionalities 5

2.1.1 General Function 5

2.1.2 Explorer Role 6

2.1.3 Guide Role 7

2.1.4 Equipment Provider Role 7

2.2 Use Case Diagram 9

2.3 System Architecture Design 10

2.4 Dynamic Data Management 21

2.4.1 User 21

2.4.2 Watchlist 29

2.4.3 Emergency SOS 34

2.4.4 Calendar 37

3.4.5 Guide 41

3.4.6 Review 44

3.4.7 Places 52

3.4.8 Chat 56

2.4.9 Wallet 64

2.4.10 Voucher 70

2.4.11 Equipment 74

2.4.12 Bookings & Events 88

2.5 User Permission 99

2.6 Wireframe 103

2.6.1 Common Functions 103

2.6.2 Explorer Role 114

2.6.3 Guide Role 128

2.6.4 Equipment Provider Role 135

3.0 User Manual 147

3.1 Common Functions 147

3.2 Explorer Role 155

3.3 Guide Role 169

3.4 Equipment Provider Role 185

4.0 Automated System Testing 197

4.1 Manual Testing 197

4.1.1 Authentication Validation Unit Test 197

4.1.2 Voucher Validation Unit Test 202

4.1.3 Equipment Validation Unit Testing 206

4.2 Automated Testing 210

4.2.1 Pre-Login Section 211

4.2.2 Explorer Section 214

4.2.3 Guide Section 218

4.2.4 Equipment Provider Section 222

5.0 Conclusion 226

6.0 Reference 227

7.0 Appendix 230

7.1 Knowledge of the Application 230

7.2 Work Breakdown Structure 231

1.0 Introduction

Based on our analysis and data gathering, we found that explorers face difficulties in finding reliable guides and comparing between them. Guides often encounter challenges in managing their trips, as they have to manually provide safety precautions and item checklists to each user. Meanwhile, equipment providers struggle with the lack of a consistent platform to manage their inventory and sales. These issues collectively motivated the development of the PathFindr application.

This report presents the complete design, implementation, and testing of PathFindr, a mobile application built to connect outdoor explorers with qualified guides and reliable equipment providers. The system is architected around the Model-View-ViewModel (MVVM) pattern, ensuring a clear separation between user interface, business logic, and data access layers. Three distinct user roles—Explorer, Guide, and Equipment Provider—have been catered for with tailored functionality: explorers can search for and book guided outdoor activities, guides can manage bookings and safety checklists, and equipment providers can maintain inventory and process orders. Throughout the report, we detail the core system components, such as dynamic data management with Firebase and Floor, the use-case diagrams, and our approach to real-time features like SOS alerts and live map sessions. A comprehensive user manual guides each role through the application, and both manual and automated testing procedures validate the app's reliability across multiple devices and scenarios.

2.0 Problem Solving and Design

2.1 Functionalities

2.1.1 General Function

- Explorer, Guide and Equipment Provider can Introduction Page.
- Explorer, Guide and Equipment Provider can give location access to the application to view nearby outdoor locations.
- Explorer, Guide and Equipment Provider can register themselves a new account by providing details like name, email address and password
- Explorer, Guide and Equipment Provider can register themselves a new account via their Google account.
- Explorer, Guide and Equipment Provider can login into their respective accounts using their email address and password.
- Explorer, Guide and Equipment Provider can login into their respective accounts via their Google account.
- Explorer, Guide and Equipment Provider can edit the application theme at the Settings page.
- Explorer, Guide and Equipment Provider can edit the application Preferences at the Settings page.
- Explorer, Guide and Equipment Provider can log out to the application.
- Explorer, Guide and Equipment Provider can chat in person or in groups.
- Explorer, Guide and Equipment Provider can search for specific users using name or email address.
- Explorer, Guide and Equipment Provider can search for specific conversation.
- Explorer, Guide and Equipment Provider can create new group chat.
- Explorer, Guide and Equipment Provider can view their current balance.
- Explorer, Guide and Equipment Provider can view their recent transactions.
- Explorer, Guide and Equipment Provider can view their transactions history.
- Explorer, Guide and Equipment Provider can generate their transaction record in PDF format.

2.1.2 Explorer Role

- Explorer can view notifications.
- Explorer can search for nearby outdoor activities.
- Explorer can add certain places into the watchlist.
- Explorer can view watchlist.
- Explorer can view details of the places, including name, type, difficulty, distance and category
- Explorer can open the place location at Google Maps.
- Explorer can book a guide for trip guides with details such as starting date, end date, number of people and expected price.
- Explorer can view the weather forecast when booking a guided trip.
- Explorer can choose their preferred guide for their trip.
- Explorer can view the guide profile when choosing a guide.
- Explorer can filter their booking states
- Explorer can view the booking details
- Explorer can withdraw their booking.
- Explorer can view the event status.
- Explorer can view the events details.
- Explorer can join a live session map.
- Explorer can send emergency alerts.
- Explorers can provide feedback after an event on its difficulty, the guide, and their overall experience.
- Explorer can make their payment.
- Explorer can top-up their wallet.
- Explorer can view provided equipment.
- Explorer can search for equipment needed.
- Explorer can view the details of the equipment.
- Explorer can rent the equipment.
- Explorer can buy the equipment.
- Explorer can choose their preferred delivery method: pickup or delivery.

- Explorer can apply voucher.
- Explorer can filter the equipment they rent, or purchase based on transaction type, delivery method or status.
- Explorer can rate the equipment.

2.1.3 Guide Role

- Guide can view the total events hosted.
- Guide can view booking requests.
- Guide can filter booking requests by their status.
- Guide can view cancelled events
- Guide can filter cancelled events by their status.
- Guide can view their earning statistics.
- Guide can filter their earnings by week, month or year.
- Guide can choose to accept to reject the booking request.
- Guide can view their schedule.
- Guide can filter their schedule by day, month, year.
- Guide can set their status to “available” or “unavailable”.
- Guide can withdraw their funds.
- Guide can add safety precautions and checklists for specific events.
- Guide can edit safety precautions and checklists for specific events.
- Guide can join a live session map.
- Guide can send emergency alerts.
- Guide can view feedback from participants.
- Guide can mark the event as “completed”.

2.1.4 Equipment Provider Role

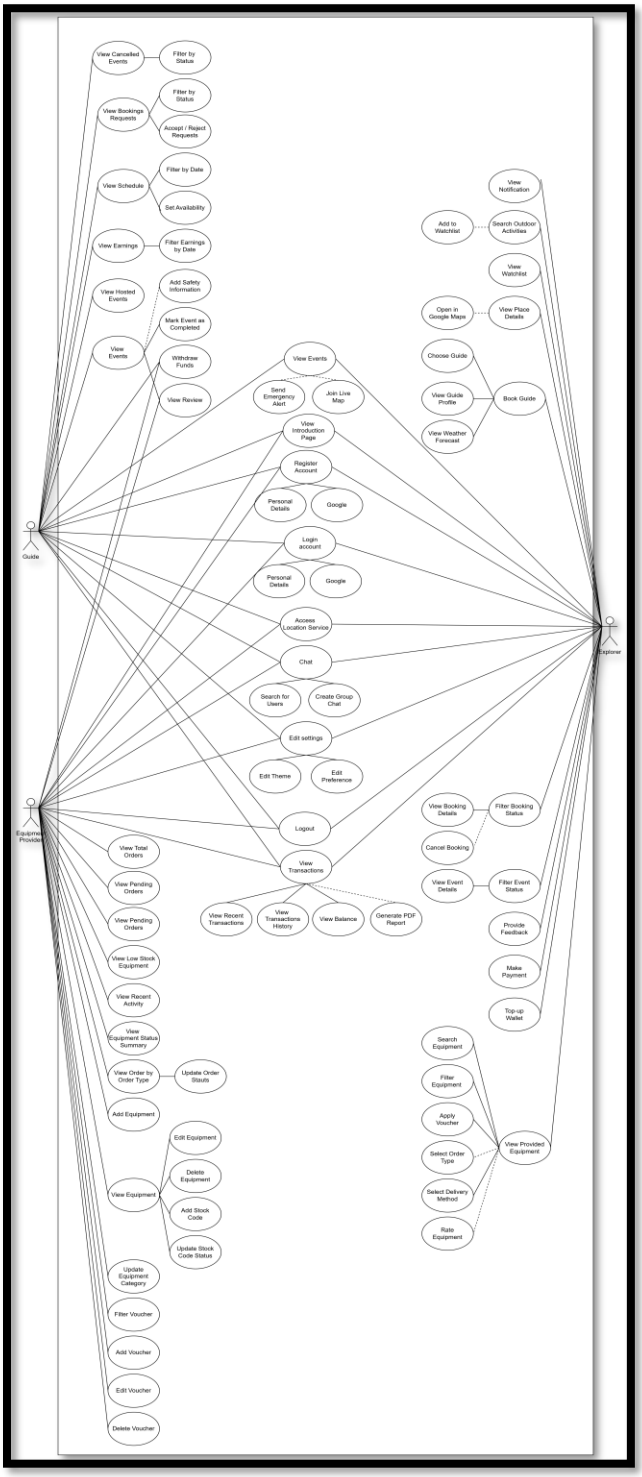
- Equipment Provider can view the total orders for the current day.
- Equipment Provider can the pending order for the current day.

- Equipment Provider can view the number of equipment that has “low stock” status.
- Equipment Provider can view recent activity
- Equipment Provider can withdraw their funds.
- Equipment Provider can view customer orders by type: rental or purchase.
- Equipment Provider can update the equipment rental status.
- Equipment Provider can update the equipment order status.
- Equipment Provider can update equipment category
- Equipment Provider can view the number of equipment items under the following categories: low stock, under maintenance, or rented out.
- Equipment Provider can view equipment based on category.
- Equipment Provider can add new equipment with details such as name, description, category, type, price, image and terms and conditions.
- Equipment Provider can edit equipment information such as name, description, category, type, and terms and conditions.
- Equipment Provider can add new equipment stock code.
- Equipment Provider can update the status of equipment stock code.
- Equipment Provider can delete equipment.
- Equipment Provider can add new voucher with details such as voucher code, discount percentage and expiry date.
- Equipment Provider can edit vouchers with details such as voucher code, discount percentage and expiry date.
- Equipment Provider can delete vouchers.
- Equipment Provider can filter vouchers based on categories: all, active, expired.
- Equipment Provider can withdraw their funds.
- Equipment Provider can view customer review on specific equipment.
- Equipment Provider can view revenue reports that include average revenue, number of transactions, growth rate, last transaction date and monthly breakdown.

2.2 Use Case Diagram

Figure 1

Use Case Diagram

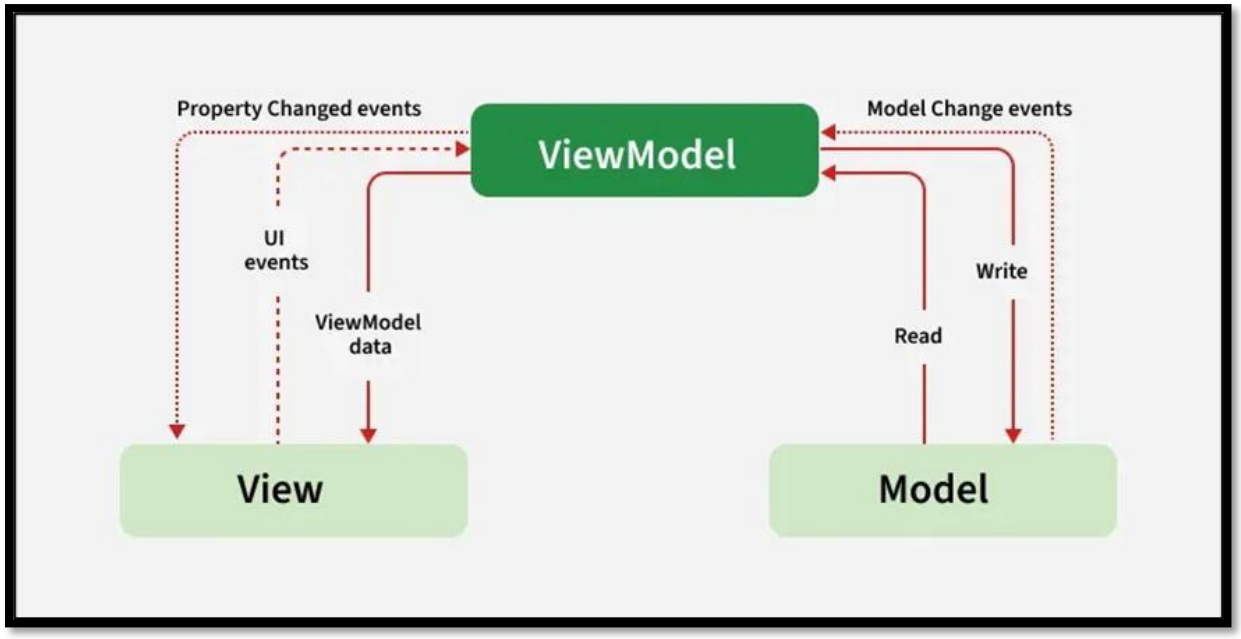


2.3 System Architecture Design

The development of the PathFindr system is guided by the adoption of the Model-View-ViewModel (MVVM) architecture pattern. MVVM provides a robust and modular framework for structuring mobile applications by dividing responsibilities into three distinct components: the Model, View, and ViewModel (Daoudi et al., 2019). The Model is responsible for managing data and business logic. In the context of PathFindr, it includes integration with cloud platforms like Firebase for real-time data, as well as the use of local storage through Floor, a SQLite-based ORM in Flutter. The View refers to the user interface (UI) layer, constructed with Flutter widgets tailored for the three distinct user roles—Explorer, Guide, and Equipment Provider. Lastly, the ViewModel serves as a mediator that processes data from the Model and prepares it for display in the View, while being lifecycle-aware and independent of the UI (Fuksa et al., 2025).

Figure 2

System Architecture



As visualized in the MVVM architecture diagram, the system utilizes a unidirectional data flow (Fuksa et al., 2025). This allows the View to observe the ViewModel, which in turn retrieves and processes data from the Repository. The Repository serves as an abstraction layer for accessing data from various sources, such as Firebase, local Floor databases, or third-party APIs like Geoapify and Open-Metro (Daoudi et al., 2019). This architectural separation improves code readability, maintainability, and testability. It also ensures that user interactions and business logic are decoupled, making it easier to scale the application or add features such as real-time trail tracking, SOS alerts, and in-app messaging.

The decision to implement MVVM in PathFindr is justified by the need to manage complex interactions across multiple user roles. Each role—Explorer, Guide, and Equipment Provider—has unique functions and interfaces yet relies on shared logic and backend services (Daoudi et al., 2019). MVVM enables each component to be developed and debugged independently, minimizing cross-impact between modules. For example, the ViewModel for the Explorer module handles live location data and user-specific filters, while the Guide's ViewModel manages trip phase tracking and emergency handling. Meanwhile, equipment providers rely on ViewModels to monitor inventory and synchronize changes to their gear listings. This approach greatly enhances the modularity of the system and ensures that the codebase remains clean and scalable as the app grows (Fuksa et al., 2025).

One of the primary advantages of MVVM is its support for testability and lifecycle awareness. Because the ViewModel does not directly reference any UI elements, its logic can be unit-tested in isolation. Furthermore, it can survive configuration changes, such as device rotations, without losing state. This is particularly advantageous for a system like PathFindr, where real-time updates must be preserved across multiple states of the app. This architecture also supports better team collaboration, allowing frontend developers to work on the UI while backend developers focus on logic and data services without interference (Daoudi et al., 2019).

The benefits and disadvantages of using MVVM in PathFindr are summarized in the tables below:

Table 1

Benefits of MVVM Architecture

Benefit	Description
Testability	ViewModel logic can be tested independently without UI dependencies (Fuksa et al., 2025).
Separation of Concerns	UI, business logic, and data access are cleanly separated, simplifying development and debugging (Fuksa et al., 2025).
Improved Collaboration	Allows UI and logic/ or backend teams to work in parallel.
Scalability	New features can be added to individual modules without affecting others (Fuksa et al., 2025).
Lifecycle Management	ViewModel retains state across configuration changes like screen rotations.

Table 2

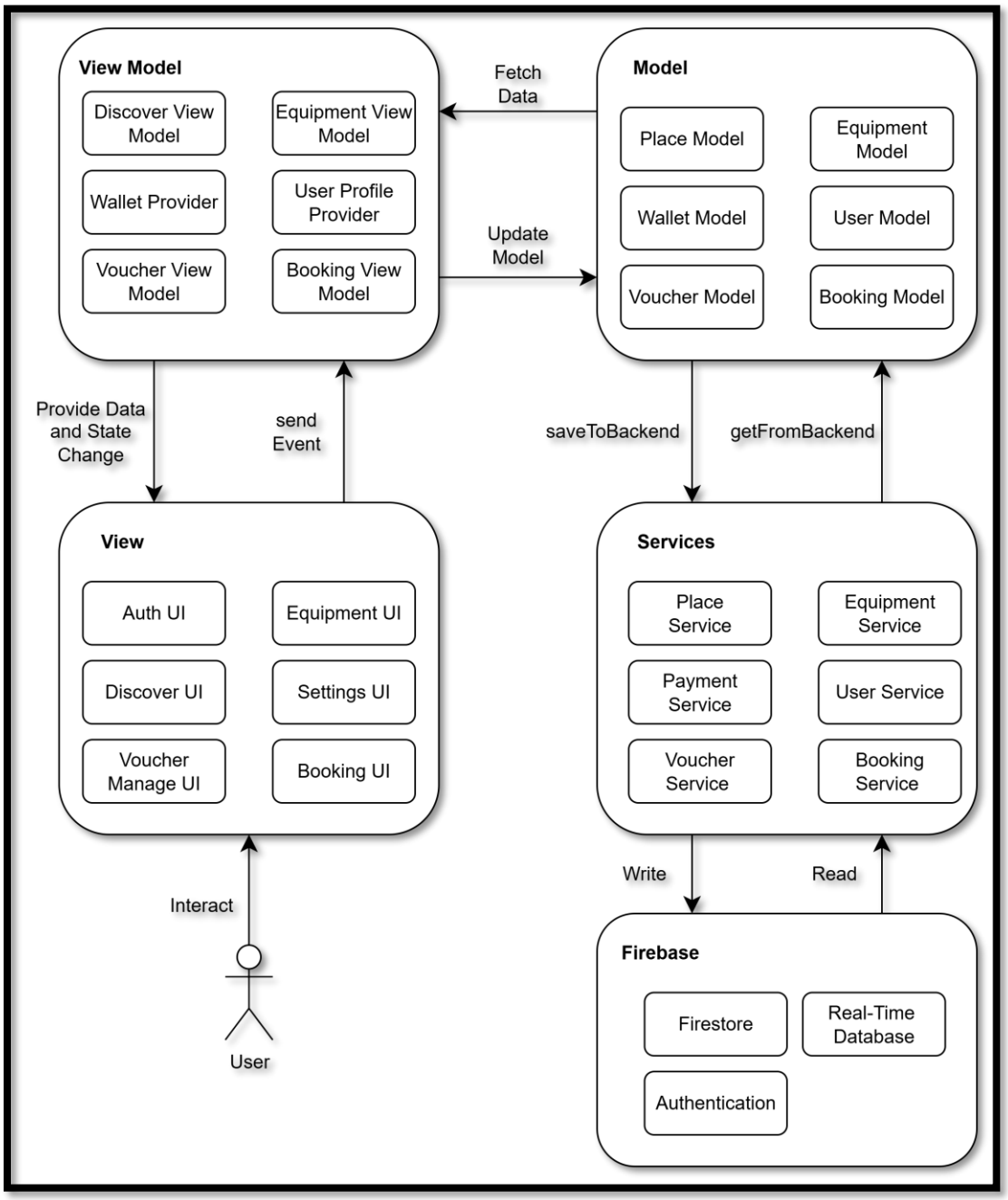
Disadvantages of MVVM Architecture

Disadvantage	Mitigation Strategy in PathFindr
Steeper Learning Curve	Mitigated by using well-documented folder structures and consistent templates across modules (Wilson et al., 2022).
Boilerplate Code	Reusing shared ViewModel and repository components while abstracting redundant code segments (Fuksa et al., 2025).

Complex State Management	Managed using Provider or flutterx_live_data for clean and efficient state handling (Wilson et al., 2022).

Figure 3

System Architecture of PathFindr



The diagram above shows the system architecture diagram, consisting of selected parts of the PathFindr system, such as a few views, view models, models, and services. In the diagram above, it is observed that the PathFindr application can be divided into five sections: View Layer, ViewModel Layer, Model Layer, Service Layer, and Firebase. The View Layer serves as the user interface to interact with the users. It will send event actions to the ViewModel for processing, and the ViewModel will return the results and state changes to the View Layer to update the user interface.

For the interaction between the Model and ViewModel, the ViewModel will consist of various models for better data storage and management. This avoids the need to manually create variables repeatedly. For services, the ViewModel will work together with the Model to call the Service Layer, which contains the logic that interacts with Firebase. The clear separation between the Service Layer, Model, and ViewModel enables the services to be reused across different files and makes future updates easier. Once the Service Layer is called, it will fetch or update data from Firebase, whether it is Firestore, Authentication, or the Realtime Database.

Here is an example for how the Pathfindr project implements under MVVM architecture.

For a start, the folder structure of the PathFindr project is organized to reflect the MVVM architecture. At the top level, files are divided into directories such as models, views, view_models, and services. This separation allows each module to handle a specific responsibility in line with the MVVM principle of separation of concerns.

Figure 4

File Architecture

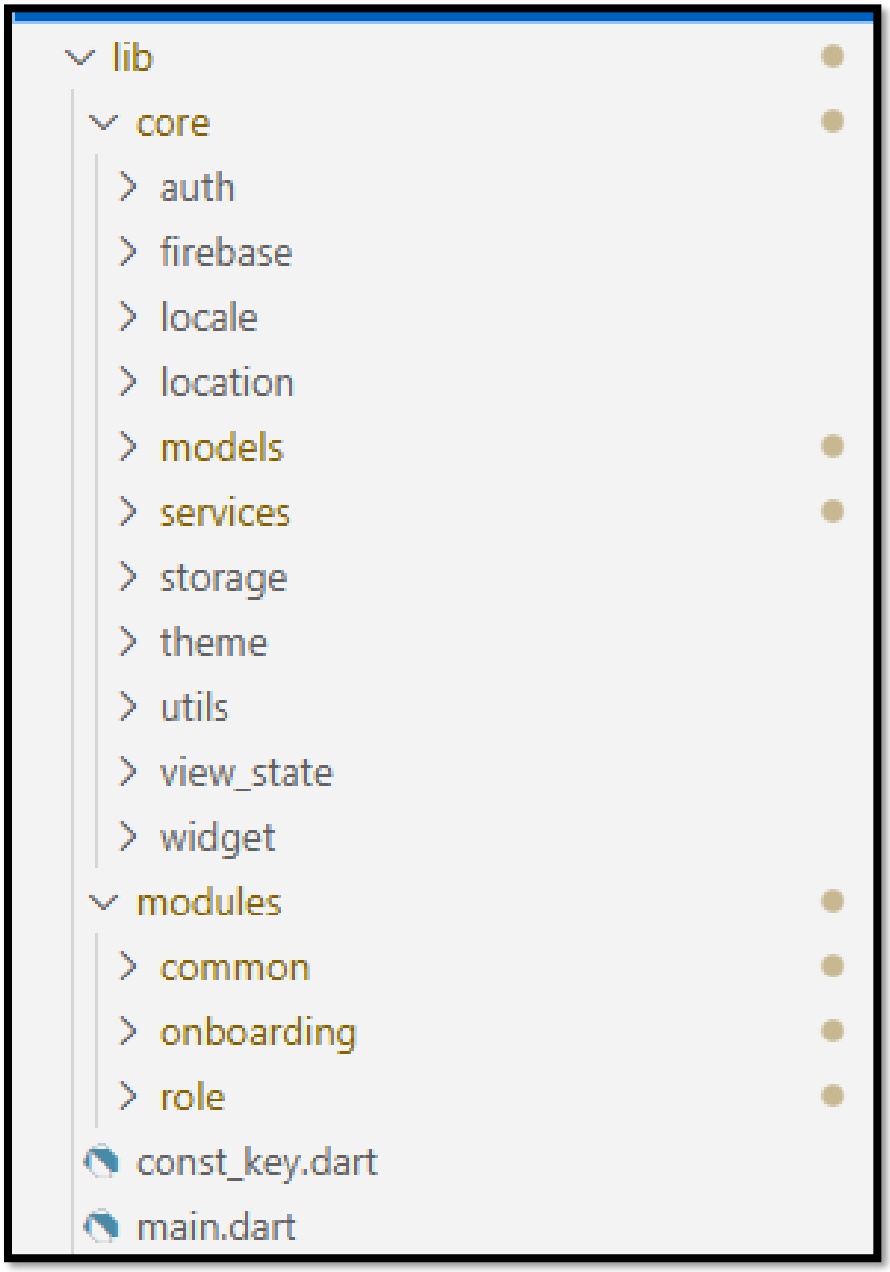


Figure 5

Data Model

```dart
class Places {
  String? id;
  List<String>? types;
  String? formattedAddress;
  Location? location;
  num? rating;
  DisplayName? displayName;
  List<Photos>? photos;

  Places({
    this.id,
    this.types,
    this.formattedAddress,
    this.location,
    this.rating,
    this.displayName,
    this.photos,
  });

  Places.fromJson(Map<String, dynamic> json) {
    id = json['id'];
    types = json['types'].cast<String>();
    formattedAddress = json['formattedAddress'];
    location =
        json['location'] != null
            ? new Location.fromJson(json['location'])
            : null;
    rating = json['rating'];
    displayName =
        json['displayName'] != null
            ? new DisplayName.fromJson(json['displayName'])
            : null;
    if (json['photos'] != null) {
      photos = <Photos>[];
      json['photos'].forEach((v) {
        photos!.add(new Photos.fromJson(v));
      });
    }
  }

  Map<String, dynamic> toJson() {
    final Map<String, dynamic> data = new Map<String, dynamic>();
    data['id'] = this.id;
    data['types'] = this.types;
    data['formattedAddress'] = this.formattedAddress;
    if (this.location != null) {
      data['location'] = this.location!.toJson();
    }
    data['rating'] = this.rating;
    if (this.displayName != null) {
      data['displayName'] = this.displayName!.toJson();
    }
    if (this.photos != null) {
      data['photos'] = this.photos!.map((v) => v.toJson()).toList();
    }
    return data;
  }
}
```

Under the model's directory, data classes are defined to represent real-world objects like users, guides, trips, and equipment. These model classes act as the data source for the ViewModel and may include methods for JSON serialization and deserialization to interact with backend services such as Firebase.

Figure 6

Data Services

```dart
class PlacesService {
  static const String _baseUrl =
      'https://places.googleapis.com/v1/places:searchNearby';

  /// Search for nearby places based on location
  static Future<List<Places>> searchNearbyPlaces({
    required double latitude,
    required double longitude,
    double radiusInMeters = 50000,
    List<String>? includedTypes,
    List<String>? excludedTypes,
    int maxResults = 20,
    String languageCode = 'en',
  }) async {
    try {
      final headers = {
        'Content-Type': 'application/json',
        'X-Goog-Api-Key': ConstKey.googlePlacesApiKey,
        'X-Goog-FieldMask':
            'places.displayName,places.formattedAddress,places.location,places.types,places.rating,places.id,places.photos',
      };

      final requestBody = {
        "locationRestriction": {
          "circle": {
            "center": {"latitude": latitude, "longitude": longitude},
            "radius": radiusInMeters,
          },
        },
        "includedTypes": includedTypes ?? _defaultIncludedTypes,
        "excludedTypes": excludedTypes ?? _defaultExcludedTypes,
        "rankPreference": "DISTANCE",
        "languageCode": languageCode,
        "maxResultCount": maxResults,
      };

      final request = http.Request('POST', Uri.parse(_baseUrl));
      request.body = json.encode(requestBody);
      request.headers.addAll(headers);

      final response = await request.send();
      final responseBody = await response.stream.bytesToString();

      if (response.statusCode == 200) {
        final data = json.decode(responseBody);
        return _parsePlacesFromResponse(data);
      } else {
        throw Exception(
          'Failed to fetch places: ${response.statusCode} - ${response.reasonPhrase}',
        );
      }
    } catch (e) {
      throw Exception('Error searching nearby places: $e');
    }
  }
}
```

The services directory includes helper classes for Firebase interaction, such as AuthService, BookingService, and PlacesServices. These services act as repositories by abstracting the logic for network calls and data fetching. The ViewModel classes rely on these services to retrieve or update data without directly referencing external APIs or databases.

Figure 7

Search ViewModel

```dart
class SearchViewModel extends ViewStateModel {
  final LocationViewModel _locationVm = LocationViewModel();

  List<Places> _searchSuggestions = [];
  List<Places> _searchResults = [];
  List<String> _searchHistory = [];
  Set<String> _watchlistPlaceIds = {};
  bool _isLoadingSuggestions = false;
  bool _isLoadingResults = false;
  bool _showResults = false;
  String _currentQuery = '';
  String? _currentCategory;
  String? _errorMessage;
  Position? _currentPosition;

  // Adventure/outdoor activity categories
  final List<Map<String, dynamic>> _categories = [
    {'name': 'Hiking Areas', 'icon': 'hiking', 'type': 'hiking_area'},
    {'name': 'National Parks', 'icon': 'forest', 'type': 'national_park'},
    {'name': 'State Parks', 'icon': 'park', 'type': 'state_park'},
    {'name': 'Adventure Sports', 'icon': 'sports', 'type': 'adventure_sports_center'},
    {'name': 'Skateboard Parks', 'icon': 'skateboarding', 'type': 'skateboard_park'},
    {'name': 'Cycling Parks', 'icon': 'pedal_bike', 'type': 'cycling_park'},
    {'name': 'Beaches', 'icon': 'beach_access', 'type': 'beach'},
    {'name': 'Marinas', 'icon': 'directions_boat', 'type': 'marina'},
    {'name': 'Wildlife Parks', 'icon': 'pets', 'type': 'wildlife_park'},
  ];

  // Getters
  List<Places> get searchSuggestions => _searchSuggestions;
  List<Places> get searchResults => _searchResults;
  List<String> get searchHistory => _searchHistory;
  Set<String> get watchlistPlaceIds => _watchlistPlaceIds;
  List<Map<String, dynamic>> get categories => _categories;
  bool get isLoadingSuggestions => _isLoadingSuggestions;
  bool get isLoadingResults => _isLoadingResults;
  bool get showResults => _showResults;
  String get currentQuery => _currentQuery;
  String? get currentCategory => _currentCategory;
  String? get errorMessage => _errorMessage;
  Position? get currentPosition => _currentPosition;
}
```

The ViewModel is the layer that holds all the data needed to be loaded in the View layer. Instead of storing the data directly in the View layer, this approach allows the data to be stored separately, reducing the risk of data being refreshed unintentionally when the View is rebuilt. In addition, the ViewModel layer also contains the functions that are called by the View, acting as a bridge between the Service layer and the View layer.

Figure 8

View Layer

```dart
@override
Widget build(BuildContext context) {
  return ChangeNotifierProvider<SearchViewModel>.value(
    value: _viewModel,
    child: Scaffold(
      backgroundColor: FColor.getColor(context, FColorKey.cloudWhite),
      appBar: AppBar(
        backgroundColor: FColor.getColor(context, FColorKey.cloudWhite),
        elevation: 0,
        leading: IconButton(
          icon: Icon(
            Icons.arrow_back,
            color: FColor.getColor(context, FColorKey.primaryText),
          ),
          onPressed: () => Navigator.pop(context),
        ),
        title: Text(
          'Search Places',
          style: TextStyle(
            color: FColor.getColor(context, FColorKey.primaryText),
            fontSize: 20,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
      body: Consumer<SearchViewModel>(
        builder: (context, viewModel, child) {
          // return widget
        }
```

In the above figure, the corresponding UI for the trip screen observes the state exposed by the View Model. Flutter widgets use reactive components like Consumer or ValueListenableBuilder to rebuild only when necessary. This keeps the UI responsive and synchronized with the backend without manual refresh.

Figure 9

Configuration

```dart
runApp(
  MultiProvider(
    providers: [
      ChangeNotifierProvider<BrightnessViewModel>.value(value: brightnessVm),
      ChangeNotifierProvider<JsonLocalization>.value(value: jsonLocalization),
      ChangeNotifierProvider<LocaleViewModel>.value(value: localeVm),
      ChangeNotifierProvider<LocationViewModel>.value(value: locationVm),
      ChangeNotifierProvider<ViewStateModel>(create: (_) => ViewStateModel()),
      ChangeNotifierProvider<AuthService>(create: (_) => AuthService()),
      ChangeNotifierProvider<UserProfileService>(
        create: (_) => UserProfileService(),
      ),
      ChangeNotifierProvider<WalletProvider>(create: (_) => WalletProvider()),
    ],
    child: AppRoot(showOnboarding: !hasSeenOnboarding),
  ),
);
```

Lastly, the main.dart file initializes all global ViewModels using the Provider package. This ensures that the ViewModel is injected into the widget tree and accessible wherever needed. Reinforcing that the MVVM architecture design.

2.4 Dynamic Data Management

2.4.1 User

Figure 10

getUsers function

```dart
Future<UserProfile?> getUsers(String userId) async {
  try {
    final doc =
        await _firestore.collection('user_profiles').doc(userId).get();
    if (doc.exists) {
      return UserProfile.fromMap(doc.data()!);
    }
    return null;
  } catch (e) {
    _setError('Failed to fetch user profile: $e');
    return null;
  }
}
```

The getUsers function is designed to retrieve user profile data from the user_profiles collection using a given userId as input. If a matching document is found, the result is converted into a UserProfile object using the fromMap function.

Figure 11

loadUserProfile function

```dart
// Load user profile from Firestore
Future<bool> loadUserProfile({String? userId}) async {
  try {
    _setLoading(true);
    _setError(null);

    final uid = userId ?? _auth.currentUser?.uid;
    if (uid == null) {
      _setError('No user signed in');
      return false;
    }

    final doc = await _firestore.collection('user_profiles').doc(uid).get();

    if (doc.exists) {
      _userProfile = UserProfile.fromMap(doc.data()!);
      return true;
    } else {
      // Create a basic profile if it doesn't exist
      _userProfile = UserProfile(
        uid: uid,
        email: _auth.currentUser?.email ?? '',
        displayName: _auth.currentUser?.displayName,
        role: null,
        isOnboardingComplete: false,
        createdAt: DateTime.now(),
        updatedAt: DateTime.now(),
      );

      // Save the basic profile to Firestore
      await _firestore
          .collection('user_profiles')
          .doc(uid)
          .set(_userProfile!.toMap());

      return true; // Profile created successfully
    }
  } catch (e) {
    _setError('Failed to load profile: $e');
    return false;
  } finally {
    _setLoading(false);
  }
}
```

The loadUserProfile function is similar to the getUsers function but includes the additional capability to create a user profile if one does not exist. If no matching document is found in the user_profiles collection, the function will create a new UserProfile object, populate it with the necessary data, convert it to JSON, and save it to the user_profiles collection.

Figure 12

createOrUpdateProfile function

```dart
// Create or update user profile
Future<bool> createOrUpdateProfile(UserProfile profile) async {
  try {
    _setLoading(true);
    _setError(null);

    final updatedProfile = profile.copyWith(updatedAt: DateTime.now());

    await _firestore
        .collection('user_profiles')
        .doc(profile.uid)
        .set(updatedProfile.toMap());

    _userProfile = updatedProfile;
    return true;
  } catch (e) {
    _setError('Failed to save profile: $e');
    return false;
  } finally {
    _setLoading(false);
  }
}
```

This function receives a new UserProfile object and searches for the corresponding document in the user_profiles collection using the uid from the profile object. If the document is found, the object is converted to JSON and used to update the existing document with the new data.

Figure 13

updateUserRole function

```dart
Future<bool> updateUserRole(UserRole role) async {
  try {
    _setLoading(true);
    _setError(null);

    final uid = _auth.currentUser?.uid;
    if (uid == null) {
      _setError('No user signed in');
      return false;
    }

    UserProfile profileToUpdate;
    if (_userProfile == null) {
      // Create new profile for new users
      profileToUpdate = UserProfile(
        uid: uid,
        email: _auth.currentUser?.email ?? '',
        displayName: _auth.currentUser?.displayName,
        role: role,
        isOnboardingComplete: false,
        createdAt: DateTime.now(),
        updatedAt: DateTime.now(),
      );
    } else {
      // Update existing profile
      profileToUpdate = _userProfile!.copyWith(
        role: role,
        updatedAt: DateTime.now(),
      );
    }

    return await createOrUpdateProfile(profileToUpdate);
  } catch (e) {
    _setError('Failed to update user role: $e');
    return false;
  } finally {
    _setLoading(false);
  }
}
```

This function is used on the role selection page. Once the user selects a role, either explorer, guide, or equipment provider, the function checks if a user profile exists. If not, it creates a new profile. It then updates the role attribute with the selected role and calls the createOrUpdateProfile function to save the updated profile to Firebase.

Figure 14

completeExplorerOnboarding function

```dart
// Complete explorer onboarding
Future<bool> completeExplorerOnboarding({
  required SkillLevel skillLevel,
  required List<String> preferredActivities,
}) async {
  if (_userProfile == null || _userProfile!.role != UserRole.explorer) {
    _setError('Invalid profile state for explorer onboarding');
    return false;
  }

  // Create explorer profile
  final explorerProfile = ExplorerProfile(
    skillLevel: skillLevel,
    preferredActivities: preferredActivities,
    createdAt: DateTime.now(),
    updatedAt: DateTime.now(),
  );

  final updatedProfile = _userProfile!.copyWith(
    explorerProfile: explorerProfile,
    isOnboardingComplete: true,
    updatedAt: DateTime.now(),
  );

  return await createOrUpdateProfile(updatedProfile);
}
```

Figure 15

completeGuideOnboarding function

```dart
// Complete guide onboarding
Future<bool> completeGuideOnboarding({
  required String experience,
  required List<String> specialties,
  required String location,
}) async {
  if (_userProfile == null || _userProfile!.role != UserRole.guide) {
    _setError('Invalid profile state for guide onboarding');
    return false;
  }

  // Create guide profile
  final guideProfile = GuideProfile(
    experience: experience,
    specialties: specialties,
    location: location,
    rating: 0.0,
    totalGuides: 0,
    createdAt: DateTime.now(),
    updatedAt: DateTime.now(),
  );

  // Store guide-specific data in additionalData for backward compatibility
  final guideData = {
    'experience': experience,
    'specialties': specialties,
    'location': location,
    'rating': 0.0,
    'totalGuides': 0,
  };

  final updatedProfile = _userProfile!.copyWith(
    guideProfile: guideProfile,
    additionalData: {..._userProfile!.additionalData, ...guideData},
    isOnboardingComplete: true,
    updatedAt: DateTime.now(),
  );

  return await createOrUpdateProfile(updatedProfile);
}
```

Figure 16

completeEquipmentProviderOnboarding

```dart
// Complete equipment provider onboarding
Future<bool> completeEquipmentProviderOnboarding({
  required String businessName,
  required List<String> equipmentTypes,
  required String location,
}) async {
  if (_userProfile == null ||
      _userProfile!.role != UserRole.equipmentProvider) {
    _setError('Invalid profile state for equipment provider onboarding');
    return false;
  }

  // Create equipment provider profile
  final equipmentProviderProfile = EquipmentProviderProfile(
    businessName: businessName,
    equipmentTypes: equipmentTypes,
    location: location,
    rating: 0.0,
    totalRentals: 0,
    createdAt: DateTime.now(),
    updatedAt: DateTime.now(),
  );

  // Store equipment provider-specific data in additionalData for backward compatibility
  final equipmentData = {
    'businessName': businessName,
    'equipmentTypes': equipmentTypes,
    'location': location,
    'rating': 0.0,
    'totalRentals': 0,
  };

  final updatedProfile = _userProfile!.copyWith(
    equipmentProviderProfile: equipmentProviderProfile,
    additionalData: {..._userProfile!.additionalData, ...equipmentData},
    isOnboardingComplete: true,
    updatedAt: DateTime.now(),
  );

  return await createOrUpdateProfile(updatedProfile);
}
```

The completeExplorerOnboarding, completeGuideOnboarding, and completeEquipmentProviderOnboarding functions share a common purpose, which is to save the basic user information, such as the business name for equipment providers, to Firebase when the user logs in for the first time. Once the user has filled in all the required information, the application sets the isOnboardingComplete parameter to true, indicating that the onboarding process is complete, and saves this update to Firebase.

Figure 17

Update Profile Functions

```dart
// Update explorer profile
Future<bool> updateExplorerProfile(ExplorerProfile explorerProfile) async {
  if (_userProfile == null || _userProfile!.role != UserRole.explorer) {
    _setError('Invalid profile state for explorer update');
    return false;
  }

  final updatedProfile = _userProfile!.copyWith(
    explorerProfile: explorerProfile.copyWith(updatedAt: DateTime.now()),
    updatedAt: DateTime.now(),
  );

  return await createOrUpdateProfile(updatedProfile);
}

// Update guide profile
Future<bool> updateGuideProfile(GuideProfile guideProfile) async {
  if (_userProfile == null || _userProfile!.role != UserRole.guide) {
    _setError('Invalid profile state for guide update');
    return false;
  }

  // Update additionalData for backward compatibility
  final guideData = guideProfile.toMap();

  final updatedProfile = _userProfile!.copyWith(
    guideProfile: guideProfile.copyWith(updatedAt: DateTime.now()),
    additionalData: {..._userProfile!.additionalData, ...guideData},
    updatedAt: DateTime.now(),
  );

  return await createOrUpdateProfile(updatedProfile);
}

// Update equipment provider profile
Future<bool> updateEquipmentProviderProfile(
  EquipmentProviderProfile equipmentProviderProfile,
) async {
  if (_userProfile == null ||
      _userProfile!.role != UserRole.equipmentProvider) {
    _setError('Invalid profile state for equipment provider update');
    return false;
  }

  // Update additionalData for backward compatibility
  final equipmentData = equipmentProviderProfile.toMap();

  final updatedProfile = _userProfile!.copyWith(
    equipmentProviderProfile: equipmentProviderProfile.copyWith(
      updatedAt: DateTime.now(),
    ),
    additionalData: {..._userProfile!.additionalData, ...equipmentData},
    updatedAt: DateTime.now(),
  );

  return await createOrUpdateProfile(updatedProfile);
}
```

The updateExplorerProfile, updateGuideProfile, and updateEquipmentProviderProfile functions are designed to save the latest updated information to Firebase. Each function receives a specific profile object:

1. ExplorerProfile object for Explorer
2. GuideProfile object for Guide
3. EquipmentProviderProfile object for Equipment Provider

The function then updates the updatedAt parameter with the current timestamp and saves the updated profile to Firebase using the createOrUpdateProfile function.

2.4.2 Watchlist

Figure 18

addToWatchlist Function

```dart
/// Add a place to user's watchlist
static Future<bool> addToWatchlist(Places place) async {
  try {
    final collection = _getUserWatchlistCollection();
    if (collection == null) return false;

    // Convert Places object to a Map for Firestore
    final placeData = {
      'id': place.id ?? '',
      'displayName': place.displayName?.text ?? '',
      'types': place.types ?? [],
      'rating': place.rating ?? 0.0,
      'latitude': place.location?.latitude ?? 0.0,
      'longitude': place.location?.longitude ?? 0.0,
      'photos':
          place.photos
              ?.map(
                (photo) => {
                  'name': photo.name,
                  'widthPx': photo.widthPx,
                  'heightPx': photo.heightPx,
                },
              )
              .toList() ??
          [],
      'addedAt': FieldValue.serverTimestamp(),
    };

    await collection.doc(place.id ?? '').set(placeData);

    // Send push notification when successfully added
    await PushNotificationService.sendWatchlistAddedNotification(place);

    return true;
  } catch (e) {
    print('Error adding to watchlist: $e');
    return false;
  }
}
```

The addToWatchlist function allows users to add items to their watchlist. It receives a Place object, converts it into JSON format, and then saves it as a document in a subcollection under the user's main collection in Firebase.

Figure 19

removeFromWatchlist function

```dart
static Future<bool> removeFromWatchlist(String placeId) async {
  try {
    final collection = _getUserWatchlistCollection();
    if (collection == null) return false;

    // Get the place data before removing for notification
    final doc = await collection.doc(placeId).get();
    Places? place;
    if (doc.exists) {
      final data = doc.data() as Map<String, dynamic>;
      place = Places(
        id: data['id'] ?? '',
        displayName: DisplayName(text: data['displayName'] ?? ''),
        types: List<String>.from(data['types'] ?? []),
        rating: (data['rating'] ?? 0.0).toDouble(),
        location: Location(
          latitude: (data['latitude'] ?? 0.0).toDouble(),
          longitude: (data['longitude'] ?? 0.0).toDouble(),
        ),
      );
    }

    await collection.doc(placeId).delete();

    // Send push notification when successfully removed
    if (place != null) {
      await PushNotificationService.sendWatchlistRemovedNotification(place);
    }

    return true;
  } catch (e) {
    print('Error removing from watchlist: $e');
    return false;
  }
}
```

The removeFromWatchlist function receives the placeId as input and deletes the corresponding document from the user's watchlist subcollection based on the provided placeId.

Figure 20

isInWatchlist function and isInWatchlistStream function

```dart
/// Check if a place is in user's watchlist
static Future<bool> isInWatchlist(String placeId) async {
  try {
    final collection = _getUserWatchlistCollection();
    if (collection == null) return false;

    final doc = await collection.doc(placeId).get();
    return doc.exists;
  } catch (e) {
    print('Error checking watchlist: $e');
    return false;
  }
}

/// Get real-time stream for checking if a specific place is in watchlist
static Stream<bool> isInWatchlistStream(String placeId) {
  final collection = _getUserWatchlistCollection();
  if (collection == null) {
    return Stream.value(false);
  }

  return collection.doc(placeId).snapshots().map((doc) => doc.exists);
}
```

These two functions serve the same purpose: to check whether a place is in the user's watchlist. They do this by checking if the watchlist subcollection contains a document with the given placeId as its ID. If the document is found, the function returns true, indicating that the place is in the watchlist; otherwise, it returns false.

Figure 21

getWatchlist and getWatchlistStream function

```dart
/// Get all places in user's watchlist
static Future<List<Places>> getWatchlist() async {
  try {
    final collection = _getUserWatchlistCollection();
    if (collection == null) return [];

    final querySnapshot =
        await collection.orderBy('addedAt', descending: true).get();

    return querySnapshot.docs.map((doc) {
      final data = doc.data() as Map<String, dynamic>;

      // Convert Firestore data back to Places object
      return Places(
        id: data['id'] ?? '',
        displayName: DisplayName(text: data['displayName'] ?? ''),
        types: List<String>.from(data['types'] ?? []),
        rating: (data['rating'] ?? 0.0).toDouble(),
        location: Location(
          latitude: (data['latitude'] ?? 0.0).toDouble(),
          longitude: (data['longitude'] ?? 0.0).toDouble(),
        ),
        photos:
            (data['photos'] as List<dynamic>? ?? []).map((photoData) {
              final photoMap = photoData as Map<String, dynamic>;
              return Photos(
                name: photoMap['name'] ?? '',
                widthPx: photoMap['widthPx'] ?? 0,
                heightPx: photoMap['heightPx'] ?? 0,
              );
            }).toList(),
      );
    }).toList();
  } catch (e) {
    print('Error getting watchlist: $e');
    return [];
  }
}

/// Get watchlist stream for real-time updates
static Stream<List<Places>>? getWatchlistStream() {
  final collection = _getUserWatchlistCollection();
  if (collection == null) return null;

  return collection.orderBy('addedAt', descending: true).snapshots().map((
    snapshot,
  ) {
    return snapshot.docs.map((doc) {
      final data = doc.data() as Map<String, dynamic>;

      return Places(
        id: data['id'] ?? '',
        displayName: DisplayName(text: data['displayName'] ?? ''),
        types: List<String>.from(data['types'] ?? []),
        rating: (data['rating'] ?? 0.0).toDouble(),
        location: Location(
          latitude: (data['latitude'] ?? 0.0).toDouble(),
          longitude: (data['longitude'] ?? 0.0).toDouble(),
        ),
        photos:
            (data['photos'] as List<dynamic>? ?? []).map((photoData) {
              final photoMap = photoData as Map<String, dynamic>;
              return Photos(
                name: photoMap['name'] ?? '',
                widthPx: photoMap['widthPx'] ?? 0,
                heightPx: photoMap['heightPx'] ?? 0,
              );
            }).toList(),
      );
    }).toList();
  });
}
```

These two functions are designed for the view layer to render all the watchlist items saved by the user. They begin by fetching the watchlist subcollection corresponding to the user. Then, each document in the subcollection is converted into a Place object and returned to the view layer for display.

Figure 22

toggleWatchlist function

```dart
/// Toggle watchlist status (add if not exists, remove if exists)
static Future<bool> toggleWatchlist(Places place) async {
  final placeId = place.id ?? '';
  if (placeId.isEmpty) return false;

  final isCurrentlyInWatchlist = await isInWatchlist(placeId);

  if (isCurrentlyInWatchlist) {
    return await removeFromWatchlist(placeId);
  } else {
    return await addToWatchlist(place);
  }
}
```

This function is triggered when the user clicks on the watchlist icon in the view layer. Once clicked, the corresponding data (a Place object) is passed to this function as input. It first checks whether the place is already in the watchlist. If the place is found, it is removed using the removeFromWatchlist function; otherwise, it is added to the watchlist using the addToWatchlist function.

2.4.3 Emergency SOS

Figure 23

listenToUserLocation functuon

```dart
void _listenToUserLocations() {
  if (widget.trip == null) return;

  _locationsStreamSubscription = _database
      .child('trips')
      .child(widget.trip!.id)
      .child('locations')
      .onValue
      .listen((DatabaseEvent event) {
        if (event.snapshot.value != null) {
          final data = Map<String, dynamic>.from(event.snapshot.value as Map);
          final Map<String, UserLocation> newLocations = {};

          data.forEach((userId, locationData) {
            if (locationData != null) {
              newLocations[userId] = UserLocation.fromJson(
                userId,
                Map<dynamic, dynamic>.from(locationData as Map),
              );
            }
          });

          setState(() {
            _userLocations = newLocations;
          });

          _checkForEmergencies();
          _updateMapBounds();
        }
      });
}
```

This function will listen in real-time to location updates from users within a specific trip using Firebase Realtime Database. When new location data is received, it updates the internal state to reflect these changes and performs additional actions like checking for emergency and updating the map view.

Figure 24

updateUserLocation

```dart
void _updateUserLocation(Position position) {
  if (widget.trip == null || _currentUserId == null) return;

  final locationData = {
    'userName': _currentUserName ?? 'Unknown User',
    'latitude': position.latitude,
    'longitude': position.longitude,
    'lastUpdated': DateTime.now().millisecondsSinceEpoch,
    'isOnline': true,
    'isEmergency': _userLocations[_currentUserId]?.isEmergency ?? false,
  };

  _database
      .child('trips')
      .child(widget.trip!.id)
      .child('locations')
      .child(_currentUserId!)
      .set(locationData);
}
```

This function updates the user's current position along with additional information, such as the last updated time, online status, and emergency status, to the Realtime Database in Firebase.

Figure 25

setPresenceOnline and setPresenceOffline function

```dart
static void setPresenceOnline() {
  final uid = _auth.currentUser?.uid;
  if (uid == null) return;

  final userStatusDatabaseRef = _db.ref("status/$uid");

  _db.ref(".info/connected").onValue.listen((event) {
    final connected = event.snapshot.value as bool? ?? false;
    if (connected) {
      userStatusDatabaseRef
          .onDisconnect()
          .set({"state": "offline", "last_changed": ServerValue.timestamp})
          .then((_) {
            userStatusDatabaseRef.set({
              "state": "online",
              "last_changed": ServerValue.timestamp,
            });
          });
    }
  });
}

static void setPresenceOffline() {
  final uid = _auth.currentUser?.uid;
  if (uid == null) return;

  _db.ref("status/$uid").set({
    "state": "offline",
    "last_changed": ServerValue.timestamp,
  });
}
```

The setPresenceOnline function updates the user's presence status in the Realtime Database under the status/{uid} path. The setPresenceOffline function updates the user's status when they disconnect unexpectedly. This allows other users to see who is currently online or offline in real time.

2.4.4 Calendar

Figure 26

saveUnavailableDates

```dart
static Future<bool> saveUnavailableDates(
  Set<DateTime> unavailableDates,
) async {
  try {
    final user = _auth.currentUser;
    if (user == null) {
      throw Exception('No user signed in');
    }

    final userId = user.uid;

    // Convert DateTime to Timestamp for Firestore storage
    final timestamps =
        unavailableDates.map((date) {
          // Normalize to start of day to ensure consistency
          final normalizedDate = DateTime(date.year, date.month, date.day);
          return Timestamp.fromDate(normalizedDate);
        }).toList();

    // Save to user_profiles collection under the unavailable_dates field
    await _firestore.collection('user_profiles').doc(userId).update({
      'unavailable_dates': timestamps,
      'updatedAt': DateTime.now().millisecondsSinceEpoch,
    });

    return true;
  } catch (e) {
    // If the field doesn't exist, create it using set with merge
    try {
      final user = _auth.currentUser;
      if (user == null) {
        throw Exception('No user signed in');
      }

      final userId = user.uid;

      // Convert DateTime to Timestamp for Firestore storage
      final timestamps =
          unavailableDates.map((date) {
            // Normalize to start of day to ensure consistency
            final normalizedDate = DateTime(date.year, date.month, date.day);
            return Timestamp.fromDate(normalizedDate);
          }).toList();

      await _firestore.collection('user_profiles').doc(userId).set({
        'unavailable_dates': timestamps,
        'updatedAt': FieldValue.serverTimestamp(),
      }, SetOptions(merge: true));

      return true;
    } catch (e) {
      print('Error saving unavailable dates: $e');
      return false;
    }
  }
}
```

This function allows guides to add their unavailable dates to Firebase, ensuring that explorers cannot book them during those periods. It starts by converting the input DateTime values into Firebase Timestamp format. Once all dates are converted, they are saved under the unavailable_dates field in the guide's document within the user_profiles collection.

Figure 27

loadUnavailableDates function

```dart
static Future<Set<DateTime>> loadUnavailableDates(String? guideId) async {
  try {
    final targetUserId = guideId ?? _auth.currentUser?.uid;
    if (targetUserId == null) {
      throw Exception('No user ID provided');
    }

    final doc =
        await _firestore.collection('user_profiles').doc(targetUserId).get();

    if (doc.exists && doc.data() != null) {
      final data = doc.data()!;
      final unavailableDatesData = data['unavailable_dates'];

      if (unavailableDatesData != null && unavailableDatesData is List) {
        return unavailableDatesData
            .cast<Timestamp>()
            .map((timestamp) => timestamp.toDate())
            .toSet();
      }
    }

    return <DateTime>{};
  } catch (e) {
    print('Error loading unavailable dates: $e');
    return <DateTime>{};
  }
}
```

This function receives a guideId as input and uses it to fetch the corresponding document from the user_profiles collection. If the document is found, it retrieves the unavailable_dates field and converts the list of timestamps into DateTime format.

Figure 28

isDateUnavailable function

```dart
static Future<bool> isDateUnavailable(DateTime date, String? guideId) async {
  try {
    final unavailableDates = await loadUnavailableDates(guideId);
    final normalizedDate = DateTime(date.year, date.month, date.day);

    return unavailableDates.any((unavailableDate) {
      final normalizedUnavailable = DateTime(
        unavailableDate.year,
        unavailableDate.month,
        unavailableDate.day,
      );
      return normalizedUnavailable.isAtSameMomentAs(normalizedDate);
    });
  } catch (e) {
    print('Error checking date availability: $e');
    return false; // Default to available if there's an error
  }
}
```

This function checks whether the selected date for a specific guide is unavailable. It begins by fetching all unavailable dates for the guide using the loadUnavailableDates function. Then, it compares the input date with the fetched dates. If a match is found, it returns true, indicating the guide is unavailable on that date; otherwise, it returns false.

Figure 29

getAvailableDatesInRange function

```dart
static Future<List<DateTime>> getAvailableDatesInRange({
  required DateTime startDate,
  required DateTime endDate,
  String? guideId,
}) async {
  try {
    final unavailableDates = await loadUnavailableDates(guideId);
    final availableDates = <DateTime>[];

    for (
      DateTime date = startDate;
      date.isBefore(endDate) || date.isAtSameMomentAs(endDate);
      date = date.add(const Duration(days: 1))
    ) {
      final normalizedDate = DateTime(date.year, date.month, date.day);
      final isUnavailable = unavailableDates.any((unavailableDate) {
        final normalizedUnavailable = DateTime(
          unavailableDate.year,
          unavailableDate.month,
          unavailableDate.day,
        );
        return normalizedUnavailable.isAtSameMomentAs(normalizedDate);
      });

      if (!isUnavailable) {
        availableDates.add(normalizedDate);
      }
    }

    return availableDates;
  } catch (e) {
    print('Error getting available dates in range: $e');
    return [];
  }
}
```

This function is designed to return a list of available dates within a specified range for a guide, by filtering out the unavailable dates hat are already stored in Firebase.

3.4.5 Guide

Figure 30

getAvailableGuides function

```dart
Future<List<UserProfile>> getAvailableGuides(DateTime date) async {
  try {
    _setLoading(true);
    _setError(null);

    final QuerySnapshot querySnapshot =
        await _firestore
            .collection('user_profiles')
            .where('role', isEqualTo: 'guide')
            .where('isOnboardingComplete', isEqualTo: true)
            .get();

    final List<UserProfile> allGuides =
        querySnapshot.docs
            .map(
              (doc) =>
                  UserProfile.fromMap(doc.data() as Map<String, dynamic>),
            )
            .where((profile) => profile.guideProfile != null)
            .toList();

    // Check availability for each guide
    final List<UserProfile> availableGuides = [];

    for (final guide in allGuides) {
      final isAvailable = await BookingAvailabilityHelper.isGuideAvailable(
        guideId: guide.uid,
        date: date,
      );
      if (isAvailable) {
        availableGuides.add(guide);
      }
    }

    // Sort by rating
    availableGuides.sort(
      (a, b) => b.guideProfile!.rating.compareTo(a.guideProfile!.rating),
    );

    _availableGuides = availableGuides;
    _setLoading(false);
    return availableGuides;
  } catch (e) {
    _setError('Failed to fetch guides: $e');
    _setLoading(false);
    throw Exception('Failed to fetch guides: $e');
  }
}
```

This function is designed to retrieve all available guides from Firebase. It first fetches all guides from the database, checks which guides are currently available, and filters them from the original list. The resulting list of available guides is then sorted based on their ratings in descending order.

Figure 31

getGuideStatisticsStream function

```dart
/// Stream version for real-time updates
static Stream<GuideStatistics> getGuideStatisticsStream() {
  final user = _auth.currentUser;
  if (user == null) {
    return Stream.value(
      GuideStatistics(
        totalEvents: 0,
        totalBookings: 0,
        pendingBookingRequests: 0,
        cancellationRequests: 0,
      ),
    );
  }

  final currentUserId = user.uid;

  // Use real-time listener on collection group for immediate updates
  return _firestore
      .collectionGroup('bookings')
      .where('guideId', isEqualTo: currentUserId)
      .snapshots()
      .asyncMap((snapshot) async {
        try {
          final allBookingsForGuide = <BookingRequest>[];
          for (final doc in snapshot.docs) {
            try {
              final booking = BookingRequest.fromJson(doc.data());
              allBookingsForGuide.add(booking);
            } catch (e) {
              print('Error parsing booking document ${doc.id}: $e');
              print('Document data: ${doc.data()}');
              // Skip this document and continue with others
            }
          }

          // Calculate statistics
          final totalEvents =
              allBookingsForGuide
                  .where(
                    (booking) => booking.status == BookingStatus.confirmed,
                  )
                  .length;

          final totalBookings =
              allBookingsForGuide.length; // All booking requests

          final pendingBookings =
              allBookingsForGuide
                  .where((booking) => booking.status == BookingStatus.pending)
                  .length;

          final cancellationRequests =
              allBookingsForGuide
                  .where(
                    (booking) =>
                        booking.status == BookingStatus.cancelled ||
                        booking.status == BookingStatus.withdraw,
                  )
                  .length;

          // Check for new pending bookings based on timestamps
          final pendingBookingsList =
              allBookingsForGuide
                  .where((booking) => booking.status == BookingStatus.pending)
                  .map((booking) => booking.toJson())
                  .toList();

          // Check for new cancelled events based on timestamps
          final cancelledBookingsList =
              allBookingsForGuide
                  .where(
                    (booking) =>
                        booking.status == BookingStatus.cancelled ||
                        booking.status == BookingStatus.withdraw,
                  )
                  .map((booking) => booking.toJson())
                  .toList();

          // Update badge status based on new bookings
          if (GuideBadgeService.hasNewPendingRequests(pendingBookingsList)) {
            await GuideBadgeService.resetPendingRequestsViewed();
          }

          if (GuideBadgeService.hasNewCancelledEvents(
            cancelledBookingsList,
          )) {
            await GuideBadgeService.resetCancelledEventsViewed();
          }

          return GuideStatistics(
            totalEvents: totalEvents,
            totalBookings: totalBookings,
            pendingBookingRequests: pendingBookings,
            cancellationRequests: cancellationRequests,
          );
        } catch (e) {
          print('Error in statistics stream: $e');
          return GuideStatistics(
            totalEvents: 0,
            totalBookings: 0,
            pendingBookingRequests: 0,
            cancellationRequests: 0,
          );
        }
      });
}
```

This function is used to get the statistical of the bookings relate to the guide. It will retrieve all the bookings corresponds to the guide, and then based on the status to analyze. For total events, it will check how many confirmed bookings the guide has, whereas for booking, it will simply check all the bookings for the guide. For the pending booking requests, it will check

Figure 32

getConfirmedBookings function

```dart
/// Get all confirmed bookings for the current guide
static Future<List<BookingRequest>> getConfirmedBookings() async {
  try {
    final user = _auth.currentUser;
    if (user == null) {
      throw Exception('User not authenticated');
    }

    final currentUserId = user.uid;
    print('Fetching confirmed bookings for guide $currentUserId');

    // Try to get confirmed bookings from Firestore
    // If index errors occur, we'll fall back to mock data for development
    try {
      print(
        'Querying Firestore for guide: $currentUserId with status: confirmed',
      );

      // First try: Get all bookings for the guide and filter in memory (no index needed)
      final allBookingsSnapshot =
          await FirebaseFirestore.instance
              .collectionGroup('bookings')
              .where('guideId', isEqualTo: currentUserId)
              .get();

      print(
        'Found ${allBookingsSnapshot.docs.length} total bookings for guide',
      );

      final confirmedBookings = <BookingRequest>[];
      for (final doc in allBookingsSnapshot.docs) {
        try {
          final data = doc.data();
          print('Processing booking ${doc.id}: status=${data['status']}');

          // Filter for confirmed bookings in memory to avoid compound index
          if (data['status'] == 'confirmed' ||
              data['status'] ==
                  BookingStatus.confirmed.toString().split('.').last) {
            final booking = BookingRequest.fromJson({'id': doc.id, ...data});
            confirmedBookings.add(booking);
            print(
              'Added confirmed booking: ${booking.id} for place: ${booking.place.displayName?.text}',
            );
          }
        } catch (e) {
          print('Error parsing booking document ${doc.id}: $e');
          print('Document data: ${doc.data()}');
        }
      }

      confirmedBookings.sort((a, b) => a.startDate.compareTo(b.startDate));
      print(
        'Found ${confirmedBookings.length} confirmed bookings from database for guide $currentUserId',
      );
      return confirmedBookings;
    } catch (firestoreError) {
      print('Firestore query failed (likely missing index): $firestoreError');

      // Try alternative query without status filter first to see what data exists
      try {
        print(
          'Trying alternative query - getting all bookings for guide to debug',
        );
        final debugSnapshot =
            await FirebaseFirestore.instance
                .collectionGroup('bookings')
                .where('guideId', isEqualTo: currentUserId)
                .limit(5)
                .get();

        print(
          'Debug query found ${debugSnapshot.docs.length} total bookings for guide',
        );
        for (final doc in debugSnapshot.docs) {
          final data = doc.data();
          print(
            'Booking ${doc.id}: status=${data['status']}, guideId=${data['guideId']}',
          );
        }
      } catch (debugError) {
        print('Debug query also failed: $debugError');
      }

      print('Falling back to mock data for development...');
      return [];
    }
  } catch (e) {
    print('Error getting confirmed bookings: $e');
    return []; // Return empty list instead of throwing
  }
}
```

This function is similar to the getGuideStatisticsStream function but is specifically designed to return all confirmed booking requests for a guide. As with the previous implementation, it first retrieves the data from Firebase, filters the bookings with a status of “confirmed”, and then sorts the filtered list based on the start date of the events in ascending order.

3.4.6 Review

Figure 33

getEquipmentReviews and getEquipmentReview function

```dart
static Stream<List<EquipmentReview>> getEquipmentReviews(String equipmentId) {
  return FirebaseFirestore.instance
      .collection('equipment_reviews')
      .where('equipmentItem.id', isEqualTo: equipmentId)
      .snapshots()
      .map(
        (snapshot) =>
            snapshot.docs
                .map((doc) => EquipmentReview.fromJson(doc.data()))
                .toList(),
      );
}

static Stream<EquipmentReview?> getEquipmentReview(
  String equipmentId,
  String userId,
) {
  return FirebaseFirestore.instance
      .collection('equipment_reviews')
      .where('equipmentTransaction.id', isEqualTo: equipmentId)
      .where('userId', isEqualTo: userId)
      .snapshots()
      .map((snapshot) {
        if (snapshot.docs.isEmpty) return null;
        return EquipmentReview.fromJson(snapshot.docs.first.data());
      });
}
```

These two functions are similar but serve different purposes. The getEquipmentReviews function takes an equipment ID as a parameter and sets up a stream listener to monitor changes in the equipment_reviewscollection, filtering by documents where the equipmentItem.id matches the given equipment ID.

On the other hand, the getEquipmentReview function is designed to check whether a specific explorer has submitted a review for a particular equipment transaction. It takes the equipment transaction ID and user ID as input, searches the equipment_reviews collection for a matching document, and returns the corresponding EquipmentReview object if found; otherwise, it returns null.

Figure 34

getEquipmentReviewsByTransaction function

```
static Stream<List<EquipmentReview>?> getEquipmentReviewsByTransaction(
  String equipmentId,
) {
  return FirebaseFirestore.instance
      .collection('equipment_reviews')
      .where('equipmentItem.id', isEqualTo: equipmentId)
      .snapshots()
      .map(
        (snapshot) =>
            snapshot.docs
                .map((doc) => EquipmentReview.fromJson(doc.data()))
                .toList(),
      );
}
```

The getEquipmentReviewsByTransaction will listen to real-time updates from the Firestore collection equipment_reviews for a specific equipment ID. It returns a stream of List<EquipmentReview> objects.

Figure 35

submitEquipmentReview function

```dart
static Future<bool> submitEquipmentReview({
  required EquipmentTransaction equipmentTransaction,
  required EquipmentItem equipmentItem,
  required String userId,
  required String email,
  required double rating,
  String? comment,
}) async {
  final review = EquipmentReview(
    id: '${equipmentTransaction.id}_$userId',
    equipmentTransaction: equipmentTransaction,
    userId: userId,
    email: email,
    equipmentItem: equipmentItem,
    rating: rating,
    comment: comment,
    createdAt: DateTime.now(),
  );

  await FirebaseFirestore.instance
      .collection('equipment_reviews')
      .add(review.toJson());

  return true;
}
```

This function enables the explorer to submit a review for a rented or purchased equipment item. It will collect relevant information first and pack it inside the EquipmentReview object. Then, the object will be converted to JSON format and save as a new document in the equipment_reviews Firestore collection.

Figure 36

getTripReviews function

```
static Stream<List<TripReview>> getTripReviews(String tripId, String userId) {
  return FirebaseFirestore.instance
      .collection('trip_reviews')
      .where('tripId', isEqualTo: tripId)
      .where('userId', isEqualTo: userId)
      .snapshots()
      .map(
        (snapshot) =>
            snapshot.docs
                .map((doc) => TripReview.fromJson(doc.data()))
                .toList(),
      );
}
```

The getTripReviews function is designed to retrieve all the reviews submitted by a specific user for a specific trip in real time from Firestore. It begins by checking the trip_reviews collection and identifying the documents where the trip ID matches the inputted trip ID and the user ID matches the inputted user ID. Then, all the corresponding documents are converted into TripReview objects and returned.

Figure 37

submitReview function

```dart
static Future<TripReview> submitReview({
  required Trip trip,
  required String userId,
  required String userName,
  required String difficulty,
  required int guideRating,
  String? comment,
}) async {
  final review = TripReview(
    tripId: trip.id,
    userId: userId,
    userName: userName,
    placeId: trip.place.id ?? '',
    placeName: trip.place.displayName?.text ?? '',
    difficulty: difficulty,
    guideRating: guideRating,
    createdAt: DateTime.now(),
    guideId: trip.guideId,
    comment: comment,
  );

  await FirebaseFirestore.instance
      .collection('trip_reviews')
      .add(review.toJson());

  return review;
}
```

The submitReview function is designed to create and submit a new review for a trip to Firestore. It will receive trip details, user information, difficulty rating, and an optional comment. Then, it will construct a TripReview object with the relevant data and add it to the trip_reviews collection in Firestore.

Figure 38

hasReviewedTrip function

```dart
static Future<bool> hasReviewedTrip(String tripId, String userId) async {
  final snapshot =
      await FirebaseFirestore.instance
          .collection('trip_reviews')
          .where('tripId', isEqualTo: tripId)
          .where('userId', isEqualTo: userId)
          .get();
  return snapshot.docs.isNotEmpty;
}

```

This function will check whether a specific user has already submitted a review for a particular trip. It will query the trip_reviews collection in Firestore using both the tripId and userId. If a matching document is found, it will return true; otherwise, it will return false. This function helps to prevent duplication reviews from the same user for the same trip.

Figure 39

getReviewsByGuideId function

```
static Stream<List<TripReview>> getReviewsByGuideId(String guideId) {
  return FirebaseFirestore.instance
      .collection('trip_reviews')
      .where('guideId', isEqualTo: guideId)
      .snapshots()
      .map(
        (snapshot) =>
            snapshot.docs
                .map((doc) => TripReview.fromJson(doc.data()))
                .toList(),
      );
}
```

The getReviewsByGuideId function will retrieve all the trip reviews associated with a specific guide in real time from Firestore. It will query the trip_reviews collection and filter documents where the guideId matches the input guideId. The matching documents are then converted into TripReview objects and returned as a stream of lists.

Figure 40

getAverageGuideRating function

```dart
static Future<Map<String, dynamic>> getAverageGuideRating(
  String guideId,
) async {
  final snapshot =
      await FirebaseFirestore.instance
          .collection('trip_reviews')
          .where('guideId', isEqualTo: guideId)
          .get();

  if (snapshot.docs.isEmpty) {
    return {'average': 0.0, 'count': 0};
  }

  double totalRating = 0.0;
  int count = snapshot.docs.length;

  for (var doc in snapshot.docs) {
    final review = TripReview.fromJson(doc.data());
    totalRating += review.guideRating;
  }

  double average = totalRating / count;

  return {'average': average, 'count': count};
}
```

This function is designed to calculate the average rating and total number of reviews for a specifc guide. It will query the trip_reviews collection in Firestore and filter the documents where the guideId matches the input guideId. If no matching reviews are found, it returns an average of 0.0 and a count of 0. If there is any matching review, the function will sum up all the guideRating values from the matched documents, calculate the average, and return both the average rating and the count in a map.

3.4.7 Places

Figure 41

searchNearbyPlaces function

```dart
static Future<List<Places>> searchNearbyPlaces({
  required double latitude,
  required double longitude,
  double radiusInMeters = 50000,
  List<String>? includedTypes,
  List<String>? excludedTypes,
  int maxResults = 20,
  String languageCode = 'en',
}) async {
  try {
    final headers = {
      'Content-Type': 'application/json',
      'X-Goog-Api-Key': ConstKey.googlePlacesApiKey,
      'X-Goog-FieldMask':
          'places.displayName,places.formattedAddress,places.location,places.types,places.rating,places.id,places.photos',
    };

    final requestBody = {
      "locationRestriction": {
        "circle": {
          "center": {"latitude": latitude, "longitude": longitude},
          "radius": radiusInMeters,
        },
      },
      "includedTypes": includedTypes ?? _defaultIncludedTypes,
      "excludedTypes": excludedTypes ?? _defaultExcludedTypes,
      "rankPreference": "DISTANCE",
      "languageCode": languageCode,
      "maxResultCount": maxResults,
    };

    final request = http.Request('POST', Uri.parse(_baseUrl));
    request.body = json.encode(requestBody);
    request.headers.addAll(headers);

    final response = await request.send();
    final responseBody = await response.stream.bytesToString();

    if (response.statusCode == 200) {
      final data = json.decode(responseBody);
      return _parsePlacesFromResponse(data);
    } else {
      throw Exception(
        'Failed to fetch places: ${response.statusCode} - ${response.reasonPhrase}',
      );
    }
  } catch (e) {
    throw Exception('Error searching nearby places: $e');
  }
}
```

The searchNearbyPlaces function is designed to fetch a list of nearby places using the Google Places API based on the given latitude and longitude. It will send a POST request to the API with configurable parameters such as search radius, types of places to include or exclude, maximum result count, and preferred language.

Figure 42

getImageUrl function

```dart
static String getImageUrl(Places place) {
  if (place.photos != null && place.photos!.isNotEmpty) {
    final photo = place.photos!.first;
    if (photo.name != null) {
      return 'https://places.googleapis.com/v1/${photo.name}/media?maxHeightPx=400&maxWidthPx=400&key=${ConstKey.googlePlacesApiKey}';
    }
  }
  // Return null or empty string instead of placeholder URL
  return '';
}
```

The getImageUrl function will extract and construct a direct image URL for a given Places object, using the photo metadata returned from the Google Places API.

Figure 43

searchPlacesByText function

```dart
static Future<List<Places>> searchPlacesByText({
  required String query,
  required double latitude,
  required double longitude,
  double radiusInMeters = 50000,
  int maxResults = 20,
  String languageCode = 'en',
}) async {
  try {
    const textSearchUrl =
        'https://places.googleapis.com/v1/places:searchText';

    final headers = {
      'Content-Type': 'application/json',
      'X-Goog-Api-Key': ConstKey.googlePlacesApiKey,
      'X-Goog-FieldMask':
          'places.displayName,places.formattedAddress,places.location,places.types,places.rating,places.id,places.photos',
    };

    final requestBody = {
      'textQuery': query,
      'locationBias': {
        'circle': {
          'center': {'latitude': latitude, 'longitude': longitude},
          'radius': radiusInMeters,
        },
      },
      'maxResultCount': maxResults,
      'languageCode': languageCode,
    };

    print('Searching places by text: $query');
    final response = await http.post(
      Uri.parse(textSearchUrl),
      headers: headers,
      body: jsonEncode(requestBody),
    );

    print('Text search response status: ${response.statusCode}');

    if (response.statusCode == 200) {
      final Map<String, dynamic> data = jsonDecode(response.body);
      final List<dynamic> placesData = data['places'] ?? [];

      final places =
          placesData.map((placeData) => Places.fromJson(placeData)).toList();

      // Client-side filtering to ensure only our allowed types are returned
      final filteredPlaces =
          places.where((place) {
            if (place.types == null || place.types!.isEmpty) return false;
            return place.types!.any(
              (type) => _defaultIncludedTypes.contains(type),
            );
          }).toList();

      print(
        'Found ${filteredPlaces.length} filtered places for text query: $query',
      );
      return filteredPlaces;
    } else {
      print(
        'Error in text search: ${response.statusCode} - ${response.body}',
      );
      throw Exception(
        'Failed to search places by text: ${response.statusCode}',
      );
    }
  } catch (e) {
    print('Exception in searchPlacesByText: $e');
    throw Exception('Failed to search places by text: $e');
  }
}
```

The searchPlacesByText function enables users to search for places by inputting the name of a place. This function utilizes the Google Places Text Search API and then applies client-side filtering to retain only those results that match specific allowed types. It begins by sending a POST request to the Text Search API with the user's query and location bias. Then, it filters out the places that do not match the allowed types.

Figure 44

getReviewByPlaceId and getReviewCountByPlaceId function

```dart
static Future<List<TripReview>?> getReviewByPlaceId(String placeId) async {
  final snapshot =
      await FirebaseFirestore.instance
          .collection('trip_reviews')
          .where('placeId', isEqualTo: placeId)
          .get();

  if (snapshot.docs.isNotEmpty) {
    return snapshot.docs
        .map((doc) => TripReview.fromJson(doc.data()))
        .toList();
  }
  return null;
}

static Future<String> getReviewCountByPlaceId(String placeId) async {
  List<TripReview>? reviews = await getReviewByPlaceId(placeId);

  if (reviews == null || reviews.isEmpty) {
    return "Neutral";
  }

  Map<String, int> difficultyCount = {
    'Easy': 0,
    'Intermediate': 0,
    'Hard': 0,
  };

  for (var review in reviews) {
    String difficulty = review.difficulty;
    if (difficultyCount.containsKey(difficulty)) {
      difficultyCount[difficulty] = difficultyCount[difficulty]! + 1;
    }
  }

  String mostCommon = 'Neutral';
  int maxCount = 0;

  difficultyCount.forEach((key, value) {
    if (value > maxCount) {
      maxCount = value;
      mostCommon = key;
    }
  });

  print('Most common difficulty for place $placeId: $mostCommon');
  return mostCommon;
}
```

The getReviewByPlaceId function is designed to retrieve all the trip reviews from the Firestore that are associated with a specific place, identified by its placeId. It returns a list of TripReview objects if reviews exist, or null if none are found.

Whereas for the getReviewCountByPlaceId function, it is used to analyze the difficulty level of these reviews. It calculates how many reviews are labeled as “Easy”, “Intermediate” or “Hard”, then returns the most frequently mentioned difficulty level for that place. If no reviews are found, it returns “Neutral” by default.

3.4.8 Chat

Figure 45

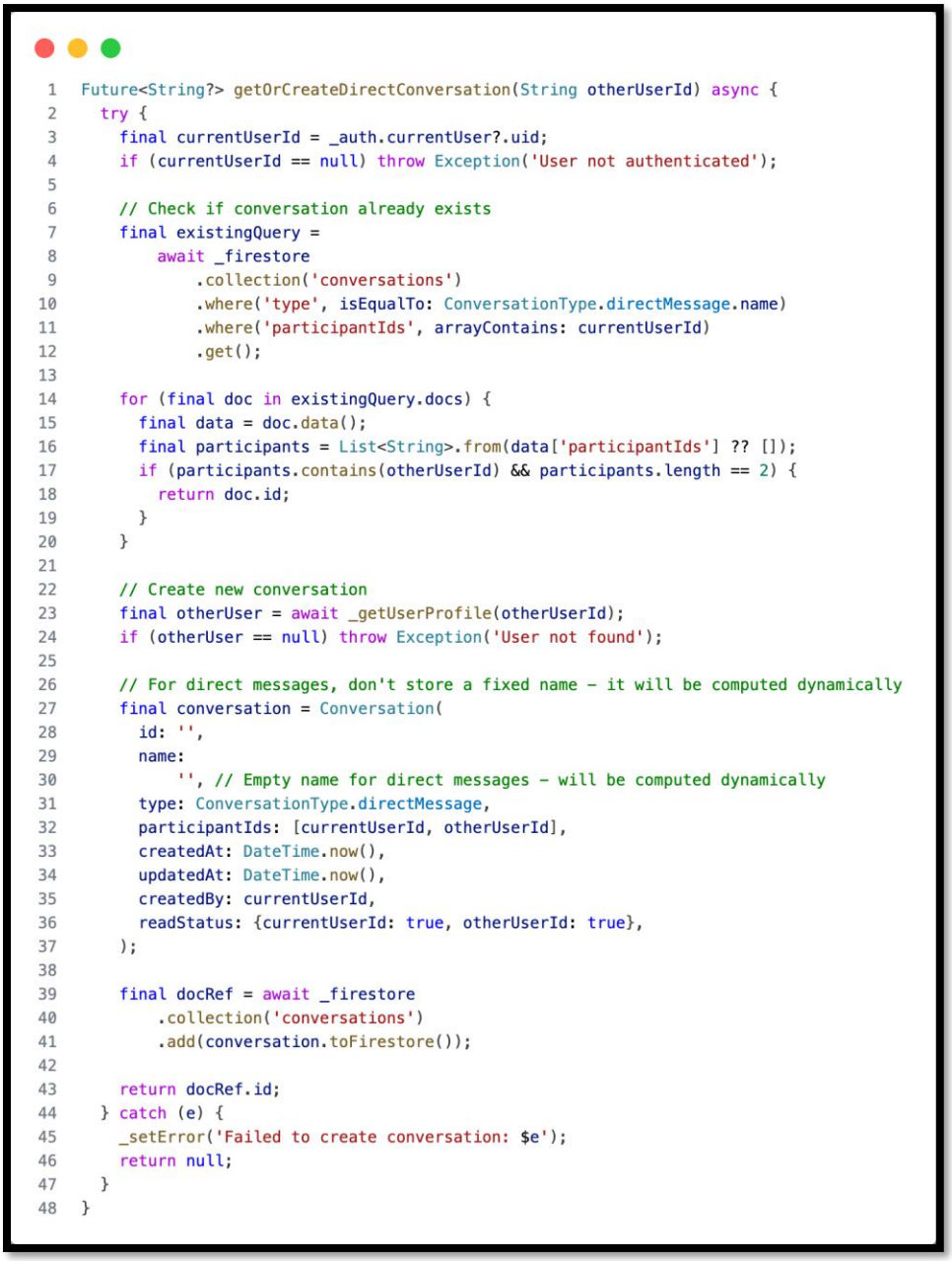
searchUsers function

```dart
Future<List<ChatUser>> searchUsers(String query) async {
  if (query.trim().isEmpty) {
    _searchResults = [];
    notifyListeners();
    return _searchResults;
  }

  try {
    _setLoading(true);
    _setError(null);

    final currentUserId = _auth.currentUser?.uid;
    if (currentUserId == null) {
      throw Exception('User not authenticated');
    }

    final lowercaseQuery = query.toLowerCase();

    // Search by email
    final emailQuery =
        await _firestore
            .collection('user_profiles')
            .where('email', isGreaterThanOrEqualTo: lowercaseQuery)
            .where('email', isLessThan: lowercaseQuery + '\uf8ff')
            .limit(20)
            .get();

    // Search by display name
    final nameQuery =
        await _firestore
            .collection('user_profiles')
            .where('displayName', isGreaterThanOrEqualTo: lowercaseQuery)
            .where('displayName', isLessThan: lowercaseQuery + '\uf8ff')
            .limit(20)
            .get();

    final users = <ChatUser>[];
    final seenIds = <String>{};

    // Process email search results
    for (final doc in emailQuery.docs) {
      final data = doc.data();
      if (data['uid'] != currentUserId && !seenIds.contains(data['uid'])) {
        users.add(ChatUser.fromUserProfile(data));
        seenIds.add(data['uid']);
      }
    }

    // Process name search results
    for (final doc in nameQuery.docs) {
      final data = doc.data();
      if (data['uid'] != currentUserId && !seenIds.contains(data['uid'])) {
        users.add(ChatUser.fromUserProfile(data));
        seenIds.add(data['uid']);
      }
    }

    // Sort by relevance (exact matches first)
    users.sort((a, b) {
      final aEmail = a.email.toLowerCase();
      final bEmail = b.email.toLowerCase();
      final aName = (a.displayName ?? '').toLowerCase();
      final bName = (b.displayName ?? '').toLowerCase();

      // Exact email match gets highest priority
      if (aEmail == lowercaseQuery) return -1;
      if (bEmail == lowercaseQuery) return 1;

      // Exact name match gets second priority
      if (aName == lowercaseQuery) return -1;
      if (bName == lowercaseQuery) return 1;

      // Email starts with query gets third priority
      if (aEmail.startsWith(lowercaseQuery) &&
          !bEmail.startsWith(lowercaseQuery))
        return -1;
      if (bEmail.startsWith(lowercaseQuery) &&
          !aEmail.startsWith(lowercaseQuery))
        return 1;

      // Name starts with query gets fourth priority
      if (aName.startsWith(lowercaseQuery) &&
          !bName.startsWith(lowercaseQuery))
        return -1;
      if (bName.startsWith(lowercaseQuery) &&
          !aName.startsWith(lowercaseQuery))
        return 1;

      // Finally, sort alphabetically
      return aEmail.compareTo(bEmail);
    });

    _searchResults = users;
    _setLoading(false);
    notifyListeners();
    return users;
  } catch (e) {
    _setError('Failed to search users: $e');
    _setLoading(false);
    return [];
  }
}
```

This function enables users to search for other users in the chat section by inputting either a name or an email address. It queries the Firebase Firestore for matches and returns a list of users whose names or emails contain the input text. The results are sorted first by relevance and then alphabetically for better user experience.

Figure 46

getOrCreateDirectConversation function



```dart
Future<String?> getOrCreateDirectConversation(String otherUserId) async {
  try {
    final currentUserId = _auth.currentUser?.uid;
    if (currentUserId == null) throw Exception('User not authenticated');

    // Check if conversation already exists
    final existingQuery =
        await _firestore
            .collection('conversations')
            .where('type', isEqualTo: ConversationType.directMessage.name)
            .where('participantIds', arrayContains: currentUserId)
            .get();

    for (final doc in existingQuery.docs) {
      final data = doc.data();
      final participants = List<String>.from(data['participantIds'] ?? []);
      if (participants.contains(otherUserId) && participants.length == 2) {
        return doc.id;
      }
    }

    // Create new conversation
    final otherUser = await _getUserProfile(otherUserId);
    if (otherUser == null) throw Exception('User not found');

    // For direct messages, don't store a fixed name - it will be computed dynamically
    final conversation = Conversation(
      id: '',
      name:
          '', // Empty name for direct messages - will be computed dynamically
      type: ConversationType.directMessage,
      participantIds: [currentUserId, otherUserId],
      createdAt: DateTime.now(),
      updatedAt: DateTime.now(),
      createdBy: currentUserId,
      readStatus: {currentUserId: true, otherUserId: true},
    );

    final docRef = await _firestore
        .collection('conversations')
        .add(conversation.toFirestore());

    return docRef.id;
  } catch (e) {
    _setError('Failed to create conversation: $e');
    return null;
  }
}
```

This function allows users to get or create a direct conversation with another user. It starts by taking the other user's ID and checks the conversations collection in Firestore to see if a conversation already exists between the two users. If a conversation is found, the existing document ID is returned. If not, a new conversation document is created in Firestore, and its ID is returned. This ensures that only one unique conversation exists between any two users.

Figure 47

createGroupChat function

```dart
Future<String?> createGroupChat({
  required String name,
  required List<String> participantIds,
  String? description,
}) async {
  try {
    final currentUserId = _auth.currentUser?.uid;
    if (currentUserId == null) throw Exception('User not authenticated');

    // Add current user to participants if not already included
    final allParticipants =
        <String>{...participantIds, currentUserId}.toList();

    if (allParticipants.length < 2) {
      throw Exception('Group chat must have at least 2 participants');
    }

    final readStatus = <String, bool>{};
    for (final participantId in allParticipants) {
      readStatus[participantId] = participantId == currentUserId;
    }

    final conversation = Conversation(
      id: '',
      name: name.trim(),
      type: ConversationType.groupChat,
      participantIds: allParticipants,
      createdAt: DateTime.now(),
      updatedAt: DateTime.now(),
      createdBy: currentUserId,
      description: description?.trim(),
      readStatus: readStatus,
    );

    final docRef = await _firestore
        .collection('conversations')
        .add(conversation.toFirestore());

    // Send system message about group creation
    await sendMessage(
      conversationId: docRef.id,
      content:
          '${_auth.currentUser?.displayName ?? _auth.currentUser?.email} created the group',
      type: MessageType.text,
      isSystemMessage: true,
    );

    return docRef.id;
  } catch (e) {
    _setError('Failed to create group chat: $e');
    return null;
  }
}
```

The createGroupChat function is designed to create a new group chat conversation in the system. It accepts a group name, a list of participant user IDs and an optional description. Firstly, this function will ensure that current user is authenticated and includes them in the group if not already listed. Then, it will validate that the group has at least 2 participants. After that, these data will be saved to Firestore, and the system will send a system message announcing the group creation.

Figure 48

sendMessage function

```dart
Future<bool> sendMessage({
  required String conversationId,
  required String content,
  required MessageType type,
  String? replyToMessageId,
  String? fileUrl,
  String? fileName,
  int? fileSize,
  bool isSystemMessage = false,
}) async {
  try {
    final currentUser = _auth.currentUser;

    if (currentUser == null) {
      throw Exception('User not authenticated');
    }

    if (content.trim().isEmpty && type == MessageType.text) {
      throw Exception('Message content cannot be empty');
    }

    // Get conversation to validate user is a participant
    final conversationDoc =
        await _firestore
            .collection('conversations')
            .doc(conversationId)
            .get();

    if (!conversationDoc.exists) {
      throw Exception('Conversation not found');
    }

    final conversationData = conversationDoc.data()!;
    final participantIds = List<String>.from(
      conversationData['participantIds'] ?? [],
    );

    if (!isSystemMessage && !participantIds.contains(currentUser.uid)) {
      throw Exception('User is not a participant in this conversation');
    }

    // Create read status (only sender has read the message initially)
    final readBy = <String, bool>{};
    for (final participantId in participantIds) {
      readBy[participantId] =
          participantId == currentUser.uid || isSystemMessage;
    }

    final message = Message(
      id: '',
      conversationId: conversationId,
      senderId: isSystemMessage ? 'system' : currentUser.uid,
      senderName:
          isSystemMessage
              ? 'System'
              : (currentUser.displayName ?? currentUser.email ?? 'User'),
      content: content.trim(),
      type: type,
      timestamp: DateTime.now(),
      readBy: readBy,
      replyToMessageId: replyToMessageId,
      fileUrl: fileUrl,
      fileName: fileName,
      fileSize: fileSize,
    );

    // Add message to subcollection
    await _firestore
        .collection('conversations')
        .doc(conversationId)
        .collection('messages')
        .add(message.toFirestore());

    // Update conversation with last message info
    final updateData = {
      'lastMessage': content.trim(),
      'lastMessageAt': Timestamp.fromDate(DateTime.now()),
      'updatedAt': Timestamp.fromDate(DateTime.now()),
    };

    // Update read status for conversation (mark as unread for others)
    if (!isSystemMessage) {
      final conversationReadStatus = <String, bool>{};
      for (final participantId in participantIds) {
        conversationReadStatus[participantId] =
            participantId == currentUser.uid;
      }
      updateData['readStatus'] = conversationReadStatus;
    }

    await _firestore
        .collection('conversations')
        .doc(conversationId)
        .update(updateData);

    return true;
  } catch (e) {
    _setError('Failed to send message: $e');
    return false;
  }
}
```

This function is responsible to send a message in a conversation. It will check if the user is authenticated and part of the conversation. Then, the function will create the message, save it to Firestore, and update the conversation's latest message and read status.

Figure 49

getMessages function

```dart
Stream<List<Message>> getMessages(String conversationId) {
  return _firestore
      .collection('conversations')
      .doc(conversationId)
      .collection('messages')
      .orderBy('timestamp', descending: false)
      .snapshots()
      .map((snapshot) {
        return snapshot.docs
            .map((doc) => Message.fromFirestore(doc))
            .toList();
      });
}
```

This function will listen to real-time updates from the message subcollection of a specific document in the conversations collection. This function will return a stream of messages ordered by timestamp.

Figure 50

getUserConversations function

```dart
/// Get conversations for current user
Stream<List<Conversation>> getUserConversations() {
  final currentUserId = _auth.currentUser?.uid;
  if (currentUserId == null) {
    return Stream.value([]);
  }

  return _firestore
      .collection('conversations')
      .where('participantIds', arrayContains: currentUserId)
      .where('isActive', isEqualTo: true)
      .orderBy('updatedAt', descending: true)
      .snapshots()
      .map((snapshot) {
        final conversations =
            snapshot.docs
                .map((doc) => Conversation.fromFirestore(doc))
                .toList();
        _conversations = conversations;
        return conversations;
      });
}
```

This function will listen to real-time updates from the conversation collection, filtering for active conversations where the current user is participant.

Figure 51

deleteConversation function

```dart
Future<bool> deleteConversation(String conversationId) async {
  try {
    final currentUserId = _auth.currentUser?.uid;
    if (currentUserId == null) throw Exception('User not authenticated');

    // For now, just mark as inactive for the current user
    // In a full implementation, you might want to handle this differently
    await _firestore.collection('conversations').doc(conversationId).update({
      'isActive': false,
      'updatedAt': Timestamp.fromDate(DateTime.now()),
    });

    return true;
  } catch (e) {
    _setError('Failed to delete conversation: $e');
    return false;
  }
}
```

The deleteConversation function sets the isActive field of a conversation to false, indicating that the conversation has been deleted.

Figure 52

leaveGroupChat function

```dart
Future<bool> leaveGroupChat(String conversationId) async {
  try {
    final currentUserId = _auth.currentUser?.uid;
    if (currentUserId == null) throw Exception('User not authenticated');

    final conversationRef = _firestore
        .collection('conversations')
        .doc(conversationId);
    final conversationDoc = await conversationRef.get();

    if (!conversationDoc.exists) {
      throw Exception('Conversation not found');
    }

    final data = conversationDoc.data()!;
    final participantIds = List<String>.from(data['participantIds'] ?? []);

    if (!participantIds.contains(currentUserId)) {
      throw Exception('User is not a participant in this conversation');
    }

    // Remove user from participants
    participantIds.remove(currentUserId);

    // Send system message
    await sendMessage(
      conversationId: conversationId,
      content:
          '${_auth.currentUser?.displayName ?? _auth.currentUser?.email} left the group',
      type: MessageType.text,
      isSystemMessage: true,
    );

    // Update conversation
    await conversationRef.update({
      'participantIds': participantIds,
      'updatedAt': Timestamp.fromDate(DateTime.now()),
    });

    return true;
  } catch (e) {
    _setError('Failed to leave group chat: $e');
    return false;
  }
}
```

The leaveGroupChat function enables current user to leave a group chat. It removes their ID from the participantsIds list in the conversations collection, sends a system message to the group chat, and updates the conversation metadata.

2.4.9 Wallet

Figure 53

initializeWallet function

```dart
Future<bool> initializeWallet() async {
  try {
    _setLoading(true);
    _setError(null);

    final user = _auth.currentUser;
    if (user == null) {
      _setError('No user signed in');
      return false;
    }

    // Check if wallet exists
    final walletDoc =
        await _firestore
            .collection(WalletConstants.walletsCollection)
            .doc(user.uid)
            .get();

    if (!walletDoc.exists) {
      // Create new wallet
      return await _createWallet(user.uid);
    } else {
      // Load existing wallet
      _wallet = WalletModel.fromFirestore(walletDoc);
      _startWalletListener(user.uid);
      return true;
    }
  } catch (e) {
    _setError('Failed to initialize wallet: $e');
    return false;
  } finally {
    _setLoading(false);
  }
}
```

This function initializes the user's wallet module each time the app is opened. It will check whether the wallet already exists in Firestore. If it exists, it sets up a real-time listener; if not, it will call the createWallet function to generate a new wallet.

Figure 54

createWallet and startWalletListener function

```dart
/// Create a new wallet for user
Future<bool> _createWallet(String userId) async {
  try {
    final now = DateTime.now();
    final newWallet = WalletModel(
      userId: userId,
      balance: 0.0,
      currency: WalletConstants.defaultCurrency,
      createdAt: now,
      updatedAt: now,
    );

    await _firestore
        .collection(WalletConstants.walletsCollection)
        .doc(userId)
        .set(newWallet.toFirestore());

    _wallet = newWallet;
    _startWalletListener(userId);

    debugPrint('Wallet created successfully for user: $userId');
    return true;
  } catch (e) {
    _setError('Failed to create wallet: $e');
    return false;
  }
}

/// Start listening to wallet changes
void _startWalletListener(String userId) {
  _walletSubscription?.cancel();
  _walletSubscription = _firestore
      .collection(WalletConstants.walletsCollection)
      .doc(userId)
      .snapshots()
      .listen(
        (snapshot) {
          if (snapshot.exists) {
            _wallet = WalletModel.fromFirestore(snapshot);
            notifyListeners();
          }
        },
        onError: (error) {
          _setError('Wallet sync error: $error');
        },
      );
}
```

The createWallet function will create a new wallet for the suer in Firestore with the default values, such as balance 0. After creation, it will start a real-time listener using startWalletListener function to keep the local wallet state in sync with Firestore updates.

Figure 55

topUpWallet function

```dart
Future<bool> topUpWallet({
  required double amount,
  required PaymentMethod paymentMethod,
  String? reference,
  Map<String, dynamic>? metadata,
}) async {
  try {
    _setLoading(true);
    _setError(null);

    final user = _auth.currentUser;
    if (user == null || _wallet == null) {
      _setError('No wallet found');
      return false;
    }

    // Validate amount
    if (amount < WalletConstants.minTopUpAmount) {
      _setError(
        'Minimum top-up amount is MYR ${WalletConstants.minTopUpAmount}',
      );
      return false;
    }

    if (amount > WalletConstants.maxTopUpAmount) {
      _setError(
        'Maximum top-up amount is MYR ${WalletConstants.maxTopUpAmount}',
      );
      return false;
    }

    // Check if new balance would exceed limit
    final newBalance = _wallet!.balance + amount;
    if (newBalance > WalletConstants.maxWalletBalance) {
      _setError(
        'Wallet balance cannot exceed MYR ${WalletConstants.maxWalletBalance}',
      );
      return false;
    }

    // Create transaction
    final transaction = Transaction(
      id: _generateTransactionId(),
      walletId: user.uid,
      type: TransactionType.topUp,
      amount: amount,
      currency: _wallet!.currency,
      timestamp: DateTime.now(),
      description: 'Wallet Top-up',
      status: TransactionStatus.processing,
      reference: reference,
      paymentMethod: paymentMethod,
      metadata: metadata ?? {},
    );

    // Create completed transaction
    final completedTransaction = Transaction(
      id: transaction.id,
      walletId: transaction.walletId,
      type: transaction.type,
      amount: transaction.amount,
      currency: transaction.currency,
      timestamp: transaction.timestamp,
      description: transaction.description,
      status: TransactionStatus.completed,
      reference: transaction.reference,
      merchantId: transaction.merchantId,
      merchantName: transaction.merchantName,
      paymentMethod: transaction.paymentMethod,
      metadata: transaction.metadata,
    );

    // Update wallet balance and add transaction
    final updatedTransactions = [
      ...?_wallet!.transactions,
      completedTransaction,
    ];
    final updatedWallet = _wallet!.copyWith(
      balance: newBalance,
      transactions: updatedTransactions,
      updatedAt: DateTime.now(),
    );

    await _firestore
        .collection(WalletConstants.walletsCollection)
        .doc(user.uid)
        .update(updatedWallet.toFirestore());

    debugPrint('Top-up successful: MYR $amount');
    return true;
  } catch (e) {
    _setError('Top-up failed: $e');
    return false;
  } finally {
    _setLoading(false);
  }
}

```

The topUpWallet function enables the user to top up their wallet. Firstly, the function will validate if the user is authenticated and the wallet exists. Then, it will handle the wallet top ups by validating the amount, creating a completed transaction, updating the wallet balance, and saving the changes to Firestore.

Figure 56

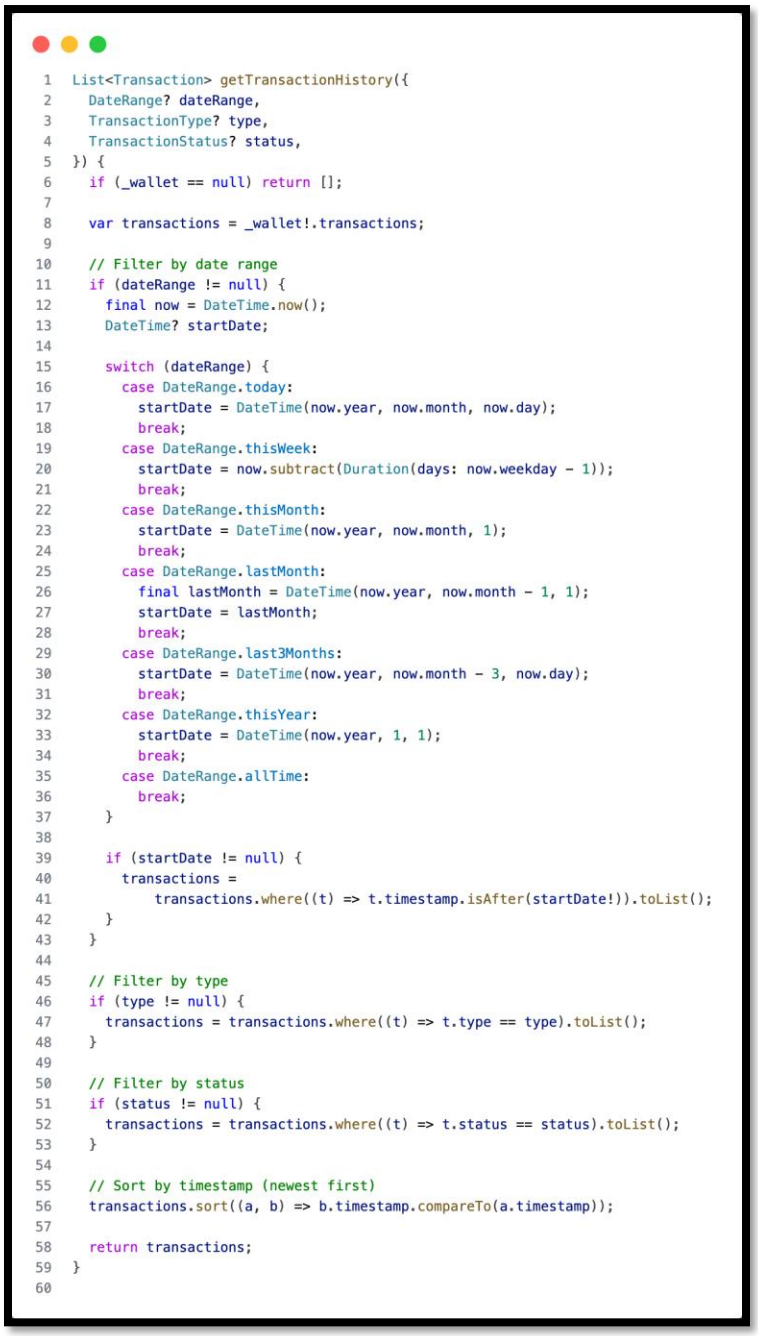
withdrawFromWallet

```dart
Future<bool> withdrawFromWallet({
  required double amount,
  String? reference,
  Map<String, dynamic>? metadata,
}) async {
  try {
    _setLoading(true);
    _setError(null);

    final user = _auth.currentUser;
    if (user == null || _wallet == null) {
      _setError('No wallet found');
      return false;
    }

    // Validate amount
    if (amount < WalletConstants.minWithdrawAmount) {
      _setError(
        'Minimum withdrawal amount is MYR ${WalletConstants.minWithdrawAmount}',
      );
      return false;
    }

    if (amount > WalletConstants.maxWithdrawAmount) {
      _setError(
        'Maximum withdrawal amount is MYR ${WalletConstants.maxWithdrawAmount}',
      );
      return false;
    }

    // Check if there is enough balance for withdrawal
    final currentBalance = _wallet!.balance;
    if (currentBalance < amount) {
      _setError('Insufficient balance for withdrawal');
      return false;
    }

    // Create transaction
    final transaction = Transaction(
      id: _generateTransactionId(),
      walletId: user.uid,
      type: TransactionType.withdrawal,
      amount: amount,
      currency: _wallet!.currency,
      timestamp: DateTime.now(),
      description: 'Wallet Withdrawal',
      status: TransactionStatus.processing,
      reference: reference,
      metadata: metadata ?? {},
    );

    // Create completed transaction
    final completedTransaction = Transaction(
      id: transaction.id,
      walletId: transaction.walletId,
      type: transaction.type,
      amount: transaction.amount,
      currency: transaction.currency,
      timestamp: transaction.timestamp,
      description: transaction.description,
      status: TransactionStatus.completed,
      reference: transaction.reference,
      merchantId: transaction.merchantId,
      merchantName: transaction.merchantName,
      paymentMethod: transaction.paymentMethod,
      metadata: transaction.metadata,
    );

    // Update wallet balance and add transaction
    final newBalance = currentBalance - amount;
    final updatedTransactions = [
      ..._wallet!.transactions,
      completedTransaction,
    ];
    final updatedWallet = _wallet!.copyWith(
      balance: newBalance,
      transactions: updatedTransactions,
      updatedAt: DateTime.now(),
    );

    await _firestore
        .collection(WalletConstants.walletsCollection)
        .doc(user.uid)
        .update(updatedWallet.toFirestore());

    debugPrint('Withdrawal successful: MYR $amount');
    return true;
  } catch (e) {
    _setError('Withdrawal failed: $e');
    return false;
  } finally {
    _setLoading(false);
  }
}
```

The withhdrawFromWallet is similar to the topUpWallet function. It processes a wallet withdrawal by validating the user, amount, and balance. If these fields are valid, the function will create a completed transaction, and update the wallet balance, and save the changes to Firebase.

Figure 57

getTransactionHistory function



```dart
List<Transaction> getTransactionHistory({
  DateRange? dateRange,
  TransactionType? type,
  TransactionStatus? status,
}) {
  if (_wallet == null) return [];

  var transactions = _wallet!.transactions;

  // Filter by date range
  if (dateRange != null) {
    final now = DateTime.now();
    DateTime? startDate;

    switch (dateRange) {
      case DateRange.today:
        startDate = DateTime(now.year, now.month, now.day);
        break;
      case DateRange.thisWeek:
        startDate = now.subtract(Duration(days: now.weekday - 1));
        break;
      case DateRange.thisMonth:
        startDate = DateTime(now.year, now.month, 1);
        break;
      case DateRange.lastMonth:
        final lastMonth = DateTime(now.year, now.month - 1, 1);
        startDate = lastMonth;
        break;
      case DateRange.last3Months:
        startDate = DateTime(now.year, now.month - 3, now.day);
        break;
      case DateRange.thisYear:
        startDate = DateTime(now.year, 1, 1);
        break;
      case DateRange.allTime:
        break;
    }

    if (startDate != null) {
      transactions =
          transactions.where((t) => t.timestamp.isAfter(startDate!)).toList();
    }
  }

  // Filter by type
  if (type != null) {
    transactions = transactions.where((t) => t.type == type).toList();
  }

  // Filter by status
  if (status != null) {
    transactions = transactions.where((t) => t.status == status).toList();
  }

  // Sort by timestamp (newest first)
  transactions.sort((a, b) => b.timestamp.compareTo(a.timestamp));

  return transactions;
}

```

The getTransactionHistory function is designed to retrieve the user's wallet transactions. It will calculate the appropriate start date based on the selected date range, filter the transaction accordingly, and sort the result in descending order by timestamp to return the most recent transactions first.

Figure 58

transferToAnotherWallet function

```dart
Future<bool> transferToAnotherWallet({
  required String toUserId,
  required double amount,
  required String note,
}) async {
  try {
    _setLoading(true);
    _setError(null);

    final fromUser = _auth.currentUser;
    if (fromUser == null || _wallet == null) {
      _setError('No wallet found');
      return false;
    }

    if (fromUser.uid == toUserId) {
      _setError('Cannot transfer to your own wallet');
      return false;
    }

    if (amount <= 0) {
      _setError('Amount must be greater than 0');
      return false;
    }

    if (_wallet!.balance < amount) {
      _setError('Insufficient balance');
      return false;
    }

    final toWalletDoc =
        await _firestore
            .collection(WalletConstants.walletsCollection)
            .doc(toUserId)
            .get();

    if (!toWalletDoc.exists) {
      _setError('Recipient wallet not found');
      return false;
    }

    final toWallet = WalletModel.fromFirestore(toWalletDoc);
    final now = DateTime.now();
    final transferId = _generateTransactionId();

    // Create transactions for both users
    final senderTransaction = Transaction(
      id: transferId,
      walletId: fromUser.uid,
      type: TransactionType.payment,
      amount: amount,
      currency: _wallet!.currency,
      timestamp: now,
      description: note,
      status: TransactionStatus.completed,
      reference: transferId,
      metadata: {'to': toUserId, 'note': note},
    );

    final receiverTransaction = Transaction(
      id: transferId,
      walletId: toUserId,
      type: TransactionType.topUp,
      amount: amount,
      currency: toWallet.currency,
      timestamp: now,
      description: note,
      status: TransactionStatus.completed,
      reference: transferId,
      metadata: {'from': fromUser.uid, 'note': note},
    );

    // Perform atomic update using Firestore transaction
    await _firestore.runTransaction((txn) async {
      final fromRef = _firestore
          .collection(WalletConstants.walletsCollection)
          .doc(fromUser.uid);
      final toRef = _firestore
          .collection(WalletConstants.walletsCollection)
          .doc(toUserId);

      // Update sender
      final newSenderBalance = _wallet!.balance - amount;
      final updatedSender = _wallet!.copyWith(
        balance: newSenderBalance,
        updatedAt: now,
        transactions: [..._wallet!.transactions, senderTransaction],
      );
      txn.update(fromRef, updatedSender.toFirestore());

      // Update receiver
      final newReceiverBalance = toWallet.balance + amount;
      final updatedReceiver = toWallet.copyWith(
        balance: newReceiverBalance,
        updatedAt: now,
        transactions: [...toWallet.transactions, receiverTransaction],
      );
      txn.update(toRef, updatedReceiver.toFirestore());
    });

    debugPrint('Transfer successful: MYR $amount to $toUserId');
    return true;
  } catch (e) {
    _setError('Transfer failed: $e');
    return false;
  } finally {
    _setLoading(false);
  }
}
```

This transferToAnotherWallet function enables the users to send fund to other users. This function will validate user identity, wallet existence, balance, and recipient before sending the fund. If these fields are valid, the function will create sender and receiver transactions and updates both wallets atomically in Firestore.

2.4.10 Voucher

Figure 59

isVoucherExists function

```dart
static Future<bool> isVoucherExists(String userId, String voucherId) async {
  final docRef = _firestore
      .collection('users')
      .doc(userId)
      .collection('vouchers')
      .doc(voucherId);

  final docSnapshot = await docRef.get();
  return docSnapshot.exists;
}
```

The function is designed to check if a voucher exists. It receives two parameters: the userId of the equipment provider and the voucherId. It then searches for the relevant document based on the given parameters. If the document is found, it returns true, indicating the voucher exists for that equipment provider; otherwise, it returns false.

Figure 60

saveVoucherToUser function

```dart
static Future<void> saveVoucherToUser(
  String userId,
  VoucherModel voucher,
) async {
  final docRef = _firestore
      .collection('users')
      .doc(userId)
      .collection('vouchers')
      .doc(voucher.id);

  await docRef.set(voucher.toJson());
}
```

This function is responsible for saving a voucher in the vouchers subcollection of the corresponding user. It will receive two parameters: userId and voucher. It creates a documents using the voucher ID as the filename, converts the voucher data to JSON format, and stores it in the document.

Figure 61

getUserVouchers function

```dart
static Stream<List<VoucherModel>> getUserVouchers(String userId) {
  return _firestore
      .collection('users')
      .doc(userId)
      .collection('vouchers')
      .snapshots()
      .map(
        (snapshot) =>
            snapshot.docs
                .map((doc) => VoucherModel.fromJson(doc.data()))
                .toList(),
      );
}
```

This function is designed for equipment providers. It takes the input userId, retrieves all vouchers created by that user, and returns the as a list of voucher model objects.

Figure 62

deleteUserVoucher function

```dart
static Future<void> deleteUserVoucher(String userId, String voucherId) async {
  final docRef = _firestore
      .collection('users')
      .doc(userId)
      .collection('vouchers')
      .doc(voucherId);

  await docRef.delete();
}
```

This function is responsible for deleting vouchers. it uses the input voucherId and userId to locate the specific document and deletes it from the Firebase.

Figure 63

validateVoucherCode function

```dart
static Future<VoucherModel?> validateVoucherCode(
  String userId,
  String code,
) async {
  final querySnapshot =
      await _firestore
          .collection('users')
          .doc(userId)
          .collection('vouchers')
          .where('code', isEqualTo: code.toUpperCase())
          .limit(1)
          .get();

  if (querySnapshot.docs.isEmpty) {
    return null; // Voucher not found
  }

  final voucherData = querySnapshot.docs.first.data();
  final voucher = VoucherModel.fromJson(voucherData);

  // Check if voucher is expired
  if (voucher.expiryDate.isBefore(DateTime.now())) {
    return null; // Voucher expired
  }

  return voucher;
}
```

This function will validate a voucher code for a given userId. It searches the user's voucher subcollection for matching code, returning the voucher if found and not expired. If the voucher does not exist or is expired, it returns null.

2.4.11 Equipment

Figure 64

getEquipmentStream function

```
static Stream<List<EquipmentItem>> getEquipmentStream(String providerId) {
  return FirebaseFirestore.instance
      .collection('equipment')
      .where('providerId', isEqualTo: providerId)
      .snapshots()
      .map((snapshot) {
        return snapshot.docs.map((doc) {
          return EquipmentItem.fromJson(doc.data(), doc.id);
        }).toList();
      });
}
```

This function is designed for the equipment provider to fetch all equipment belonging to them. It receives the provider ID as input and searches for the corresponding documents in the equipment collection, where the attribute providerId matches the given provider ID.

Figure 65

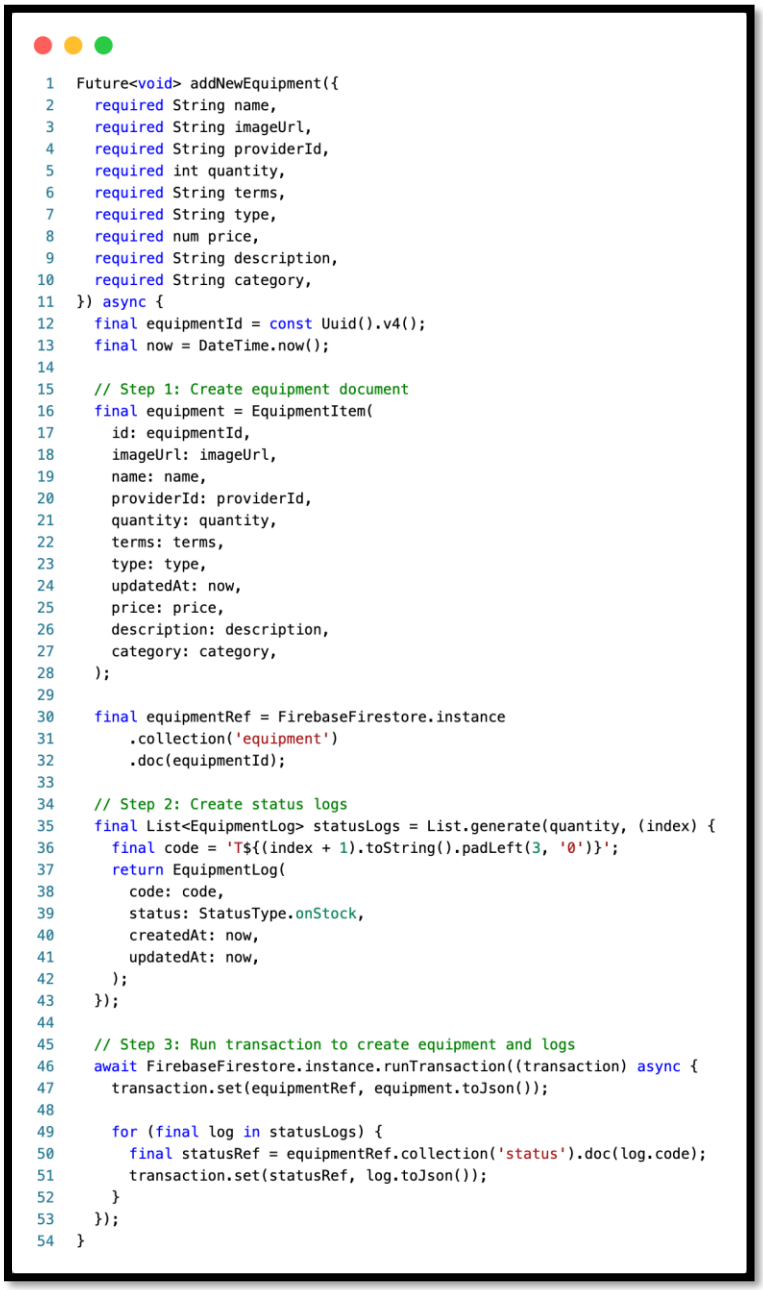
fetchAllEquipment, getEquipmentByProvider and getALlEquipment function

```dart
static Future<List<EquipmentItem>> fetchAllEquipment() async {
  final snapshot = await _firestore.collection('equipment').get();

  return snapshot.docs.map((doc) {
    return EquipmentItem.fromJson(doc.data(), doc.id);
  }).toList();
}

static Stream<List<EquipmentItem>?> getEquipmentByProvider(
  String providerId,
) {
  return _firestore
      .collection('equipment')
      .where('providerId', isEqualTo: providerId)
      .snapshots()
      .map((snapshot) {
        return snapshot.docs
            .map((doc) => EquipmentItem.fromJson(doc.data(), doc.id))
            .toList();
      });
}

static Stream<List<EquipmentItem>> getAllEquipment() {
  final snapshot = _firestore.collection('equipment').snapshots();

  return snapshot.map((querySnapshot) {
    return querySnapshot.docs.map((doc) {
      return EquipmentItem.fromJson(doc.data(), doc.id);
    }).toList();
  });
}
```

These functions can be split into two main categories. The getEquipmentByProvider function is similar to the getEquipmentStream, as both return all equipment items that belong to the specific equipment provider. On the other hand, the fetchAllEquipment and getAllEquipment functions are used to retrieve all equipment from Firebase, without filtering by any provider.

Figure 66

addNewEquipment function



```dart
Future<void> addNewEquipment({
  required String name,
  required String imageUrl,
  required String providerId,
  required int quantity,
  required String terms,
  required String type,
  required num price,
  required String description,
  required String category,
}) async {
  final equipmentId = const Uuid().v4();
  final now = DateTime.now();

  // Step 1: Create equipment document
  final equipment = EquipmentItem(
    id: equipmentId,
    imageUrl: imageUrl,
    name: name,
    providerId: providerId,
    quantity: quantity,
    terms: terms,
    type: type,
    updatedAt: now,
    price: price,
    description: description,
    category: category,
  );

  final equipmentRef = FirebaseFirestore.instance
      .collection('equipment')
      .doc(equipmentId);

  // Step 2: Create status logs
  final List<EquipmentLog> statusLogs = List.generate(quantity, (index) {
    final code = 'T${(index + 1).toString().padLeft(3, '0')}';
    return EquipmentLog(
      code: code,
      status: StatusType.onStock,
      createdAt: now,
      updatedAt: now,
    );
  });

  // Step 3: Run transaction to create equipment and logs
  await FirebaseFirestore.instance.runTransaction((transaction) async {
    transaction.set(equipmentRef, equipment.toJson());

    for (final log in statusLogs) {
      final statusRef = equipmentRef.collection('status').doc(log.code);
      transaction.set(statusRef, log.toJson());
    }
  });
}
```

This function is designed for the equipment provider to add new equipment to Firebase. It begins by receiving the relevant information, packs them into an EquipmentItem object, and then saves the object to Firebase.

Figure 67

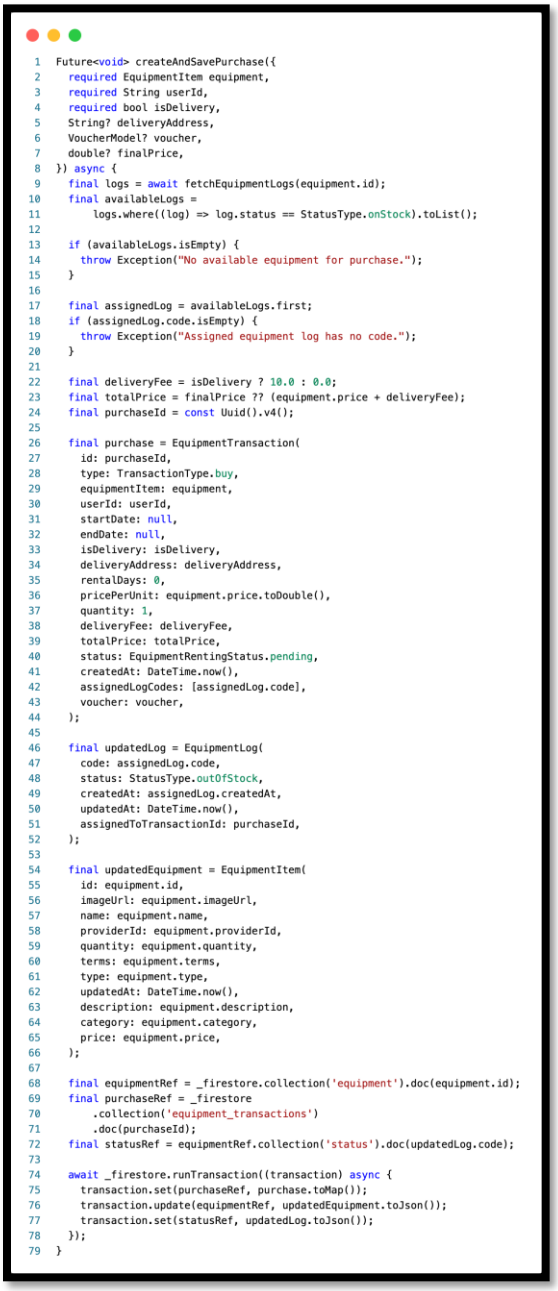
createAndSaveBooking function

```dart
Future<void> createAndSaveBooking({
  required EquipmentItem equipment,
  required String userId,
  required DateTime startDate,
  required DateTime endDate,
  required bool isDelivery,
  String? deliveryAddress,
  VoucherModel? voucher,
  double? finalPrice,
}) async {
  final logs = await fetchEquipmentLogs(equipment.id);
  final availableLogs =
      logs.where((log) => log.status == StatusType.onStock).toList();

  if (availableLogs.isEmpty) {
    throw Exception("No available equipment at the moment.");
  }

  final assignedLog = availableLogs.first;
  if (assignedLog.code.isEmpty) {
    throw Exception("Assigned equipment log has no code.");
  }

  final rentalDays = endDate.difference(startDate).inDays + 1;
  final deliveryFee = isDelivery ? 10.0 : 0.0;
  final totalPrice =
      finalPrice ?? ((equipment.price * rentalDays) + deliveryFee);
  final bookingId = const Uuid().v4();

  final booking = EquipmentTransaction(
    id: bookingId,
    type: TransactionType.rent,
    equipmentItem: equipment,
    userId: userId,
    startDate: startDate,
    endDate: endDate,
    isDelivery: isDelivery,
    deliveryAddress: deliveryAddress,
    rentalDays: rentalDays,
    pricePerUnit: equipment.price.toDouble(),
    quantity: 1,
    deliveryFee: deliveryFee,
    totalPrice: totalPrice,
    status: EquipmentRentingStatus.pending,
    createdAt: DateTime.now(),
    assignedLogCodes: [assignedLog.code],
    voucher: voucher,
  );

  final updatedLog = EquipmentLog(
    code: assignedLog.code,
    status: StatusType.rentedOut,
    createdAt: assignedLog.createdAt,
    updatedAt: DateTime.now(),
    assignedToTransactionId: bookingId,
  );

  final updatedEquipment = EquipmentItem(
    id: equipment.id,
    imageUrl: equipment.imageUrl,
    name: equipment.name,
    providerId: equipment.providerId,
    quantity: equipment.quantity,
    terms: equipment.terms,
    type: equipment.type,
    updatedAt: DateTime.now(),
    description: equipment.description,
    category: equipment.category,
    price: equipment.price,
  );

  final equipmentRef = _firestore.collection('equipment').doc(equipment.id);
  final bookingRef = _firestore
      .collection('equipment_transactions')
      .doc(bookingId);
  final statusRef = equipmentRef.collection('status').doc(updatedLog.code);

  await _firestore.runTransaction((transaction) async {
    transaction.set(bookingRef, booking.toMap());
    transaction.update(equipmentRef, updatedEquipment.toJson());
    transaction.set(statusRef, updatedLog.toJson());
  });
}
```

This function will be called when the explorer books a rental equipment. It receives the relevant information, packs it inside an EquipmentTransaction object, and saves it to Firebase. At the same time, it will update the equipment log by changing the first available equipment status to “rented” and save the updated log to Firebase.

Figure 68

createAndSavePurchase function



```dart
Future<void> createAndSavePurchase({
  required EquipmentItem equipment,
  required String userId,
  required bool isDelivery,
  String? deliveryAddress,
  VoucherModel? voucher,
  double? finalPrice,
}) async {
  final logs = await fetchEquipmentLogs(equipment.id);
  final availableLogs =
      logs.where((log) => log.status == StatusType.onStock).toList();

  if (availableLogs.isEmpty) {
    throw Exception("No available equipment for purchase.");
  }

  final assignedLog = availableLogs.first;
  if (assignedLog.code.isEmpty) {
    throw Exception("Assigned equipment log has no code.");
  }

  final deliveryFee = isDelivery ? 10.0 : 0.0;
  final totalPrice = finalPrice ?? (equipment.price + deliveryFee);
  final purchaseId = const Uuid().v4();

  final purchase = EquipmentTransaction(
    id: purchaseId,
    type: TransactionType.buy,
    equipmentItem: equipment,
    userId: userId,
    startDate: null,
    endDate: null,
    isDelivery: isDelivery,
    deliveryAddress: deliveryAddress,
    rentalDays: 0,
    pricePerUnit: equipment.price.toDouble(),
    quantity: 1,
    deliveryFee: deliveryFee,
    totalPrice: totalPrice,
    status: EquipmentRentingStatus.pending,
    createdAt: DateTime.now(),
    assignedLogCodes: [assignedLog.code],
    voucher: voucher,
  );

  final updatedLog = EquipmentLog(
    code: assignedLog.code,
    status: StatusType.outOfStock,
    createdAt: assignedLog.createdAt,
    updatedAt: DateTime.now(),
    assignedToTransactionId: purchaseId,
  );

  final updatedEquipment = EquipmentItem(
    id: equipment.id,
    imageUrl: equipment.imageUrl,
    name: equipment.name,
    providerId: equipment.providerId,
    quantity: equipment.quantity,
    terms: equipment.terms,
    type: equipment.type,
    updatedAt: DateTime.now(),
    description: equipment.description,
    category: equipment.category,
    price: equipment.price,
  );

  final equipmentRef = _firestore.collection('equipment').doc(equipment.id);
  final purchaseRef = _firestore
      .collection('equipment_transactions')
      .doc(purchaseId);
  final statusRef = equipmentRef.collection('status').doc(updatedLog.code);

  await _firestore.runTransaction((transaction) async {
    transaction.set(purchaseRef, purchase.toMap());
    transaction.update(equipmentRef, updatedEquipment.toJson());
    transaction.set(statusRef, updatedLog.toJson());
  });
}
```

This function will be called when the user purchases a for-purchase item. Similar to the createAndSaveBooking function, it performs the same operations, but it will update the first equipment log by changing its status to outOfStock, indicating that the equipment has been sold to the user.

Figure 69

getUserBookings function

```
Stream<List<EquipmentTransaction>> getUserBookings(String userId) {
  return _firestore
      .collection('equipment_transactions')
      .where('userId', isEqualTo: userId)
      .snapshots()
      .map((snapshot) {
        return snapshot.docs
            .map((doc) => EquipmentTransaction.fromMap(doc.data()))
            .toList();
      });
}
```

This function is designed for the explorer to fetch all the equipment they have rented or purchased from Firebase. It receives the user ID as input, and searches the equipment_transactions collection for documents where the userId attribute matches the inputted user ID. Then, it returns a list of EquipmentTransaction objects.

Figure 70

getEquipmentByProviderId function

```dart
static Stream<List<EquipmentTransaction>?> getEquipmentByProviderId(
  String providerId,
) {
  return _firestore
      .collection('equipment_transactions')
      .where('equipmentItem.providerId', isEqualTo: providerId)
      .snapshots()
      .map((snapshot) {
        return snapshot.docs
            .map((doc) => EquipmentTransaction.fromMap(doc.data()))
            .toList();
      });
}
```

This function is designed for the equipment provider to fetch all the purchased and rented equipment transactions. It receives the provider ID as input, and searches for documents in which the providerId matches the inputted provider ID. The results are returned as a stream of EquipmentTransaction objects.

Figure 71

updateEquipmentToPaid function

```dart
static Future<bool> updateEquipmentToPaid(
  String equipmentTransactionId,
) async {
  try {
    final equipmentRef = _firestore
        .collection('equipment_transactions')
        .doc(equipmentTransactionId);

    final equipmentDoc = await equipmentRef.get();

    if (!equipmentDoc.exists) {
      throw Exception("Equipment transaction not found.");
    }

    await equipmentRef.update({'isPaid': true});

    return true;
  } catch (e) {
    print("Error processing payment: $e");
    return false;
  }
}
```

This function will update the transaction record to paid. It receives the equipment transaction ID, fetches the relevant document, and updates the isPaid attribute to true.

Figure 72

updateTransactionStatus function

```dart
static Future<bool> updateTransactionStatus(
  String transactionId,
  EquipmentRentingStatus newStatus,
) async {
  try {
    await _firestore
        .collection('equipment_transactions')
        .doc(transactionId)
        .update({
          'status': newStatus.name,
          'updatedAt': FieldValue.serverTimestamp(),
        });
    return true;
  } catch (e) {
    print('Error updating transaction status: $e');
    return false;
  }
}
```

This function is used for updating the equipment transaction status. It uses the inputted transaction ID to find the relevant transaction and updates its status field with the inputted status. The status indicates the current state of the transaction, such as completed, delivered, or pending.

Figure 73

addLog function

```dart
static Future<void> addLog(String equipmentId, EquipmentLog log) async {
  final statusRef = _firestore
      .collection('equipment')
      .doc(equipmentId)
      .collection('status')
      .doc(log.code);

  await statusRef.set(log.toJson());
}
```

The addLog function is used to add a new equipment log to a specific equipment. It creates a new document in the status subcollection using the equipment log code as the document ID. Then, it converts the equipment log to JSON format and saves it inside the document.

Figure 74

doesLogCodeExist function

```dart
static Future<bool> doesLogCodeExist(String equipmentId, String code) async {
  final docRef = _firestore
      .collection('equipment')
      .doc(equipmentId)
      .collection('status')
      .doc(code);

  final doc = await docRef.get();
  return doc.exists;
}
```

This function checks whether the log code already exists in Firebase. It does this by attempting to fetch a document using the inputted code. If a document is found, it means the code exists and the function returns true. Otherwise, it returns false.

Figure 75

updateLogStatus function

```dart
static Future<void> updateLogStatus(
  String equipmentId,
  String code,
  StatusType status,
) async {
  final statusRef = _firestore
      .collection('equipment')
      .doc(equipmentId)
      .collection('status')
      .doc(code);

  await statusRef.update({'status': statusTypeToString(status)});
}
```

This function receives the latest log status and a code. It uses the code to fetch the relevant document and then updates the status attribute with the inputted log status.

Figure 76

deleteLog function

```dart
static Future<void> deleteLog(String equipmentId, String code) async {
  final statusRef = _firestore
      .collection('equipment')
      .doc(equipmentId)
      .collection('status')
      .doc(code);

  await statusRef.delete();
}
```

This function receives the equipment log code, uses it to locate the corresponding document, and deletes the document from Firebase.

Figure 77

getLogs and fetchEquipmentLogs functions

```
static Stream<List<EquipmentLog>> getLogs(String equipmentId) {
  return _firestore
      .collection('equipment')
      .doc(equipmentId)
      .collection('status')
      .orderBy('createdAt', descending: true)
      .snapshots()
      .map(
        (snapshot) =>
            snapshot.docs
                .map((doc) => EquipmentLog.fromJson(doc.data()))
                .toList(),
      );
}

Future<List<EquipmentLog>> fetchEquipmentLogs(String equipmentId) async {
  final snapshot =
      await _firestore
          .collection('equipment')
          .doc(equipmentId)
          .collection('status')
          .get();

  return snapshot.docs
      .map((doc) => EquipmentLog.fromJson(doc.data()))
      .toList();
}
```

These two functions are designed to fetch the logs for a specific equipment. They receive the equipment ID as input and use it to locate the relevant status subcollection. Each equipment log is then converted into an EquipmentLog object and returned.

2.4.12 Bookings & Events

Figure 78

watchBookingByUserId and getUserBookings function

```dart
static Stream<List<BookingRequest>> watchBookingsByUserId(String userId) {
  return FirebaseFirestore.instance
      .collection('users')
      .doc(userId)
      .collection('bookings')
      .orderBy('createdAt', descending: true)
      .snapshots()
      .map(
        (snapshot) =>
            snapshot.docs
                .map((doc) => BookingRequest.fromJson(doc.data()))
                .toList(),
      );
}

Future<List<BookingRequest>> getUserBookings(String userId) async {
  final snapshot =
      await FirebaseFirestore.instance
          .collection('users')
          .doc(userId)
          .collection('bookings')
          .get();

  return snapshot.docs
      .map((doc) => BookingRequest.fromJson(doc.data()))
      .toList();
}
```

These two functions are designed to fetch user bookings corresponding to the input user ID. They check the relevant bookings subcollection within the users collection.

Figure 79

updateBooking function

```dart
static Future<BookingRequest> updateBooking({
  required String userId,
  required String bookingId,
  required Map<String, dynamic> updatedData,
}) async {
  try {
    await FirebaseFirestore.instance
        .collection('users')
        .doc(userId)
        .collection('bookings')
        .doc(bookingId)
        .update(updatedData);

    final updatedDoc =
        await FirebaseFirestore.instance
            .collection('users')
            .doc(userId)
            .collection('bookings')
            .doc(bookingId)
            .get();

    return BookingRequest.fromJson(updatedDoc.data()!);
  } catch (e) {
    throw Exception('Failed to update booking: $e');
  }
}
```

This function allows users to update a booking. It takes the user ID, booking ID, and a map of key-value pairs as input. The function starts by updating the booking document in the bookings subcollection, then fetches the updated document, converts it into a BookingRequest object, and returns it.

Figure 80

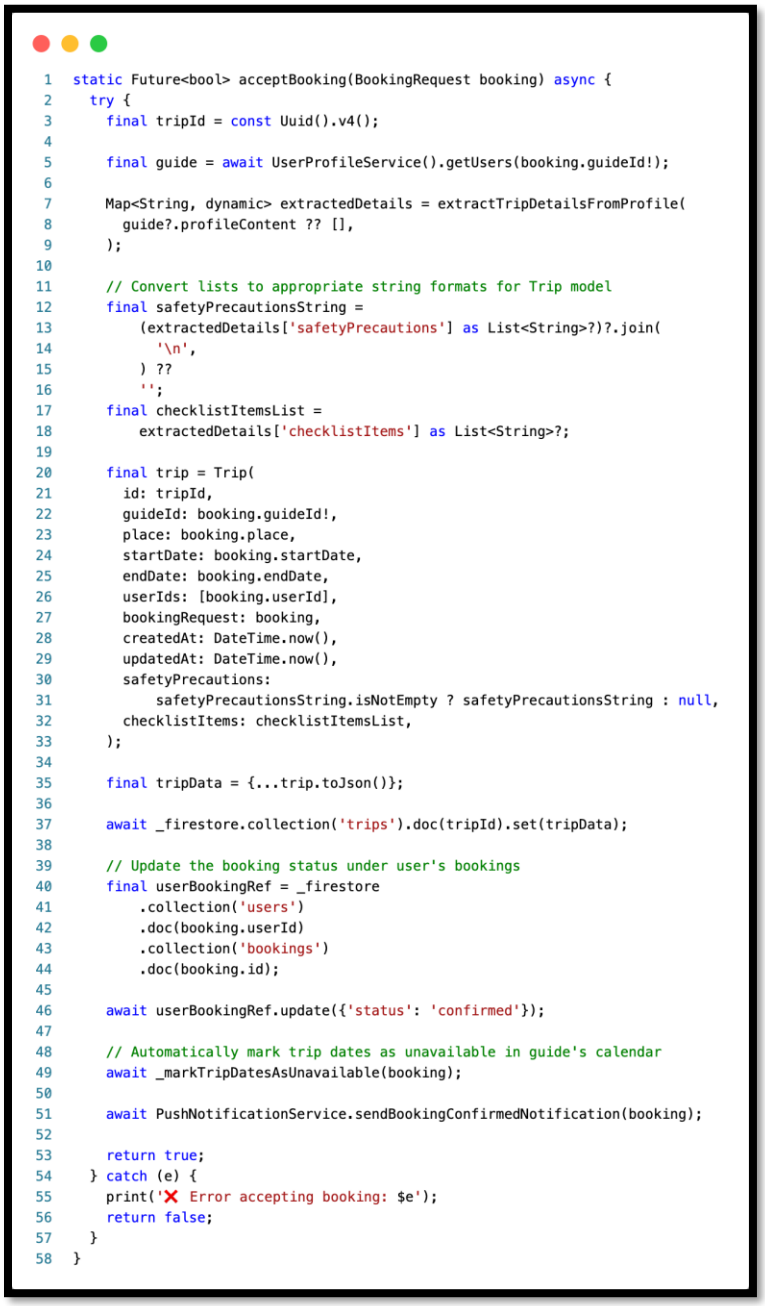
submitBooking function

```dart
static Future<bool> submitBooking(BookingRequest booking) async {
  try {
    final user = _auth.currentUser;
    if (user == null) {
      throw Exception('No user signed in');
    }

    // Test JSON conversion first
    Map<String, dynamic> bookingData;
    try {
      bookingData = booking.toJson();
    } catch (jsonError) {
      throw Exception('Failed to convert booking to JSON: $jsonError');
    }

    final bookingsCollection = _firestore
        .collection('users')
        .doc(user.uid)
        .collection('bookings');

    await bookingsCollection.doc(booking.id).set(bookingData);

    // Verify the document was created
    try {
      final docSnapshot = await bookingsCollection.doc(booking.id).get();
      if (docSnapshot.exists) {
      } else {
        throw Exception('Document not found after creation');
      }
    } catch (verifyError) {
      print('❌ Error verifying document: $verifyError');
    }

    // await PushNotificationService.sendBookingSubmittedNotification(booking);

    return true;
  } catch (e) {
    return false;
  }
}
```

The submitBooking function is created for explorers to submit new bookings. It will receive a BookingRequest object and convert it into JSON format first. Then, it will use the booking ID as the document ID and save the JSON-formatted data into the document.

Figure 81

acceptBooking function



```dart
static Future<bool> acceptBooking(BookingRequest booking) async {
  try {
    final tripId = const Uuid().v4();

    final guide = await UserProfileService().getUsers(booking.guideId!);

    Map<String, dynamic> extractedDetails = extractTripDetailsFromProfile(
      guide?.profileContent ?? [],
    );

    // Convert lists to appropriate string formats for Trip model
    final safetyPrecautionsString =
        (extractedDetails['safetyPrecautions'] as List<String>?)?.join(
          '\n',
        ) ??
        '';
    final checklistItemsList =
        extractedDetails['checklistItems'] as List<String>?;

    final trip = Trip(
      id: tripId,
      guideId: booking.guideId!,
      place: booking.place,
      startDate: booking.startDate,
      endDate: booking.endDate,
      userIds: [booking.userId],
      bookingRequest: booking,
      createdAt: DateTime.now(),
      updatedAt: DateTime.now(),
      safetyPrecautions:
          safetyPrecautionsString.isNotEmpty ? safetyPrecautionsString : null,
      checklistItems: checklistItemsList,
    );

    final tripData = {...trip.toJson()};

    await _firestore.collection('trips').doc(tripId).set(tripData);

    // Update the booking status under user's bookings
    final userBookingRef = _firestore
        .collection('users')
        .doc(booking.userId)
        .collection('bookings')
        .doc(booking.id);

    await userBookingRef.update({'status': 'confirmed'});

    // Automatically mark trip dates as unavailable in guide's calendar
    await _markTripDatesAsUnavailable(booking);

    await PushNotificationService.sendBookingConfirmedNotification(booking);

    return true;
  } catch (e) {
    print('❌ Error accepting booking: $e');
    return false;
  }
}
```

This function is designed to allow the guide to accept a booking made by users. It begins by extracting the checklist and safety precautions from the guide's user profile. Then, it creates a Trip object, converts it to JSON, and saves it in the trips collection. After that, it updates the corresponding booking status to “confirmed”.

Figure 82

rejectBooking function

```dart
static Future<bool> rejectBooking(BookingRequest booking) async {
  try {
    print('📝 Rejecting booking: ${booking.id}');

    // Update the booking status under user's bookings to cancelled
    final userBookingRef = _firestore
        .collection('users')
        .doc(booking.userId)
        .collection('bookings')
        .doc(booking.id);

    await userBookingRef.update({'status': 'cancelled'});

    print('✅ Booking ${booking.id} rejected successfully');

    // Send notification to user about booking rejection
    await PushNotificationService.sendBookingRejectedNotification(booking);

    return true;
  } catch (e) {
    print('❌ Error rejecting booking: $e');
    return false;
  }
}
```

The rejectBooking function is designed for the guide to reject a booking made by an explorer. It begins by fetching the booking document from Firebase using the provided booking ID. Then, it updates the booking status to “cancelled” to reflect the rejection.

Figure 83

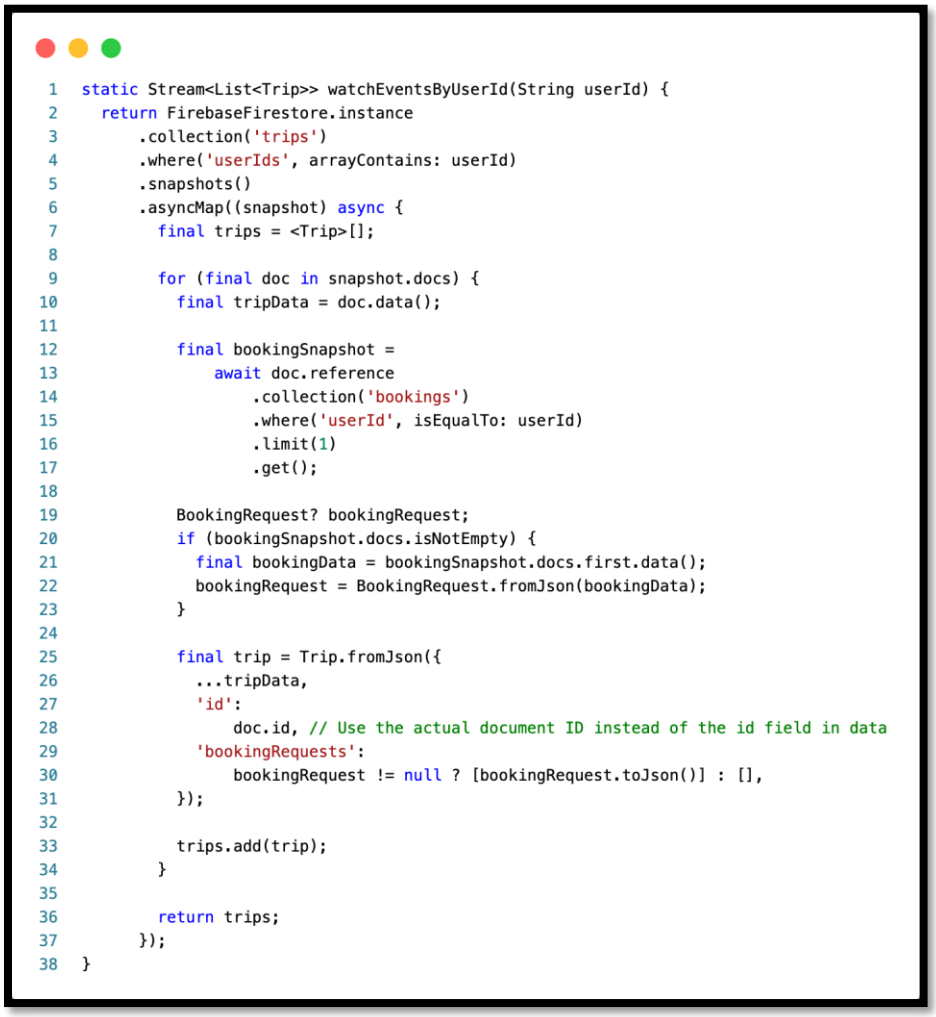
getGuideBookingByStatus function

```dart
static Future<List<BookingRequest>> getGuideBookingsByStatus(
  String guideId,
  BookingStatus? status,
) async {
  final allBookings = await getGuideBookings(guideId);

  if (status == null) {
    return allBookings;
  }

  print(
    'Filtering ${allBookings.length} bookings for guide $guideId by status $status',
  );

  List<BookingRequest> filteredBookings;

  // If filtering for cancelled, include both cancelled and withdraw statuses
  if (status == BookingStatus.cancelled) {
    filteredBookings =
        allBookings
            .where(
              (booking) =>
                  booking.status == BookingStatus.cancelled ||
                  booking.status == BookingStatus.withdraw,
            )
            .toList();
  } else {
    filteredBookings =
        allBookings.where((booking) => booking.status == status).toList();
  }

  print('Found ${filteredBookings.length} bookings with status $status');

  return filteredBookings;
}
```

This function is designed to retrieve bookings based on the booking status provided. It first fetches all bookings associated with the given guide ID from Firebase. Then, it performs client-side filtering based on the status. If the user provides “cancelled” as the desired status, the function will filter bookings with either “cancelled” or “withdraw” status.

Figure 84

watchEventsByUserId function



```dart
static Stream<List<Trip>> watchEventsByUserId(String userId) {
  return FirebaseFirestore.instance
      .collection('trips')
      .where('userIds', arrayContains: userId)
      .snapshots()
      .asyncMap((snapshot) async {
        final trips = <Trip>[];

        for (final doc in snapshot.docs) {
          final tripData = doc.data();

          final bookingSnapshot =
              await doc.reference
                  .collection('bookings')
                  .where('userId', isEqualTo: userId)
                  .limit(1)
                  .get();

          BookingRequest? bookingRequest;
          if (bookingSnapshot.docs.isNotEmpty) {
            final bookingData = bookingSnapshot.docs.first.data();
            bookingRequest = BookingRequest.fromJson(bookingData);
          }

          final trip = Trip.fromJson({
            ...tripData,
            'id':
                doc.id, // Use the actual document ID instead of the id field in data
            'bookingRequests':
                bookingRequest != null ? [bookingRequest.toJson()] : [],
          });

          trips.add(trip);
        }

        return trips;
      });
}
```

This function is created to watch events related to a specific user. It takes a userId as input, queries the trip collection for documents where the userIds field contains that userId. Then, it loads the related booking data into each trip object before returning the list of enriched trips.

Figure 85

watchEventById function

```
static Stream<Trip> watchEventById(String tripId) {
  return FirebaseFirestore.instance
      .collection('trips')
      .doc(tripId)
      .snapshots()
      .asyncMap((snapshot) async {
        if (!snapshot.exists) {
          throw Exception('Trip not found');
        }

        final tripData = snapshot.data()!;
        final bookingSnapshot =
            await snapshot.reference.collection('bookings').limit(1).get();

        BookingRequest? bookingRequest;
        if (bookingSnapshot.docs.isNotEmpty) {
          final bookingData = bookingSnapshot.docs.first.data();
          bookingRequest = BookingRequest.fromJson(bookingData);
        }

        return Trip.fromJson({
          ...tripData,
          'id':
              snapshot
                  .id, // Use the actual document ID instead of the id field in data
          'bookingRequests':
              bookingRequest != null ? [bookingRequest.toJson()] : [],
        });
      });
}
```

This function listens to real-time updates of a specific trip by tripId. It fetches the trip document and retrieves its first booking from the bookings subcollection. The booking is added to the trip data, and the combined result is returned as a Trip object in a stream.

Figure 86

sendMessage function

```dart
static Future<void> sendMessage({
  required String tripId,
  required String senderId,
  required String message,
}) async {
  final ref = FirebaseFirestore.instance
      .collection('trips')
      .doc(tripId)
      .collection('chatMessages');

  await ref.add({
    'senderId': senderId,
    'message': message,
    'timestamp': FieldValue.serverTimestamp(),
  });
}
```

This function will send a chat message within a specific trip. It will take the tripId, senderId, and message content as input, and add the message to the chatMessages subcollection under the trip, including a server-generated timestamp.

Figure 87

getMessages function

```dart
static Stream<List<ChatMessage>> getMessages(String tripId) {
  return FirebaseFirestore.instance
      .collection('trips')
      .doc(tripId)
      .collection('chatMessages')
      .orderBy('timestamp', descending: false)
      .snapshots()
      .map((snapshot) {
        return snapshot.docs.map((doc) {
          final data = doc.data();

          final rawTimestamp = data['timestamp'];
          DateTime safeTimestamp;

          if (rawTimestamp is Timestamp) {
            safeTimestamp = rawTimestamp.toDate();
          } else if (rawTimestamp is DateTime) {
            safeTimestamp = rawTimestamp;
          } else {
            // fallback to now or any default
            safeTimestamp = DateTime.now();
          }

          return ChatMessage.fromJson({
            'id': doc.id,
            'senderId': data['senderId'],
            'message': data['message'],
            'timestamp': safeTimestamp,
          });
        }).toList();
      });
}
```

This function retrieves real-time chat messages for a specific trip. It listens to the chatMessages subcollection of the given tripId. Then, it will convert the timestamp and return a stream of ChatMessage objects.

Figure 88

markBookingAsPaid function

```dart
static Future<void> markBookingAsPaid({required String tripId}) async {
  final tripRef = FirebaseFirestore.instance.collection('trips').doc(tripId);

  await tripRef.update({'bookings.isPaid': true});
}
```

This function updates the paymentStatus of a specific trip to true in the trips collection, indicating that the trip has been paid for.

2.5 User Permission

In order to improve user experience when using the PathFindr application, some permissions are required to enable certain features in PathFindr. This includes location permission for both foreground and background, internet access permission, and vibration permission.

Figure 89

Location Configuration in android/app/src/main/AndroidManifest.xml

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.VIBRATE"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
```

Figure 90

Location Configuration in ios/Runner/Info.plist

```xml
<key>NSLocationWhenInUseUsageDescription</key>
<string>This app needs your location to find nearby adventure places.</string>
<key>NSLocationAlwaysUsageDescription</key>
<string>This app needs access to your location even when the app is in the background.</string>
<key>NSLocationAlwaysAndWhenInUseUsageDescription</key>
<string>We use your location to help plan outdoor activities.</string>
```

These two screenshots shows the root configuration files for Android and iOS: AndroidManifest.xml for Android and Info.plist for iOS. On iOS, certain permissions, such as

internet access and vibration do not require explicit configuration, whereas on Android, they must be declared in the AndroidManifest.xml file.

Regarding location permission, although the permission_handler library is known for its ease of use and flexibility in handling various permissions, this application only requires location access. Therefore, the geolocator library (version 14.0.1) is more suitable, as it not only manages location permission internally but also provides functionality for retrieving the device's current location.

A global provider, LocationViewModel, is created to manage location permissions and retrieve the user's current location efficiently. This view model is initialized at the start of the application, ensuring that all parts of the app can easily access and utilize it.

Figure 91

LocationViewModel code snippet

```dart
LocationViewModel() {
  _initializeStatus();
}

/// Initialize service and permission status
Future<void> _initializeStatus() async {
  await _updateServiceStatus();
  await _updatePermissionStatus();
  debugPrint("Location service enabled: $_isServiceEnabled");
  debugPrint("Location permission granted: $_isPermissionGranted");
  if (hasListeners) {
    notifyListeners();
  }
}

/// Check and update the current service status
Future<void> _updateServiceStatus() async {
  _isServiceEnabled = await Geolocator.isLocationServiceEnabled();
}

/// Check and update the current permission status
Future<void> _updatePermissionStatus() async {
  final permission = await Geolocator.checkPermission();
  _isPermissionGranted = _isGranted(permission);
}
```

The screenshot above shows that the LocationViewModel checks whether the user has granted location permission and updates its local attribute accordingly to facilitate data retrieval.

Figure 92

requestPermission function in LocationViewModel

```dart
/// Request location permission and update state
Future<void> requestPermission() async {
  LocationPermission permission = await Geolocator.checkPermission();

  if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
  }

  _isPermissionGranted = _isGranted(permission);
  if (hasListeners) {
    notifyListeners();
  }
}
```

A function, requestPermission(), has also been developed to facilitate the permission request process. It checks whether location permission has been granted, and if not, it uses the Geolocator API to request the necessary permission from the user.

Figure 93

getCurrentLocation function in LocationViewModel

```dart
Future<Position?> getCurrentLocation() async {
  await _updateServiceStatus();
  await requestPermission();

  if (!isLocationReady) {
    debugPrint(
      "Location not ready. Permission: $_isPermissionGranted, Service: $_isServiceEnabled",
    );
    return null;
  }

  try {
    _currentPosition = await Geolocator.getCurrentPosition(
      desiredAccuracy: LocationAccuracy.high,
    );
    if (hasListeners) {
      notifyListeners();
    }
    return _currentPosition;
  } catch (e) {
    debugPrint("Error getting current location: $e");
    return null;
  }
}
```

For location retrieval, a function called getCurrentLocation has been created to obtain a high-accuracy location. It begins by checking the location permission, and only if the user has granted access, the application will proceed to retrieve the user's one-time location.

2.6 Wireframe

2.6.1 Common Functions

Figure 94

Onboarding Page

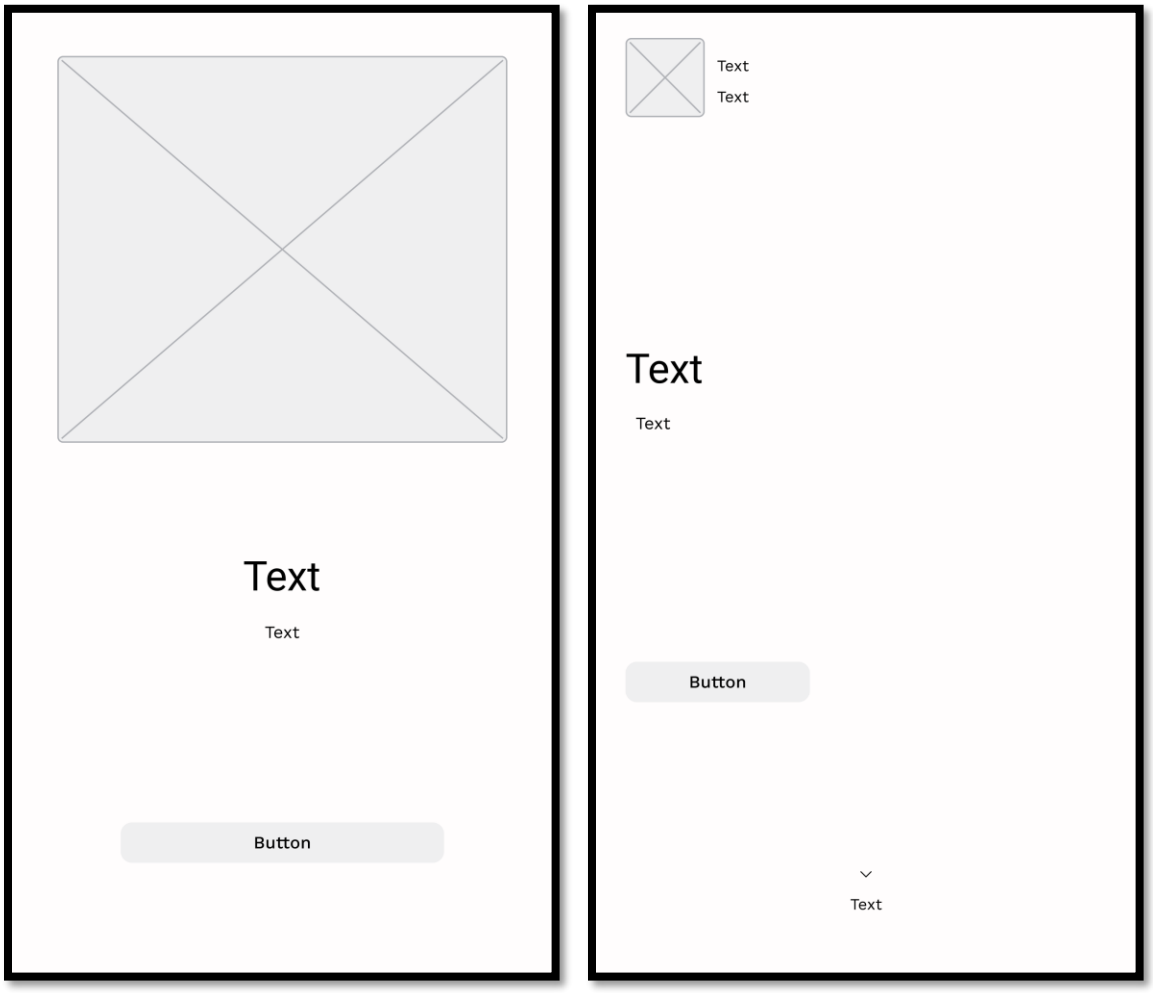


Figure 95

Login Page

Figure 96

Sign Up Page

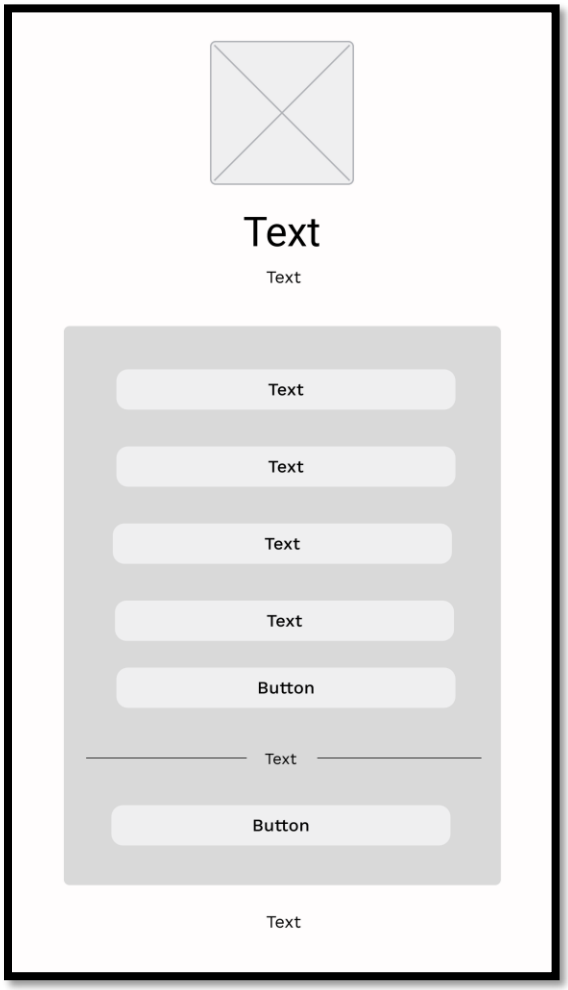


Figure 97

User Choose Roles

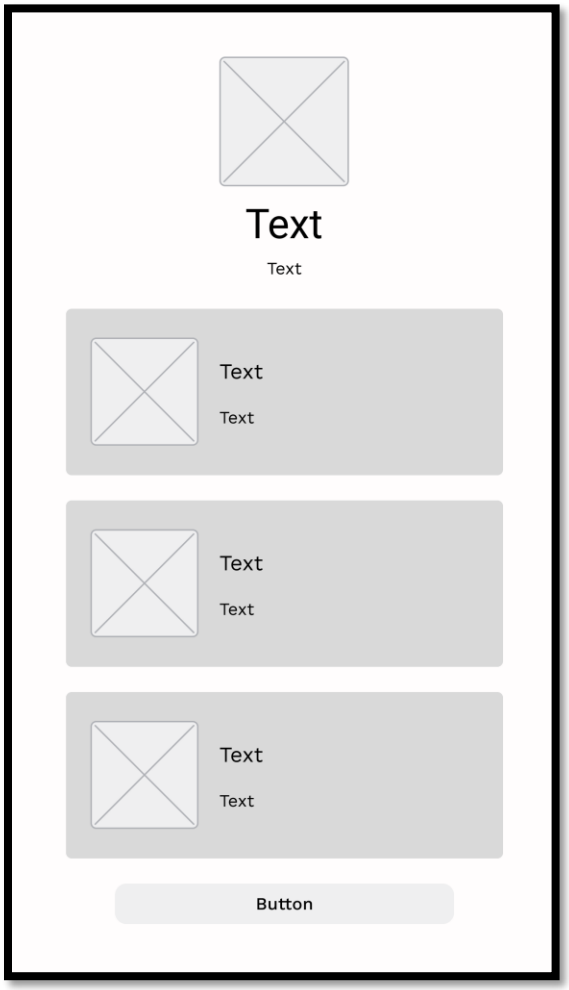


Figure 98

User Choose Roles Details

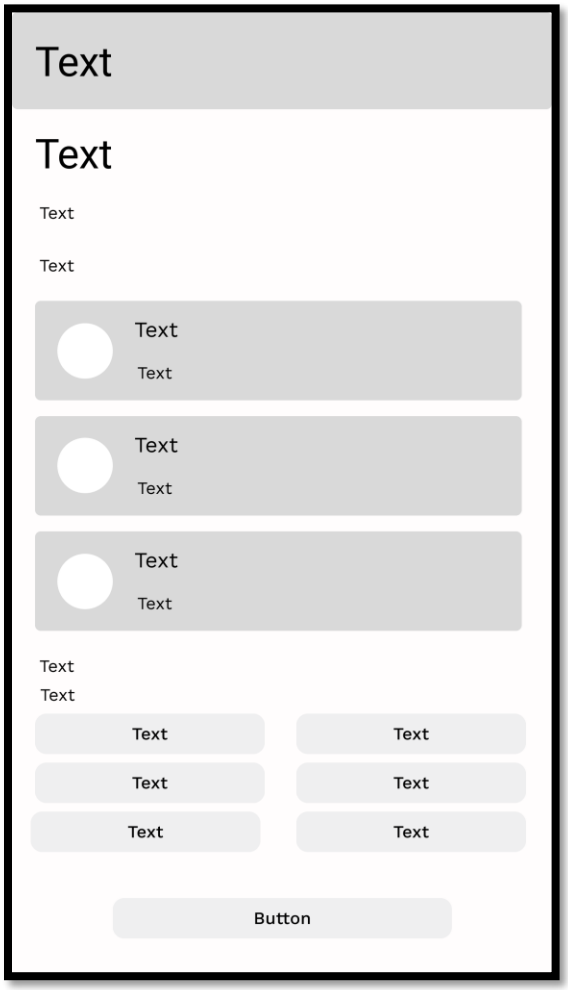


Figure 99

User Notification

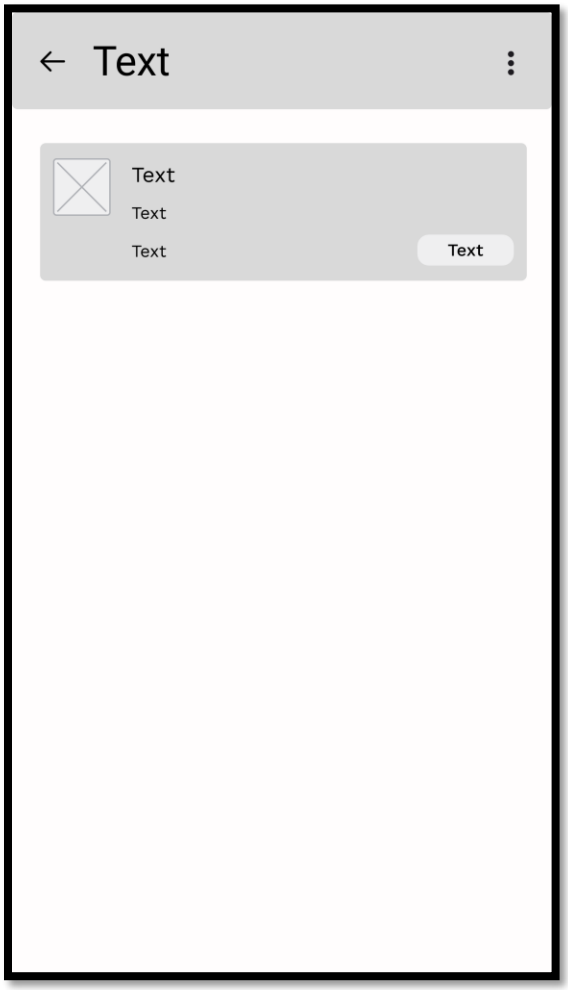


Figure 100

Wallet Function

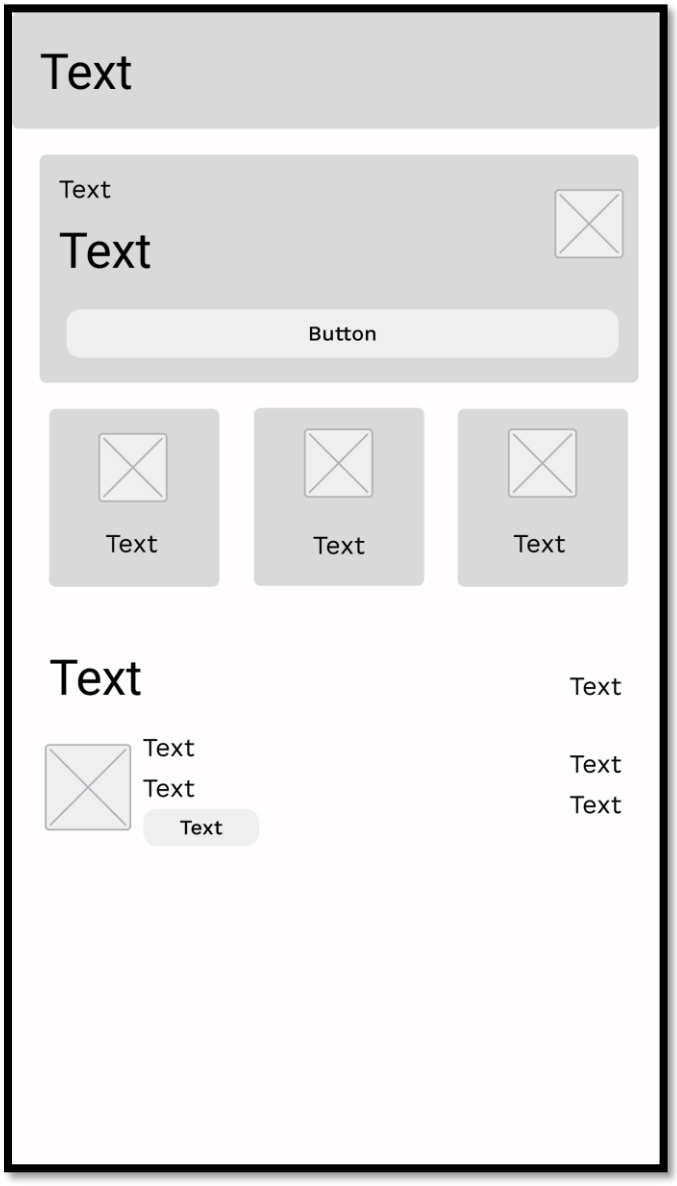


Figure 101

Topup / Withdraw Wallet

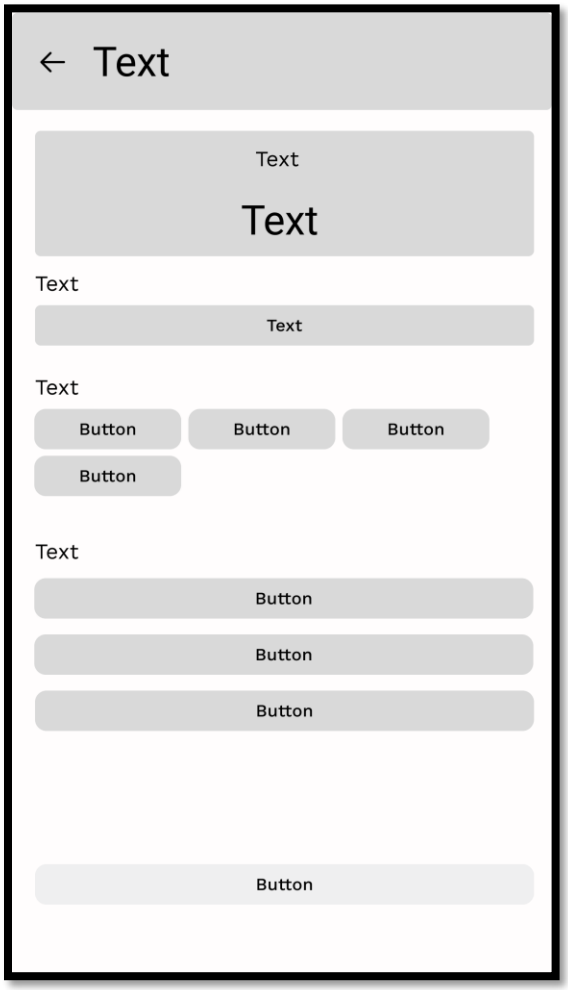


Figure 102

View Wallet Transaction History

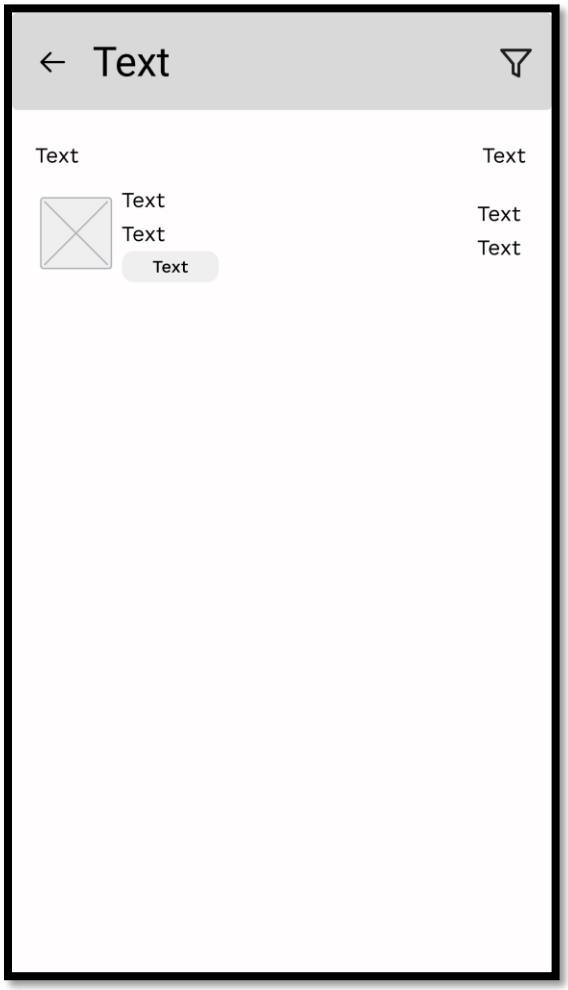


Figure 103

Chat Function

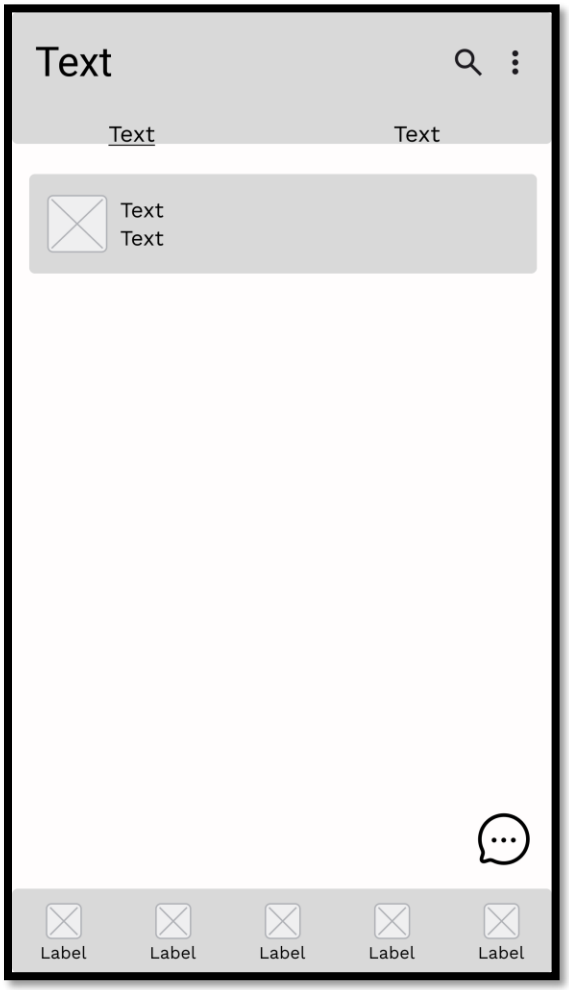
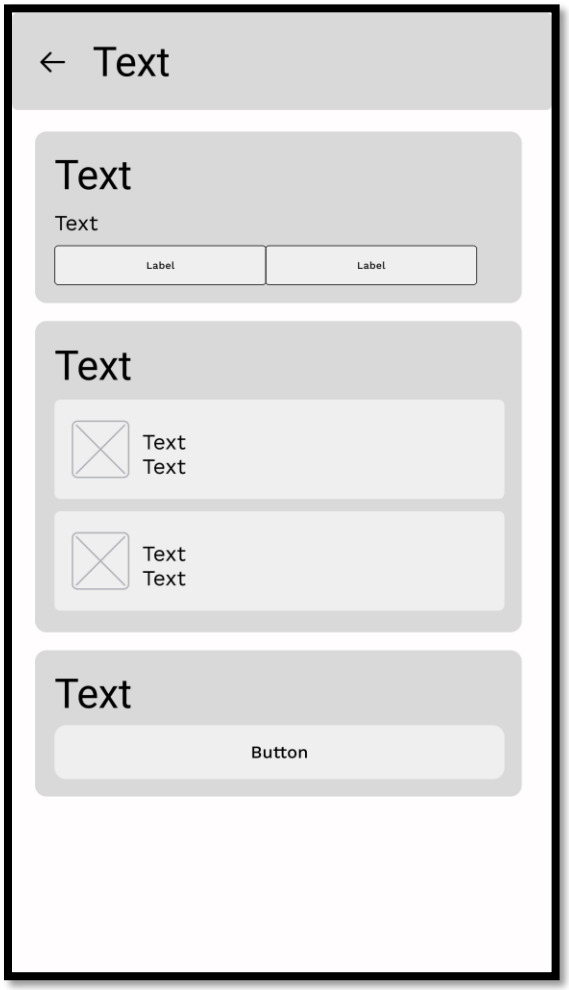


Figure 104

Settings



2.6.2 Explorer Role

Figure 105

Explorer Main Page

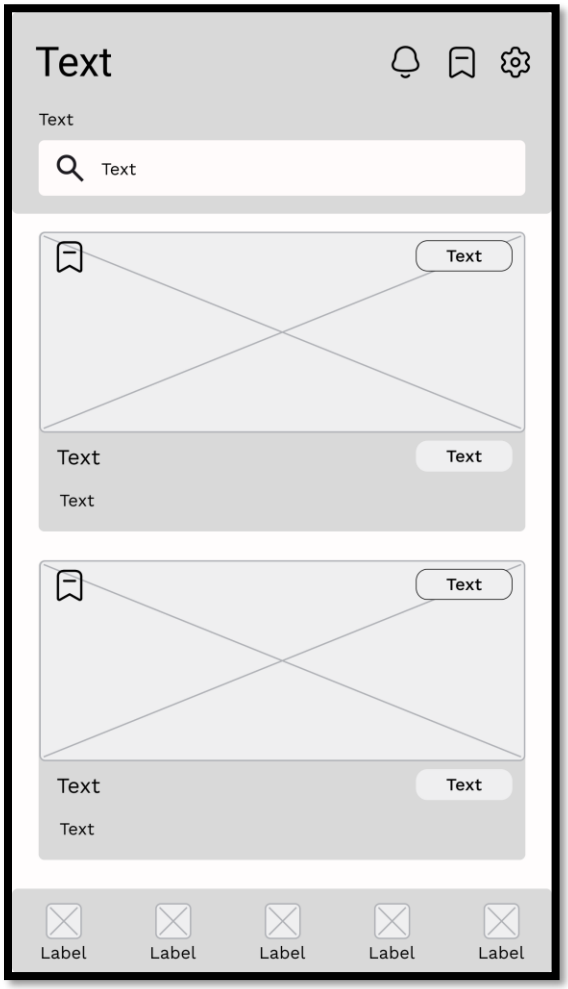


Figure 106

Explorer Saved Watchlist

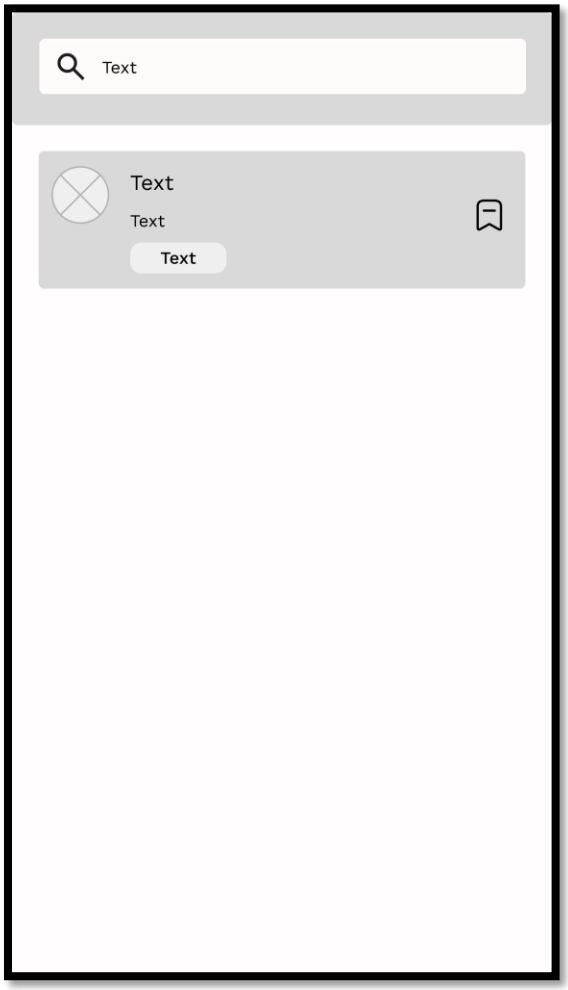


Figure 107

Explorer Outdoor Location Details

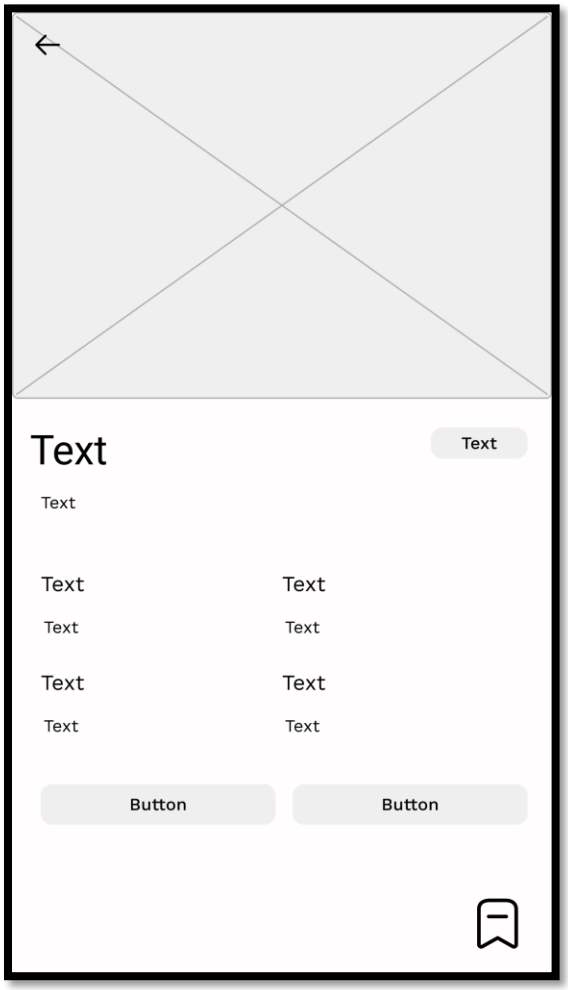


Figure 108

Explorer Book Events – State 1

Figure 109

Explorer Book Events – State 2

Figure 110

Explorer Book Events – State 3

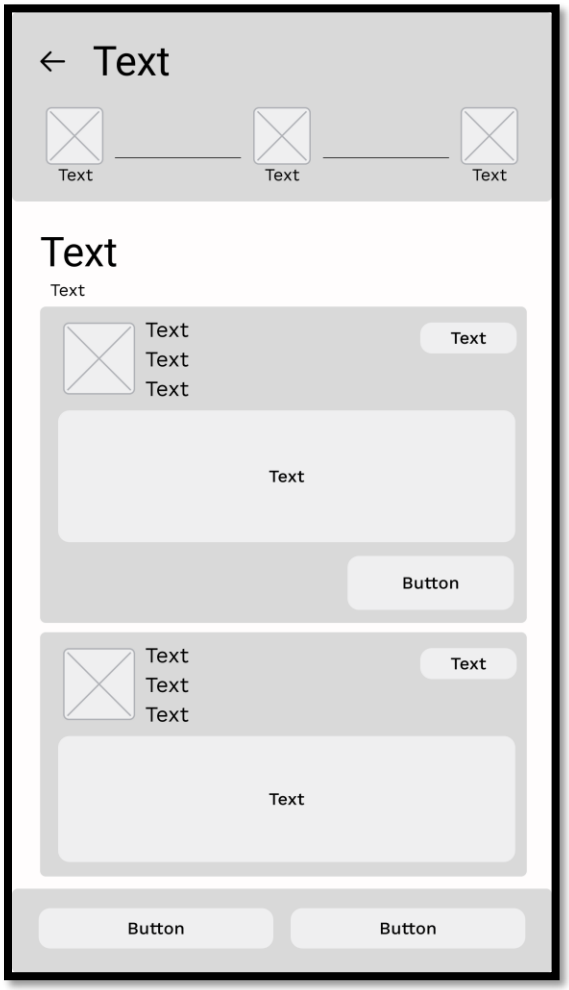


Figure 111

Explorer View Guide Profile

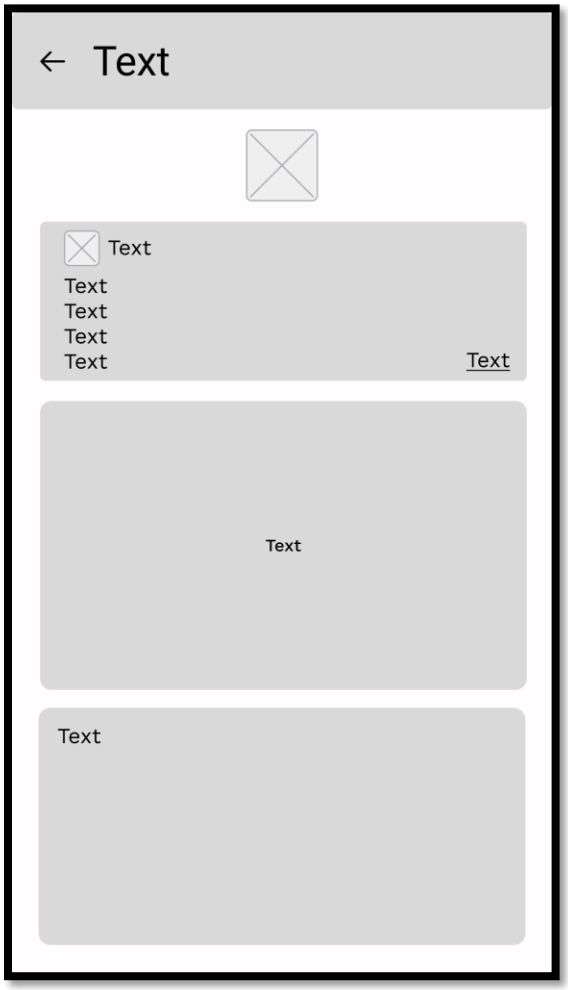


Figure 112

Explorer Bookings & Events Details

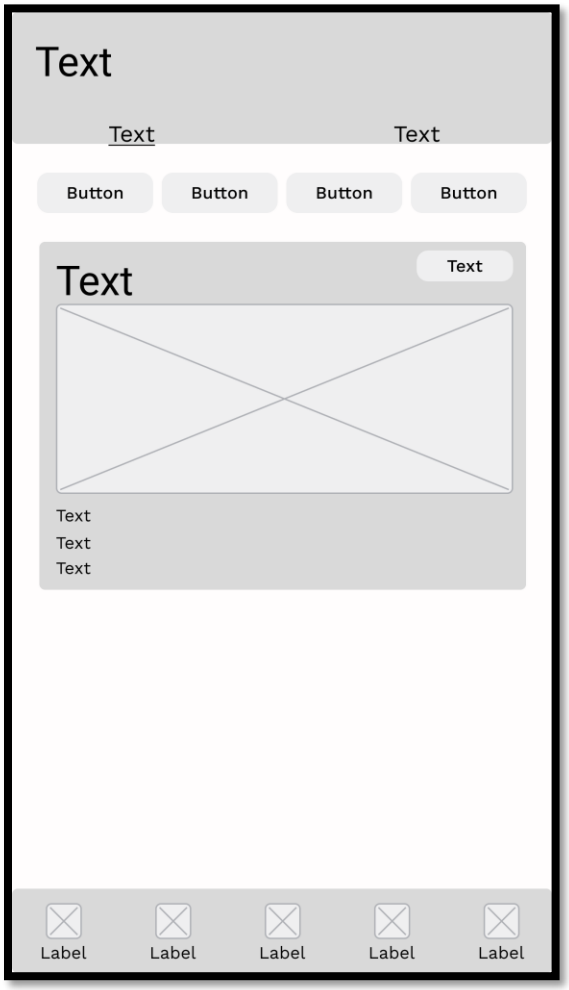


Figure 113

Explorer Withdraw Bookings

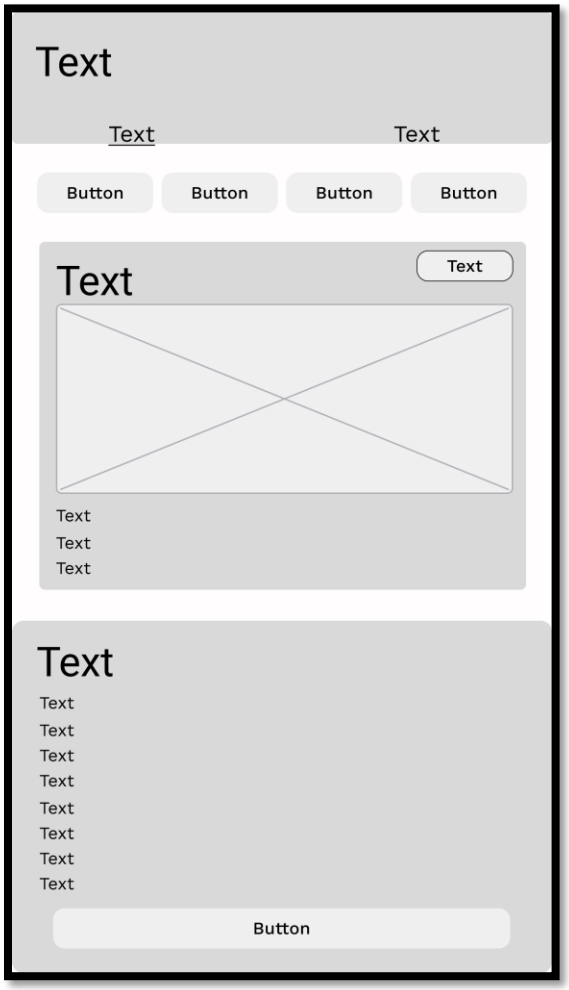


Figure 114

Explorer View Available Equipment

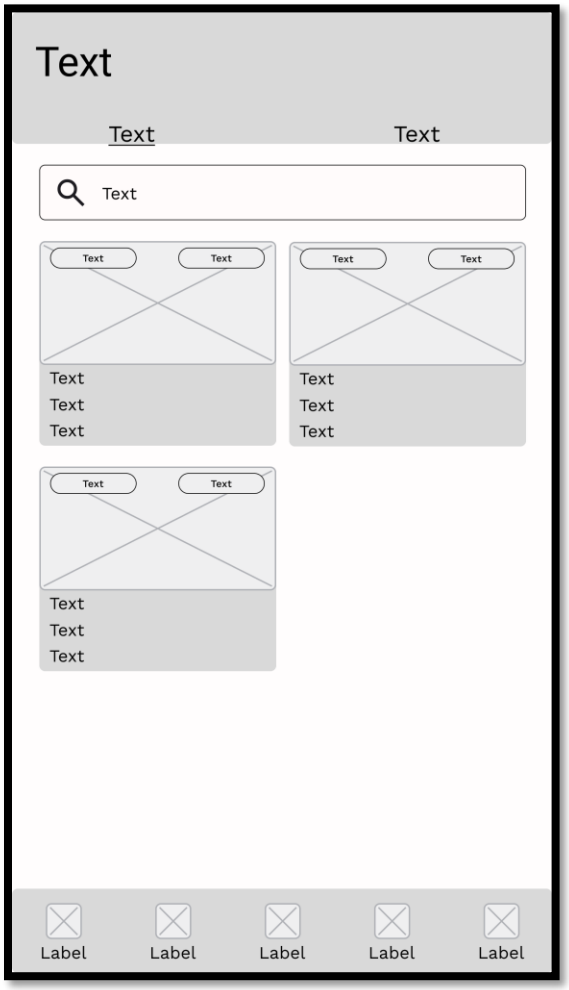


Figure 115

Explorer View Equipment Details

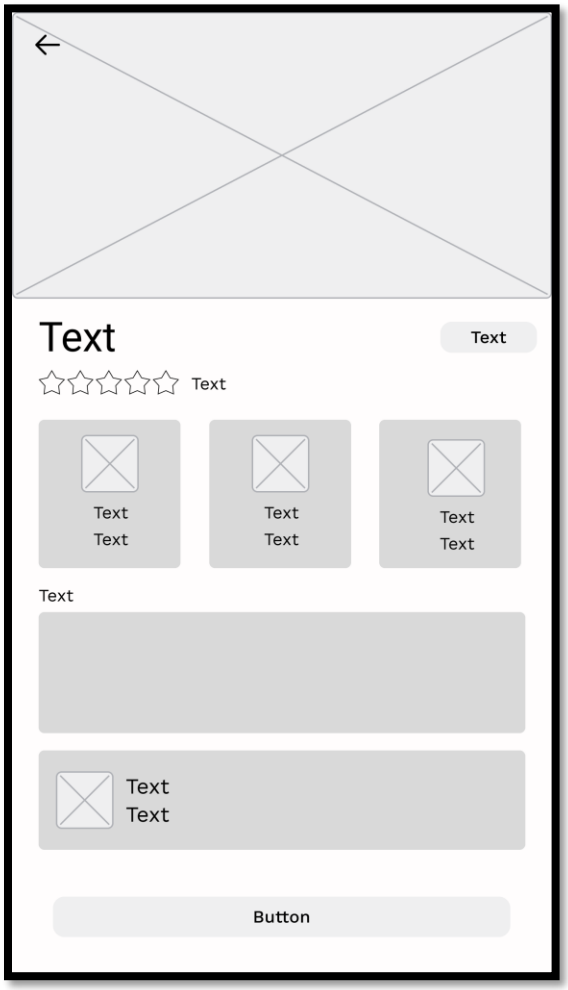


Figure 116

Explorer Rent / Purchase Equipment

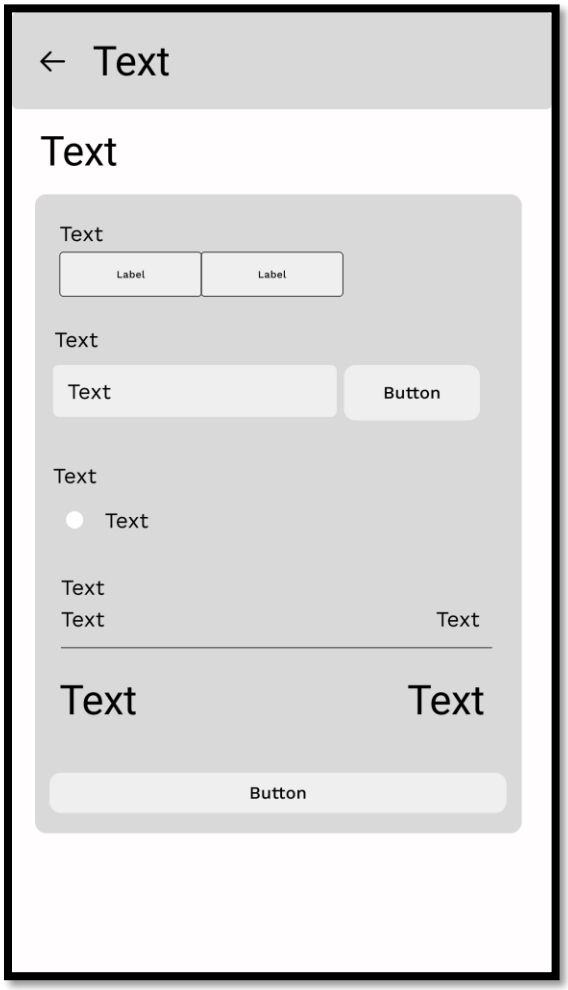


Figure 117

Explorer View Rented / Purchase Equipment Details

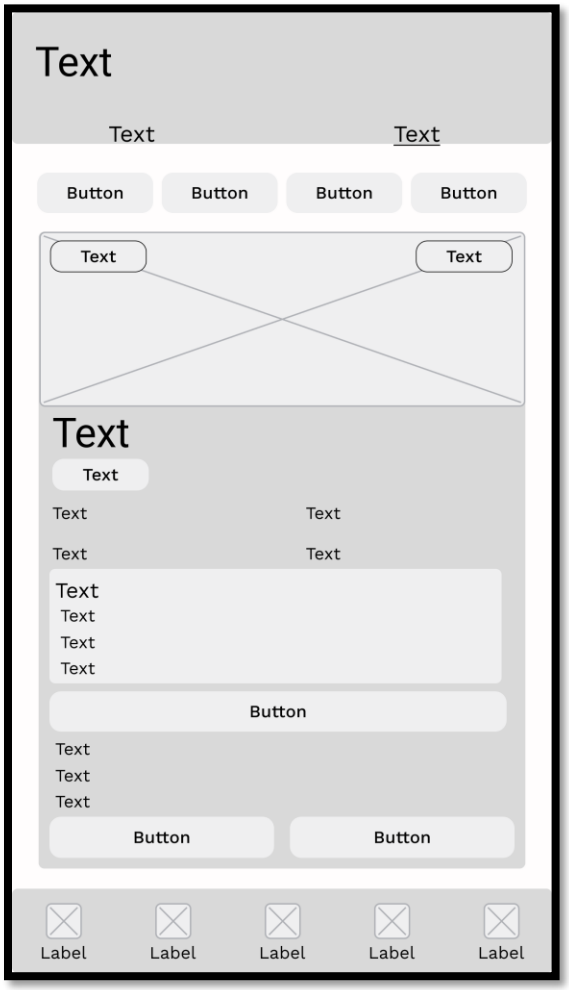
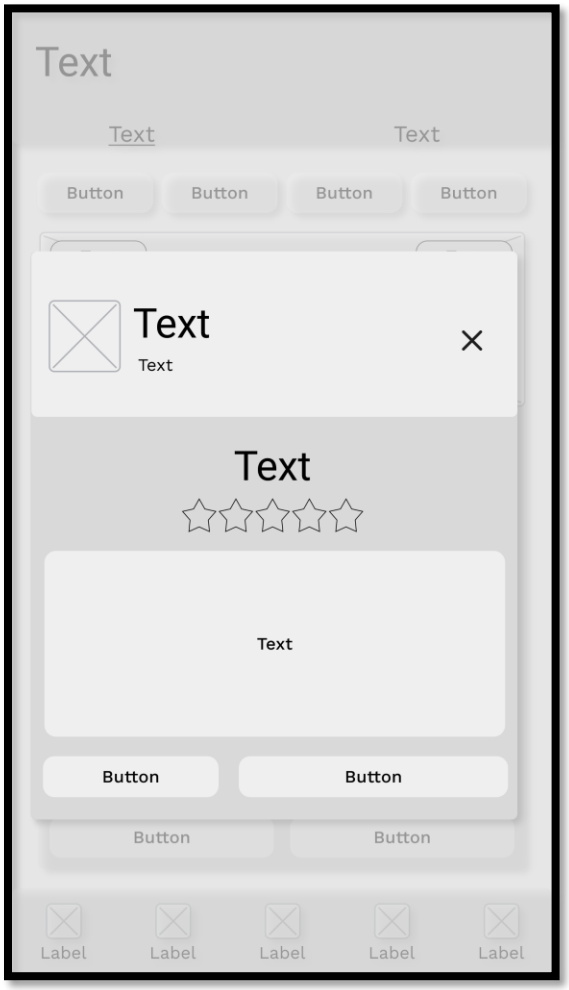


Figure 118

Explorer Provide Feedback



2.6.3 Guide Role

Figure 119

Guide Main Page

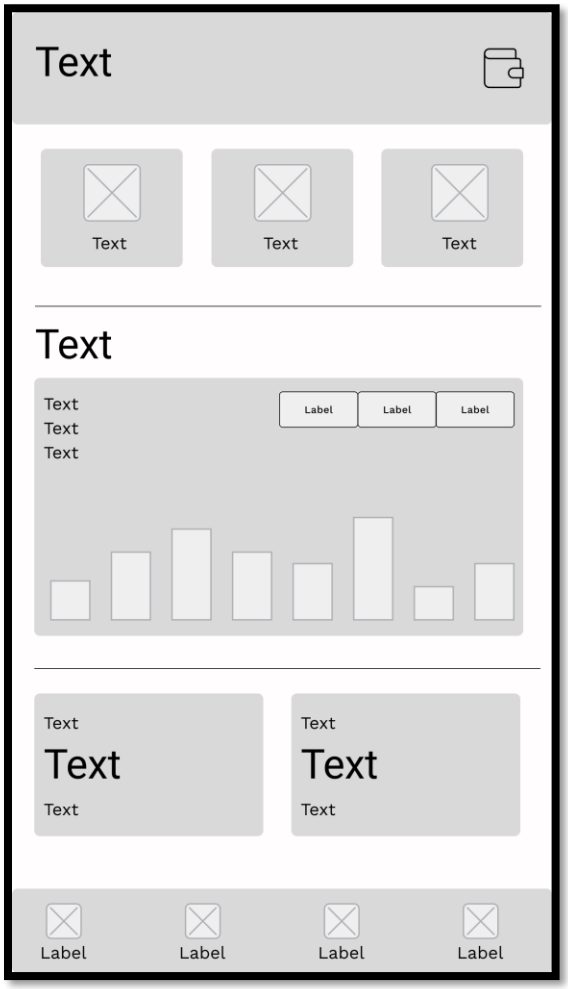


Figure 120

Guide View Events Hosted Events, Guide View Upcoming Adventure Details

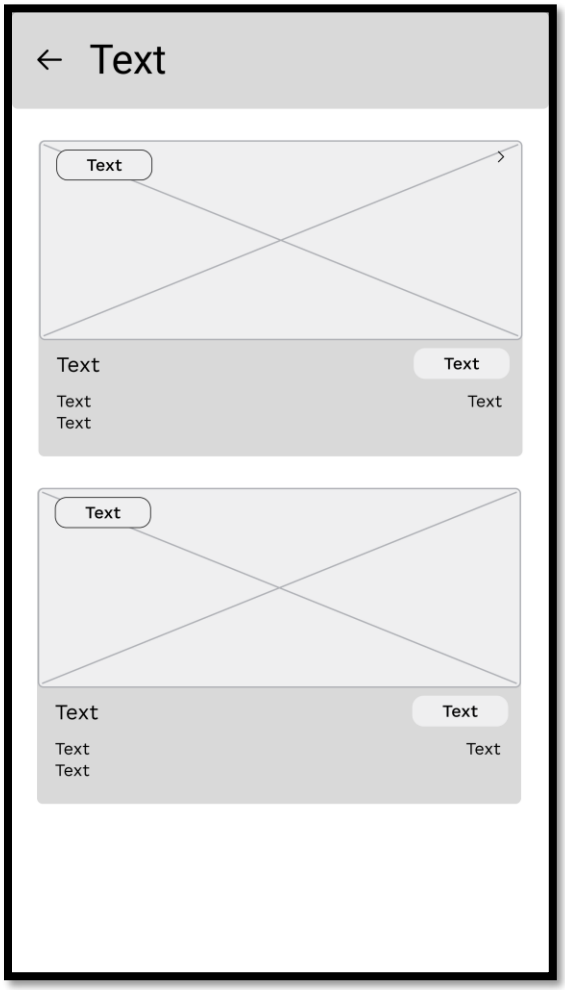


Figure 121

Guide View Booking Requests, Guide View Cancelled Events, Guide View Total Bookings

Figure 122

Guide Join Events – Stage 1

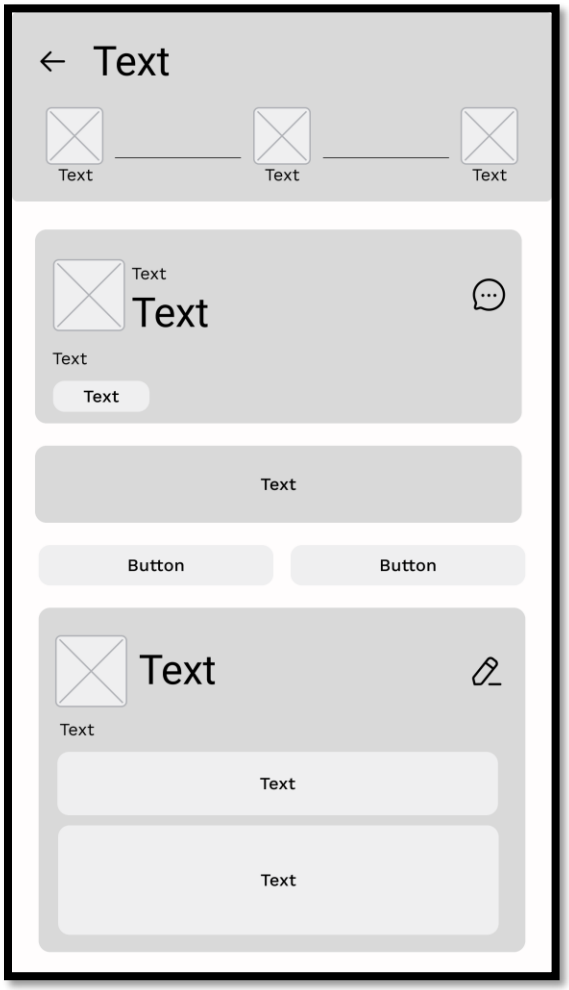


Figure 123

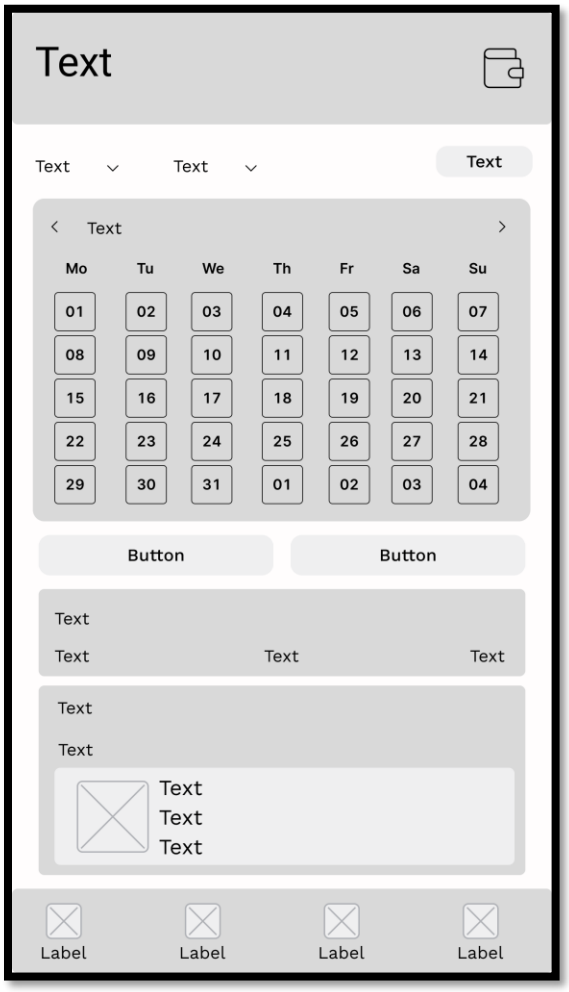
Guide Join Events – Stage 2

Figure 124

Guide Join Events – Stage 3

Figure 125

Guide View Calander



2.6.4 Equipment Provider Role

Figure 126

Equipment Provider Main Page

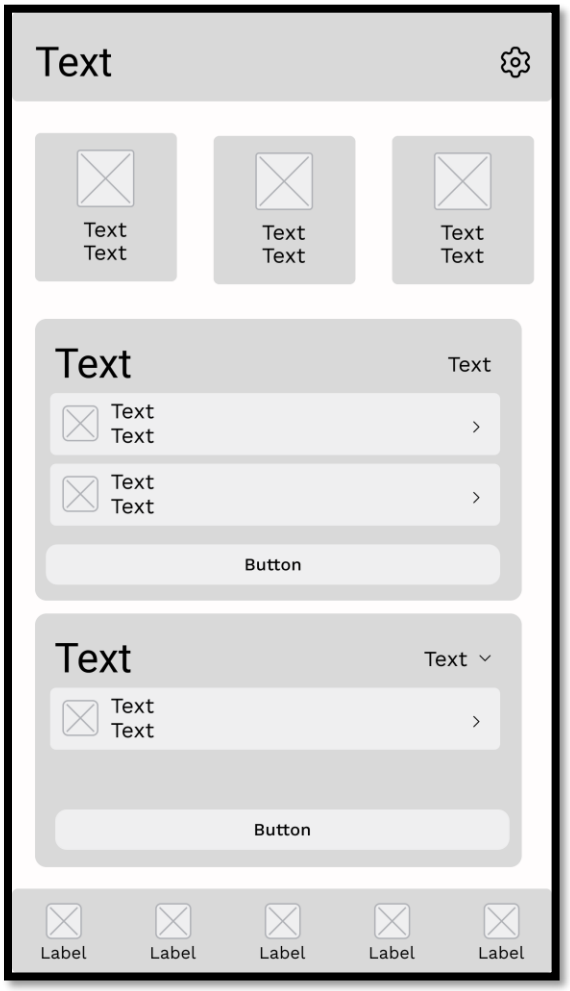


Figure 127

Equipment Provider Manage Orders & Purchase Details

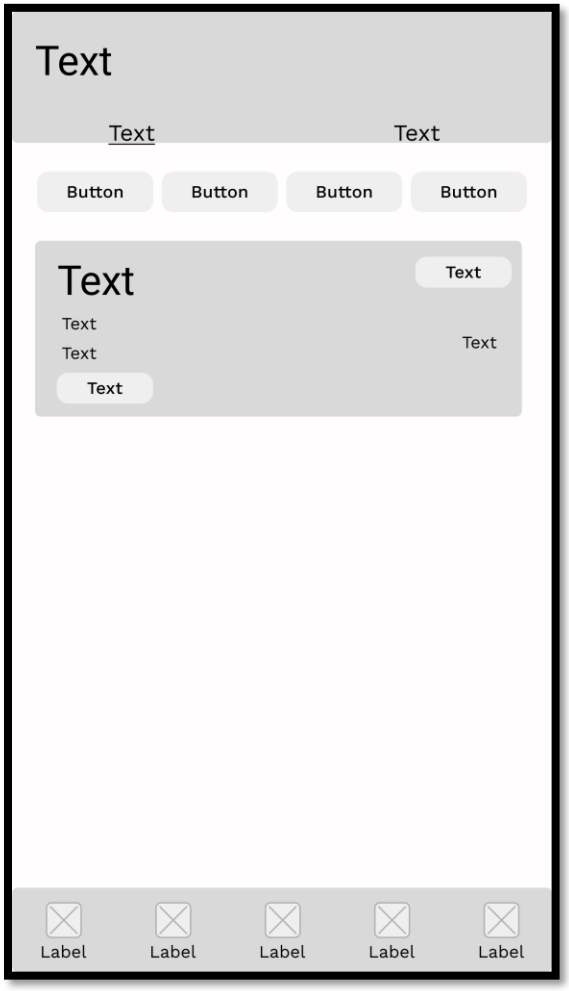


Figure 128

Equipment Provider Manage Inventory

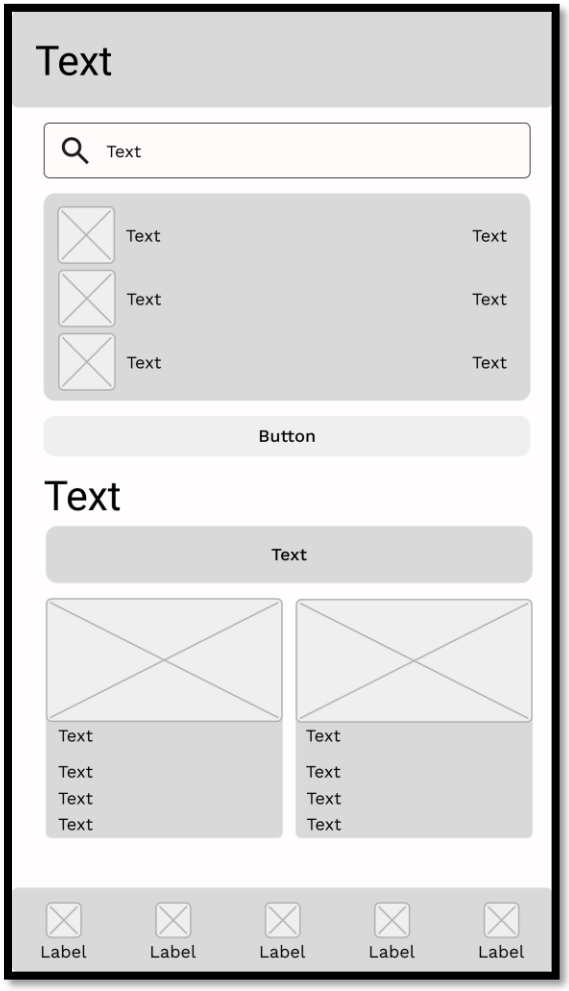


Figure 129

Equipment Provider Add New Equipment

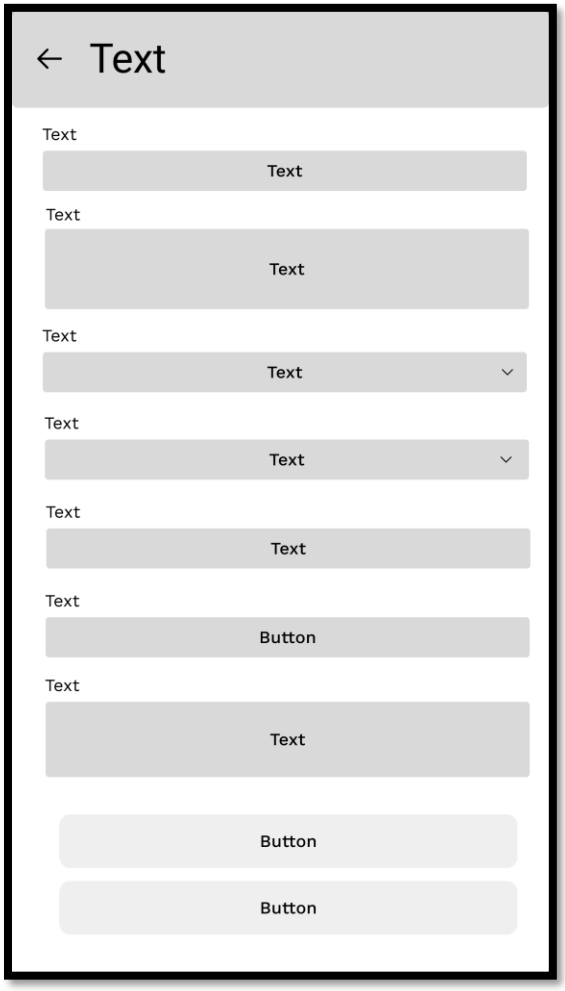


Figure 130

Equipment Provider View Equipment Details

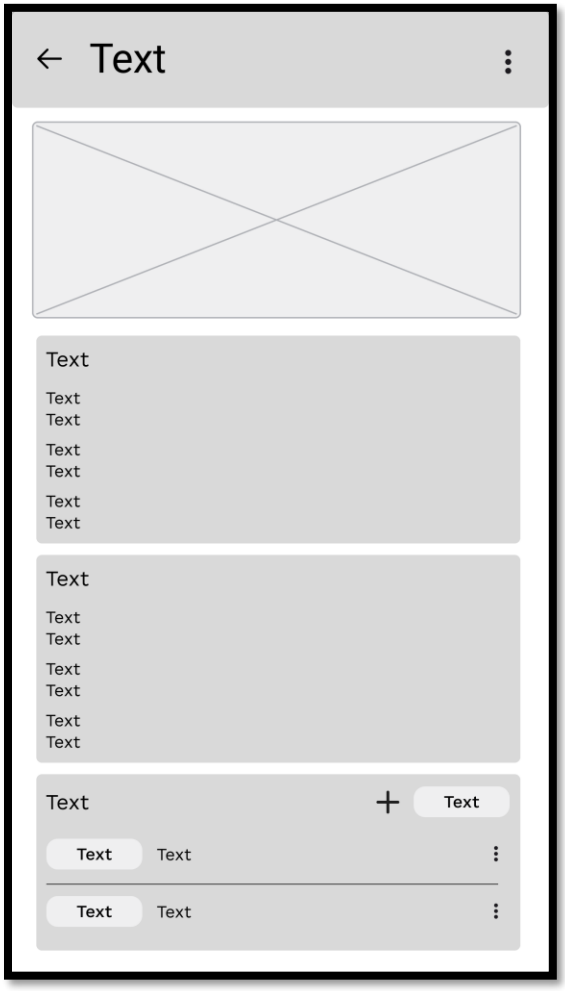


Figure 131

Equipment Provider Edit Equipment

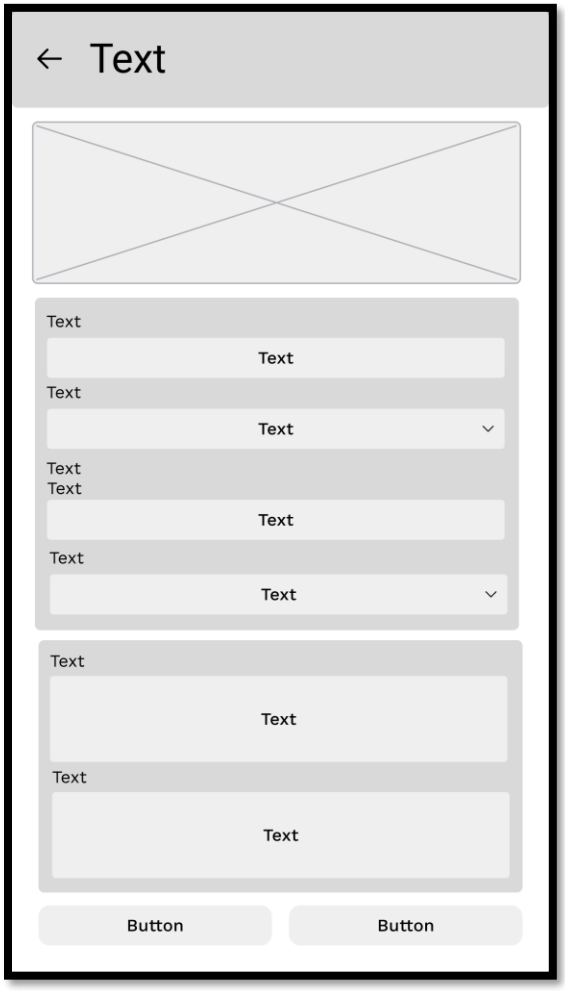


Figure 132

Equipment Provider More Tab

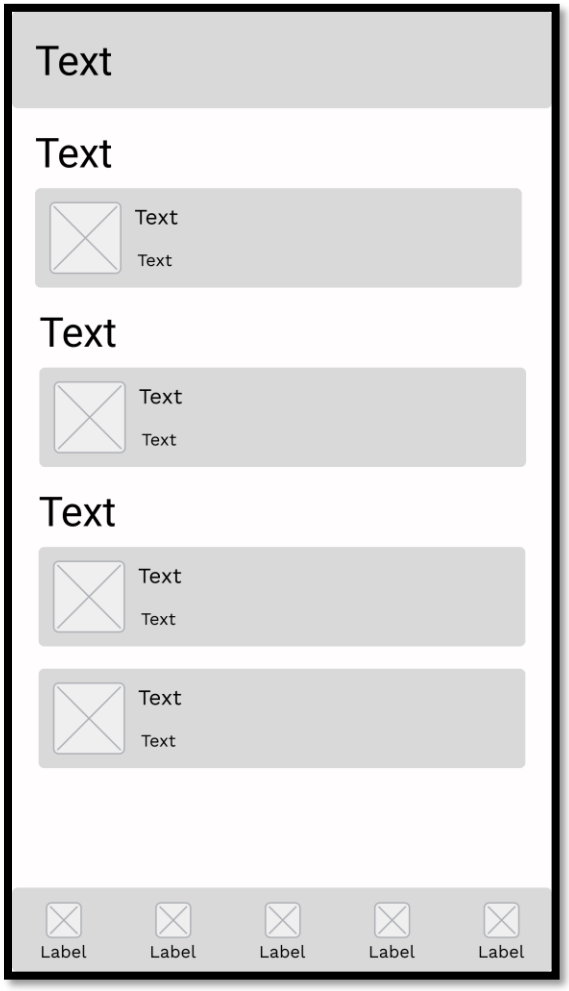


Figure 133

Equipment Provider View Voucher

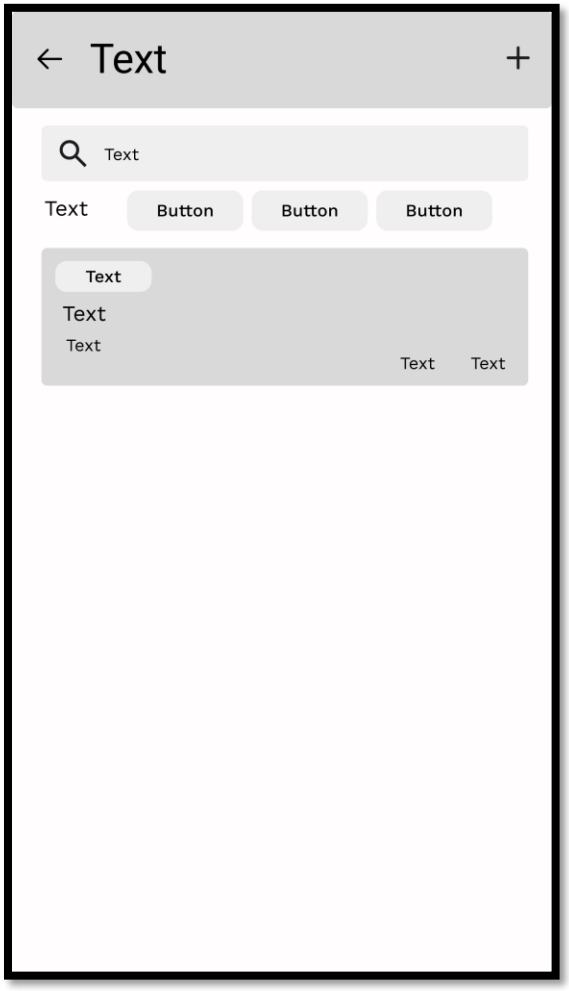


Figure 134

Equipment Provider Add / Edit Voucher

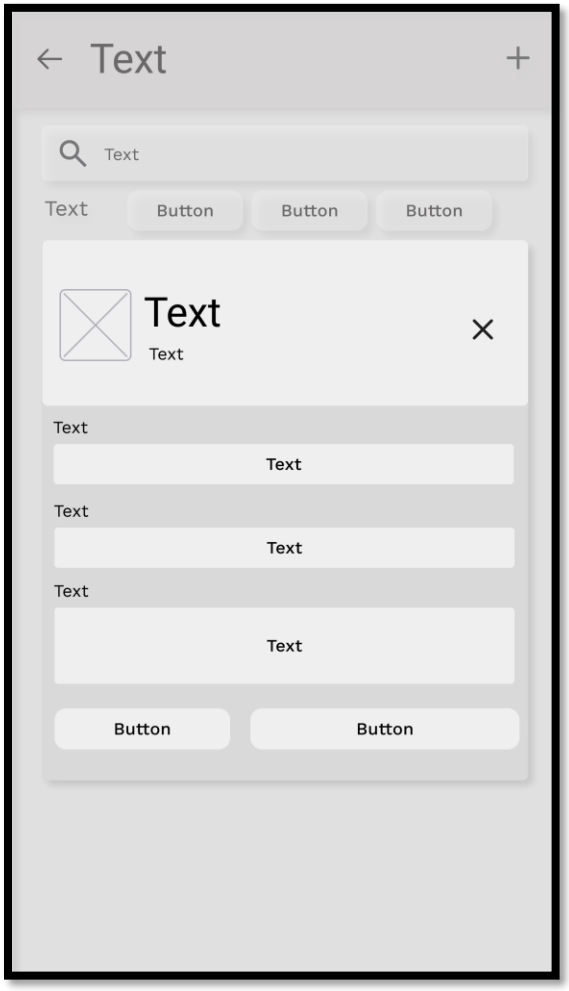


Figure 135

Equipment Provider View Review

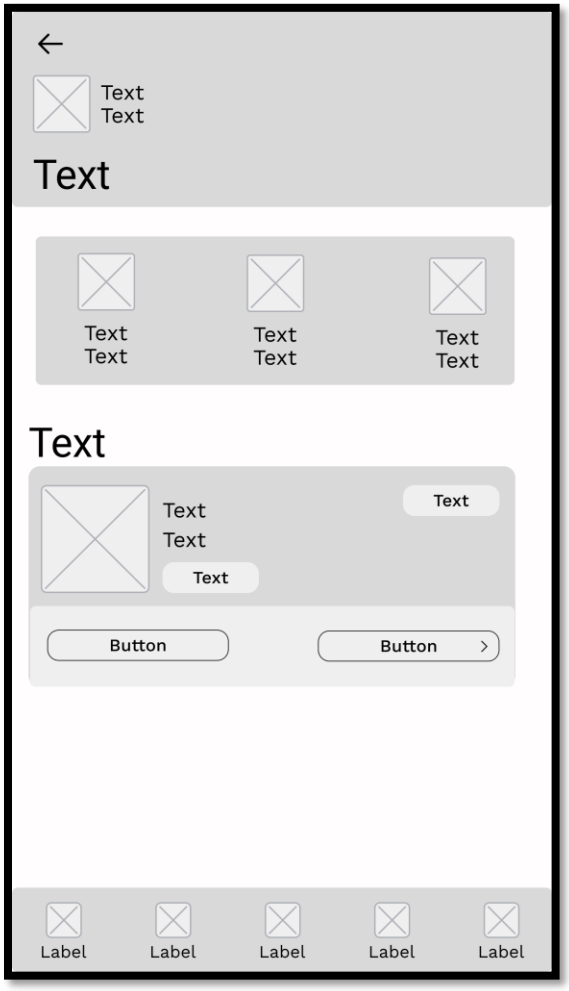


Figure 136

Equipment Provider View Review Details

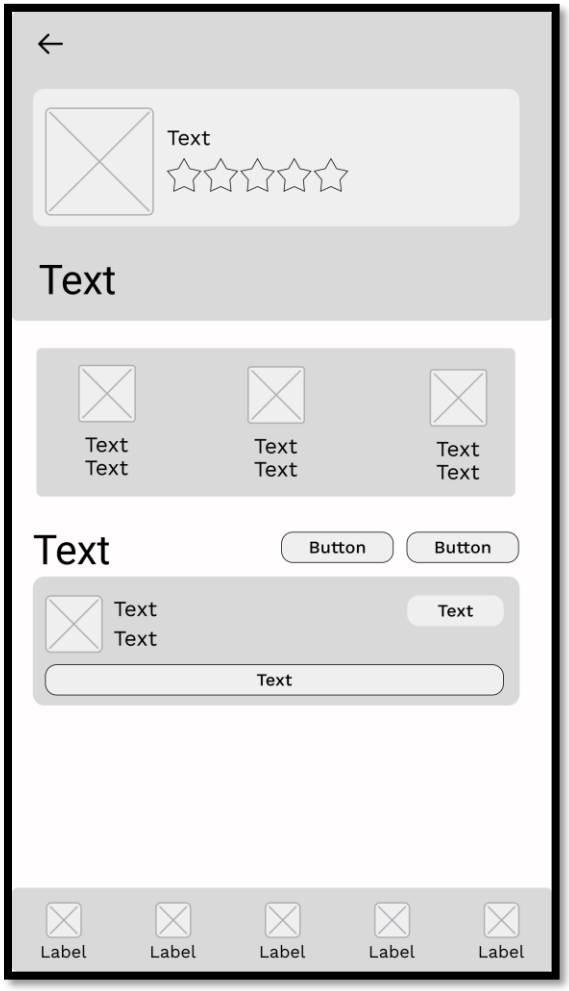


Figure 137

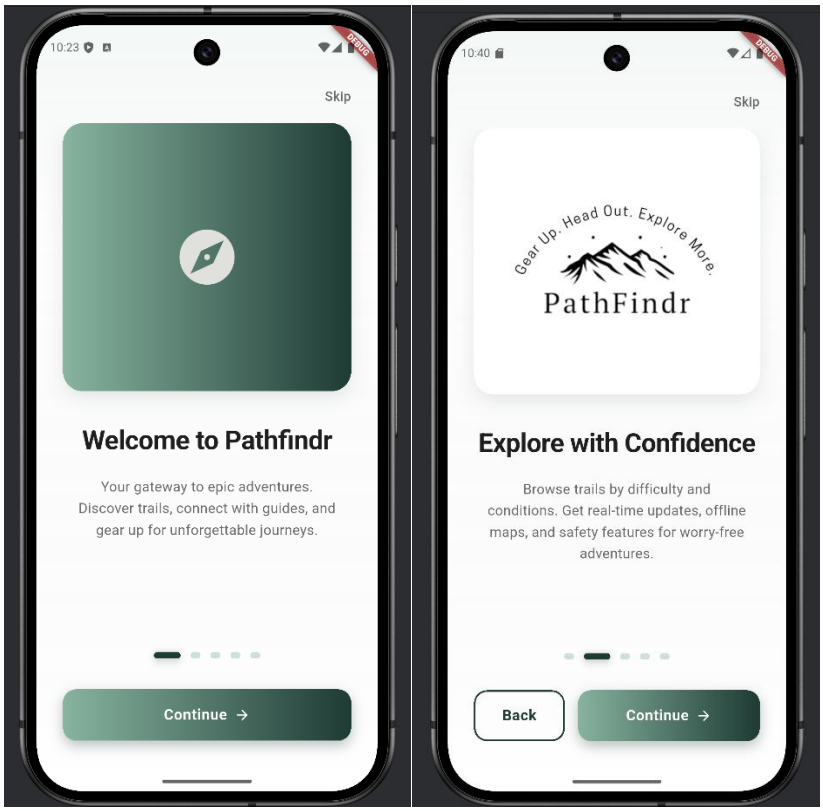
Equipment Provider View Revenue Report

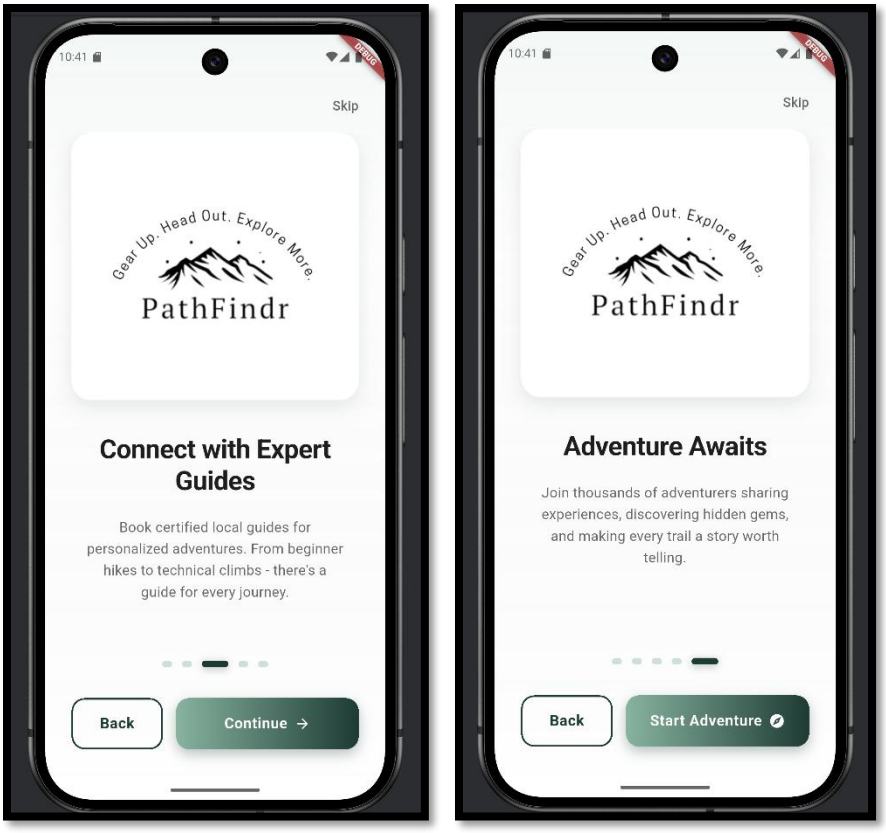
3.0 User Manual

3.1 Common Functions

Figure 138

Startup Pages

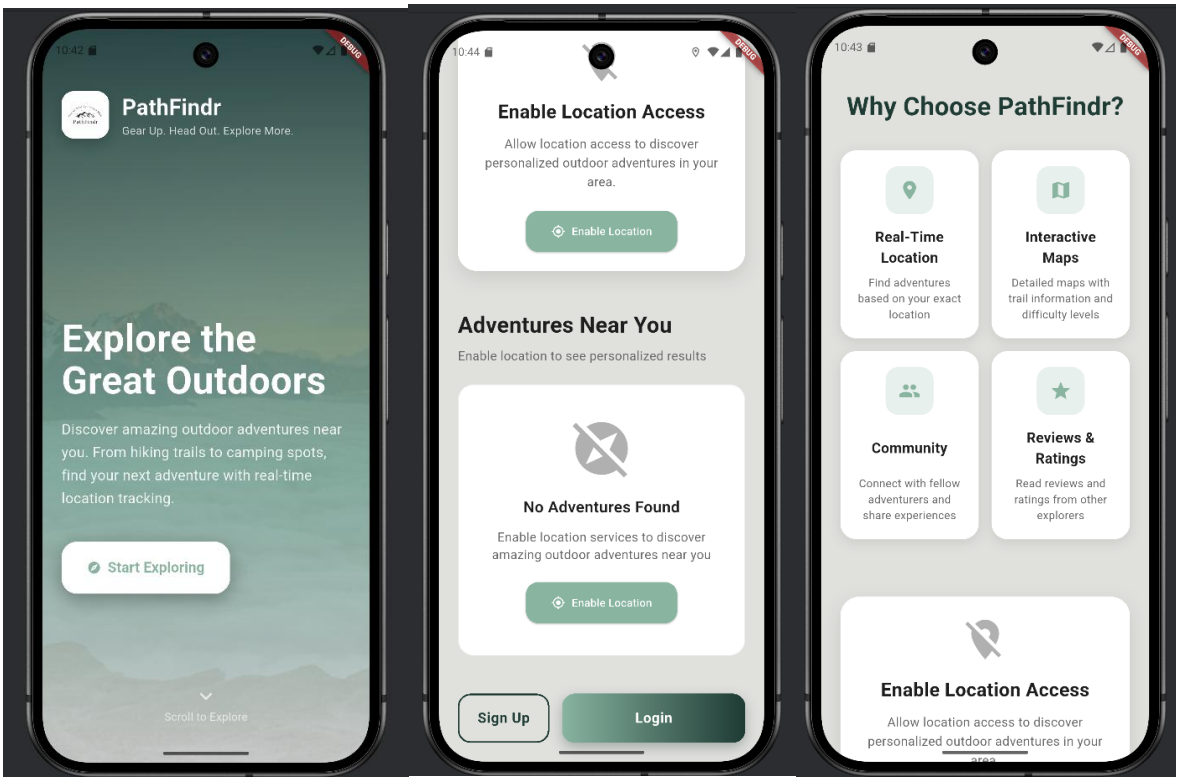


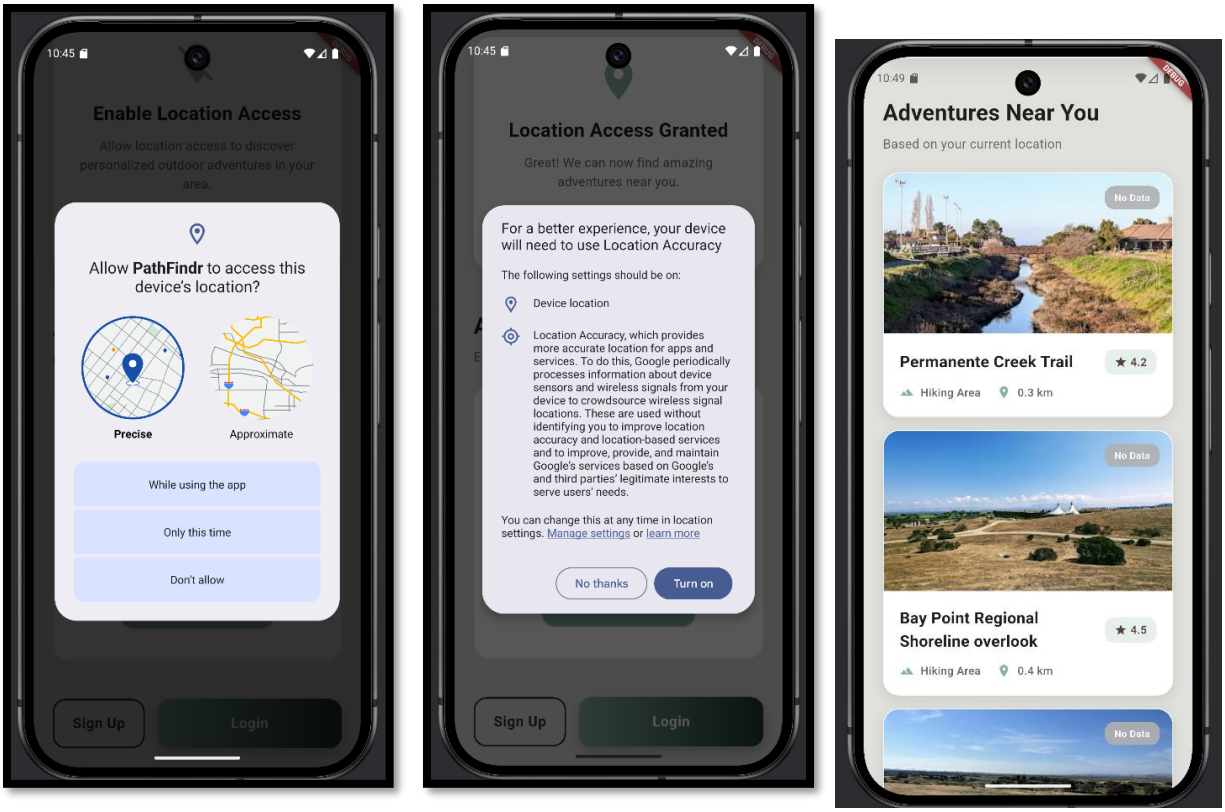


When the application is first opened, users see the welcome introduction screen. From here, they can view a brief walkthrough of the objectives and motives of the system. In addition, users can also get an overview on what the system offers.

Figure 139

Main Dashboard



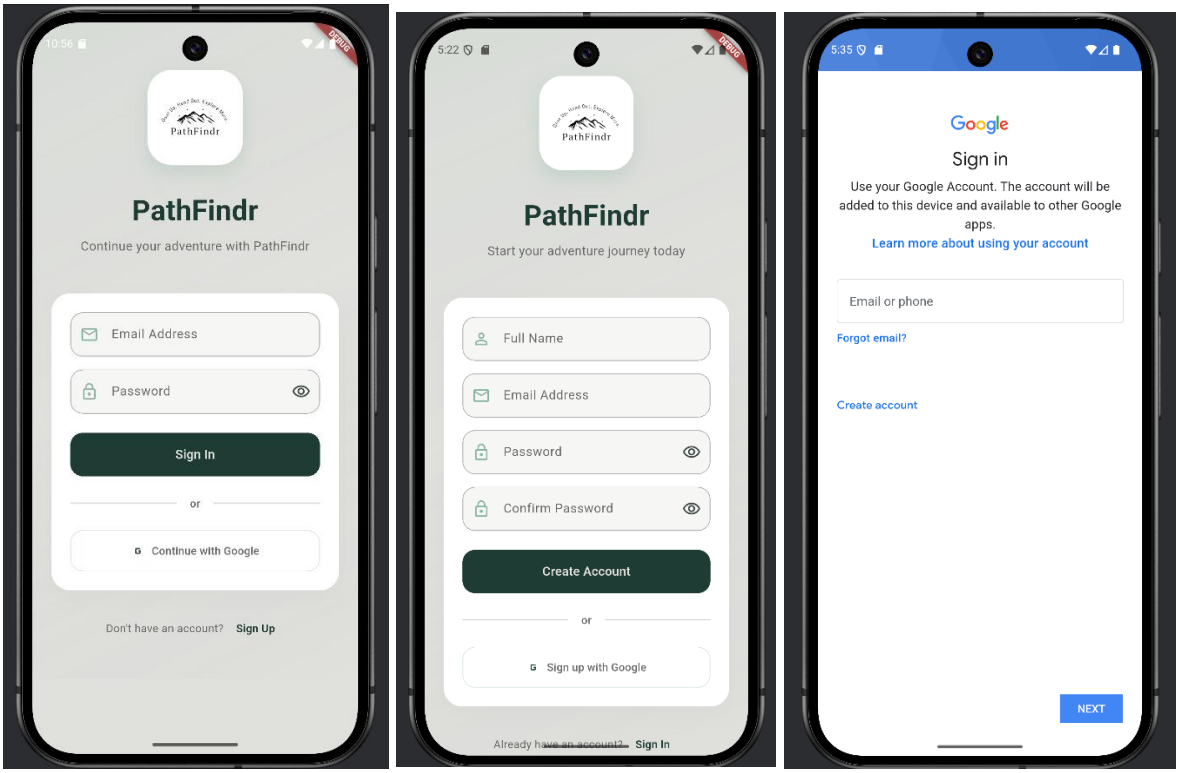


Before logging in or signing up, the dashboard displays a limited view without personalized trip information or location data. When users attempt to access nearby outdoor activities, the app request's location permission. Initially, a prompt is shown asking for access.

Once enabled, the dashboard refreshes to display nearby locations based on the user's current position.

Figure 140

Sign up and Login Page



These pages allows users to create a new account or log in to an existing one. Google login and signup is also available for quicker access.

After registering an account succesfully, users are prompt to select the role they will be handling within the app. The selection will then guide them through different onboarding processes to gather more customized details of the user in terms different informations, experiences and preferences.

Figure 141

Explorer Role Setup Page

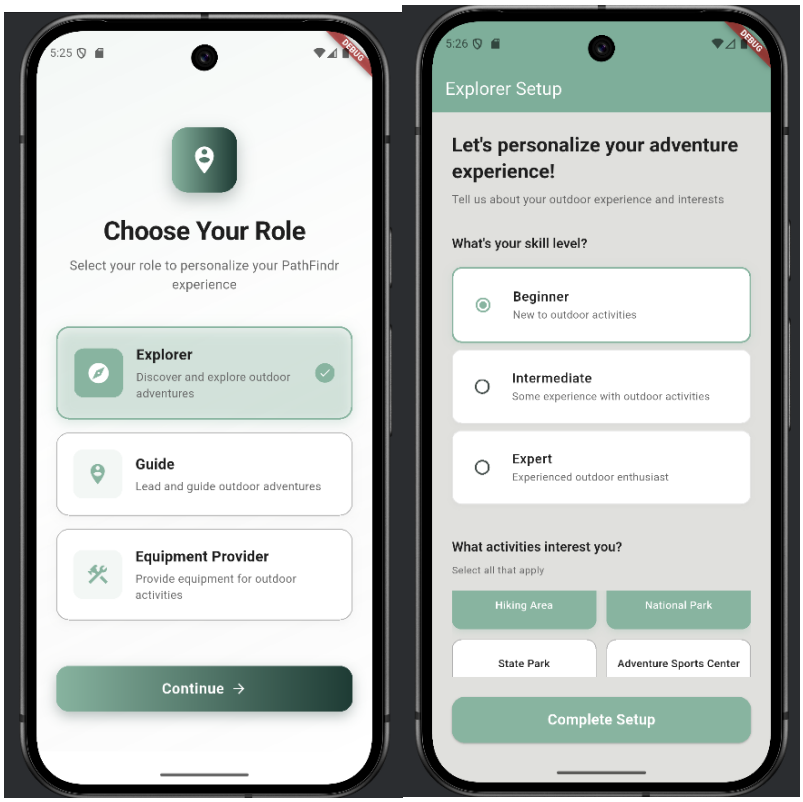


Figure 142

Guide Role Setup Page

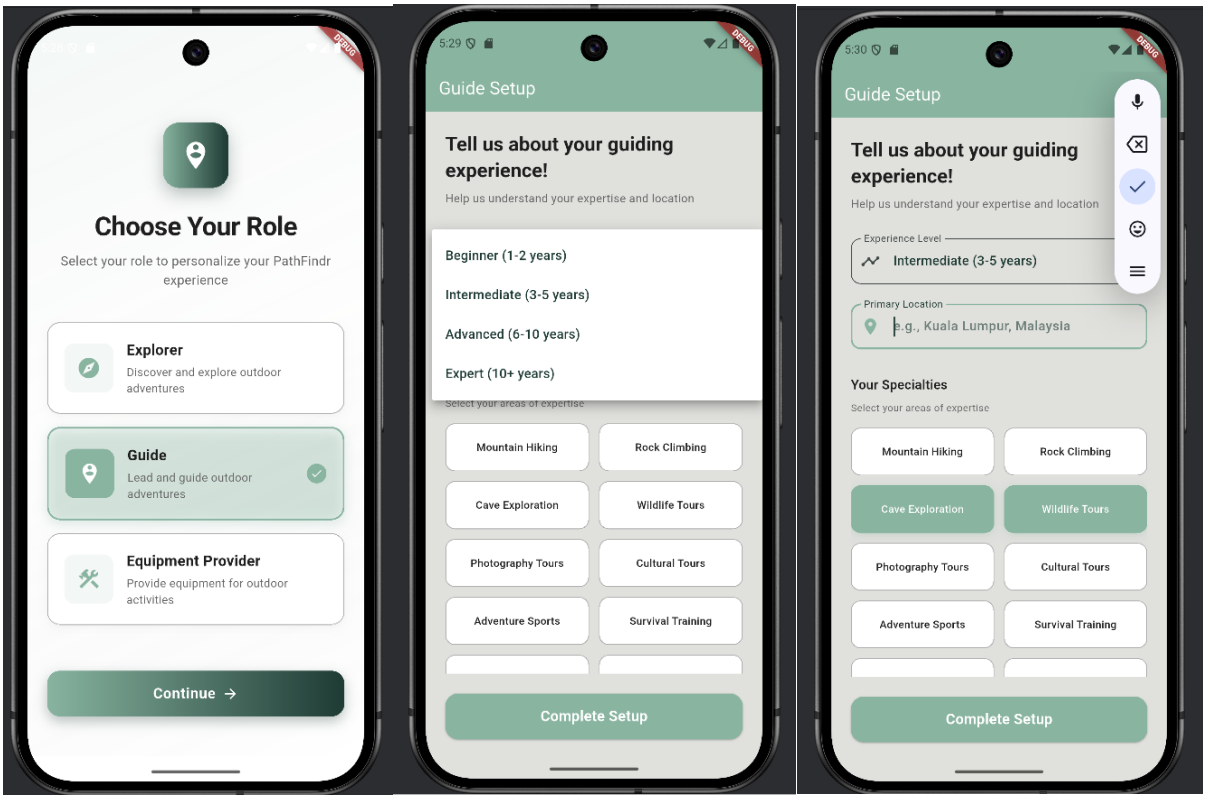
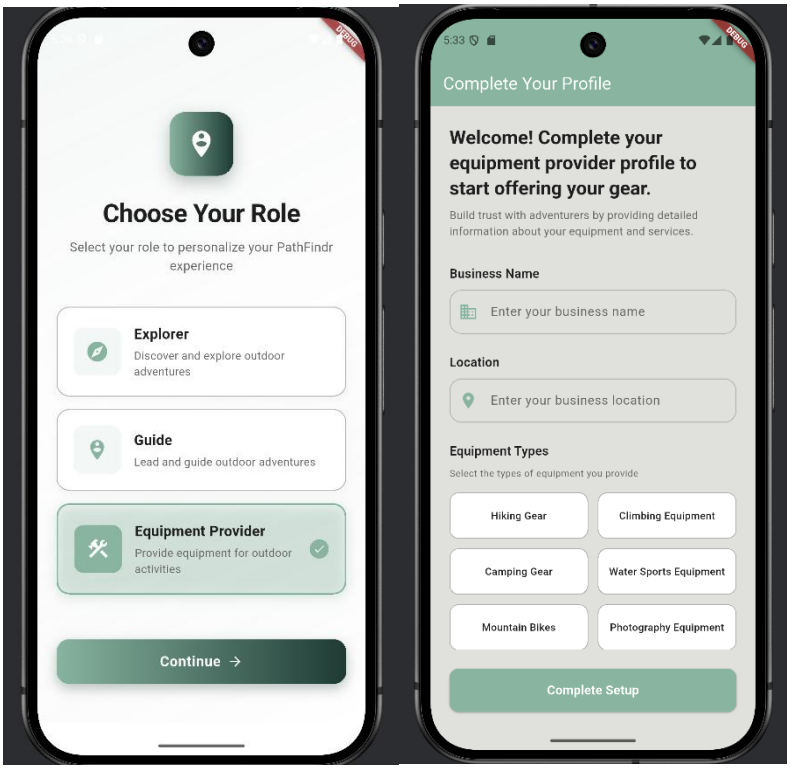


Figure 143

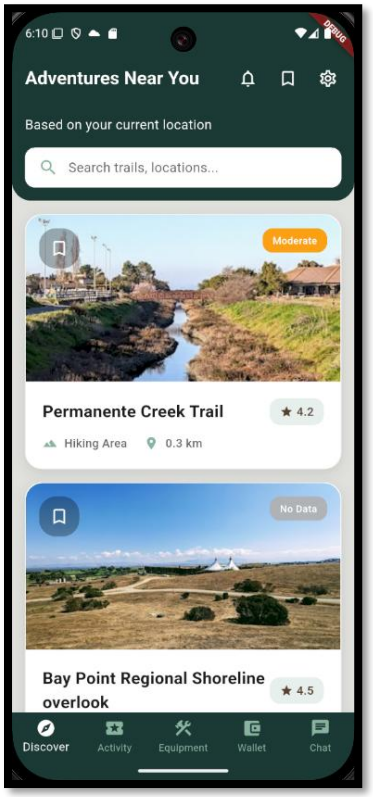
Equipment Provider Setup Page



3.2 Explorer Role

Figure 144

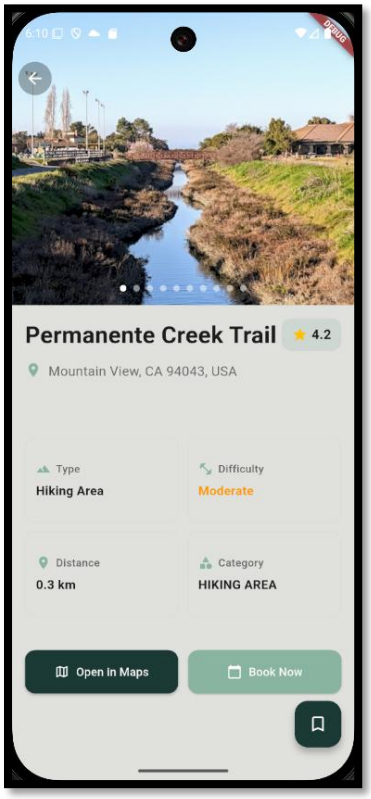
Main Dashboard



The main dashboard for the Explorer role allows the user to grant location access to the system. Once access is given, the system will display nearby outdoor locations based on the user's current location. The user will be able to view each location’s name, type, distance, difficulty, and rating. Additionally, the user can use the search bar to look for specific locations. If the user wants to save a location, they can click the "bookmark" icon at the top left corner of the location card. All saved locations can then be accessed by clicking the "bookmark" icon located in the top bar.

Figure 145

View Location Details Page



When the user clicks on a specific location card, they can view details such as the location name, address, rating, activity type, difficulty, distance, and category. If the user wants to view the location in more detail, they can click the "Open in Maps" button to access it in Google Maps. If the user requires a guide for the trip, they can click the "Book Now" button.

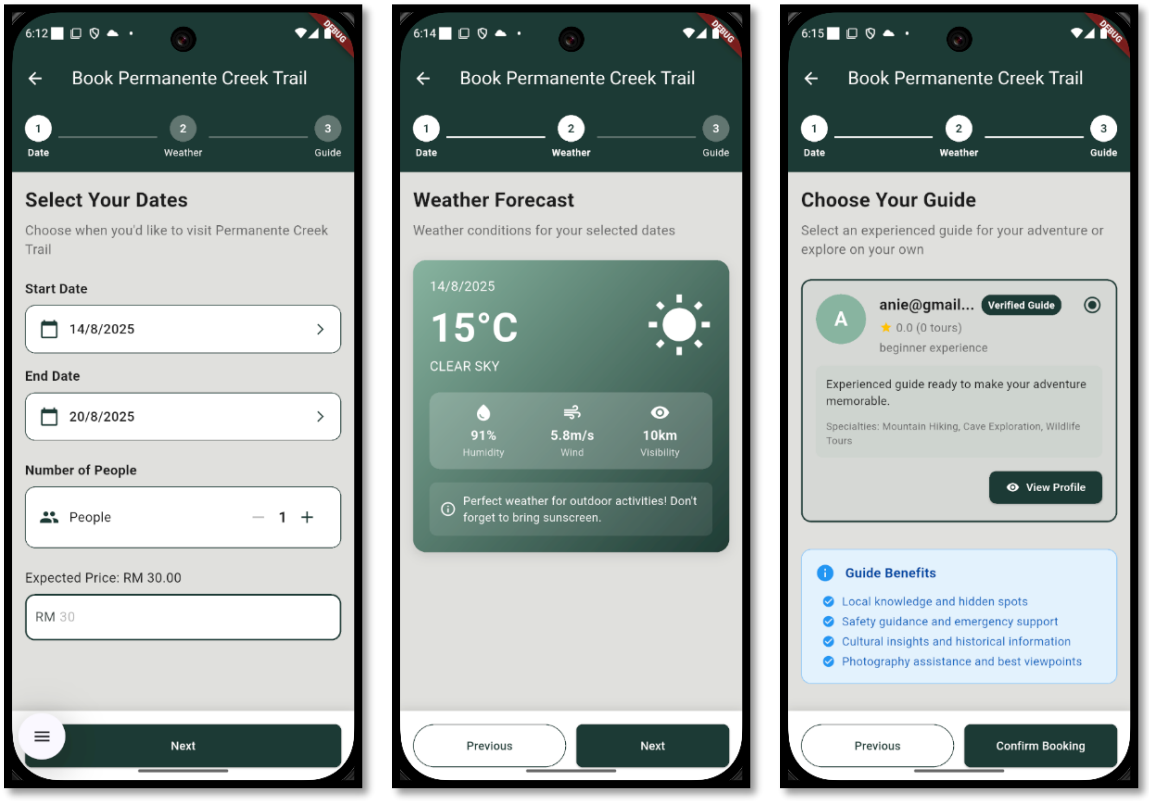
Figure 146

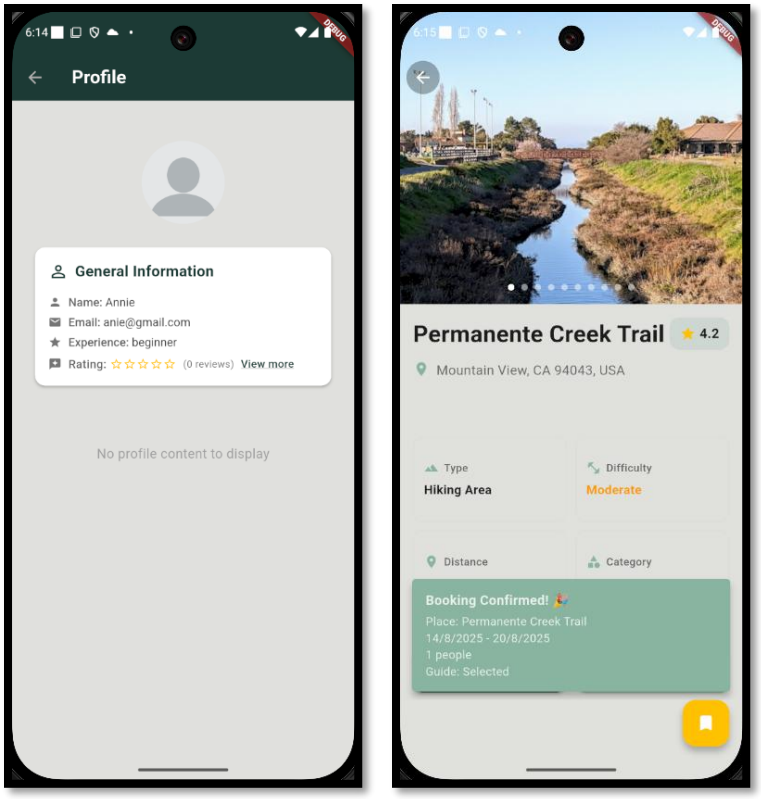
Notification Page

As mentioned in the Explorer - "Main Dashboard", the user can add locations to the watchlist for future reference. When a location is added to or removed from the watchlist, the user will receive a notification, which can be viewed here.

Figure 147

Book New Event

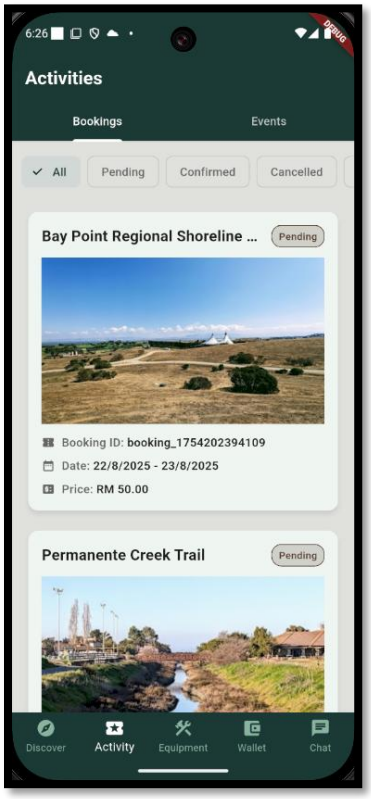




When the user clicks on the "Book Now" button, they will be directed to the booking function, where they are required to fill in details such as the start date, end date, number of people, and expected price. After clicking the "Next" button, the user can view the weather forecast to help them decide whether to proceed with the booking. If they choose to continue and click "Next" again, they will be presented with a list of available guides. The user can also click the "View Profile" button to see more details about a guide, including their general information, safety information, and the checklist provided by the guide.

Figure 148

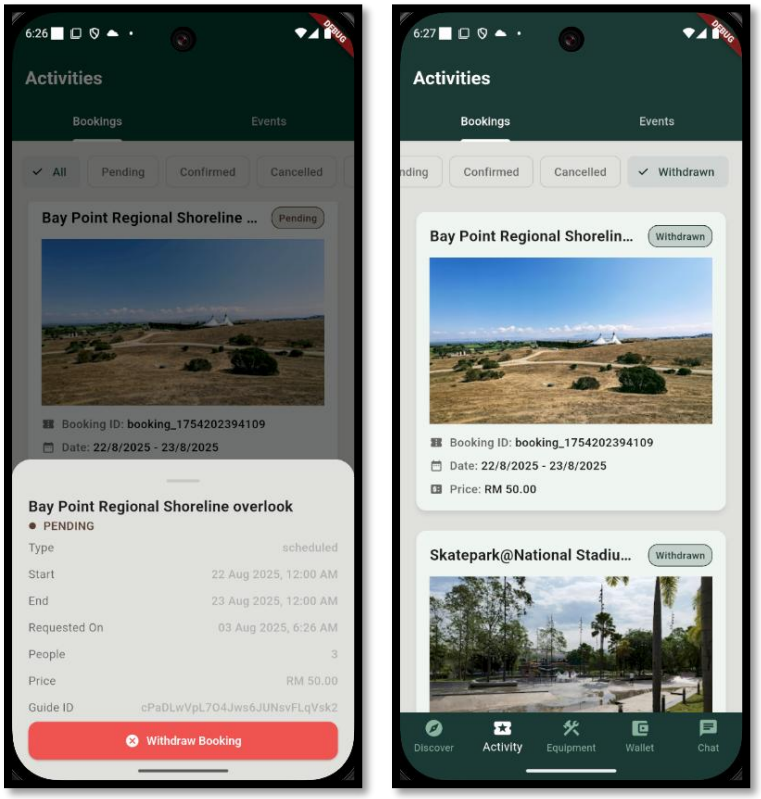
View Bookings



When the user clicks on the "Order" option from the bottom navigation bar, two categories will be displayed: "Booking" and "Events". The "Booking" category represents trips that have been booked with a guide but are still awaiting confirmation. The "Events" category represents trips that have already been confirmed by the guide and will be carried out according to the scheduled time. Then, users can filter bookings by different statuses: All, Pending, Confirmed, and Cancelled.

Figure 149

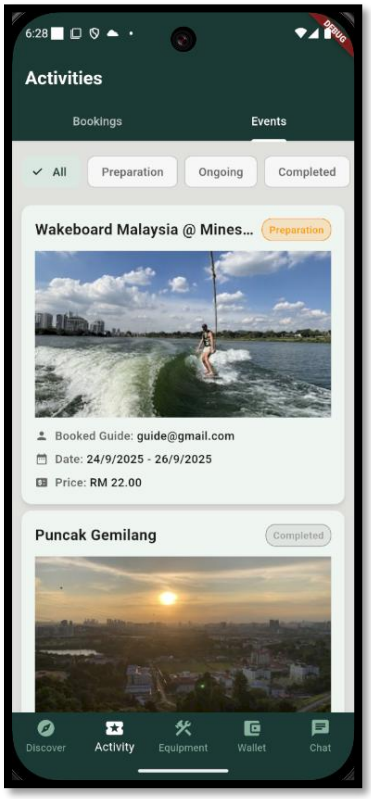
Withdraw Bookings



In the "Booking" section, When the Explorer clicks on a specific activity, they will be able to view the booking details. If the user wishes to cancel the booking, they can click the "Withdraw Booking" button. Then, the specific booking will be moved to the “Cancelled” section.

Figure 150

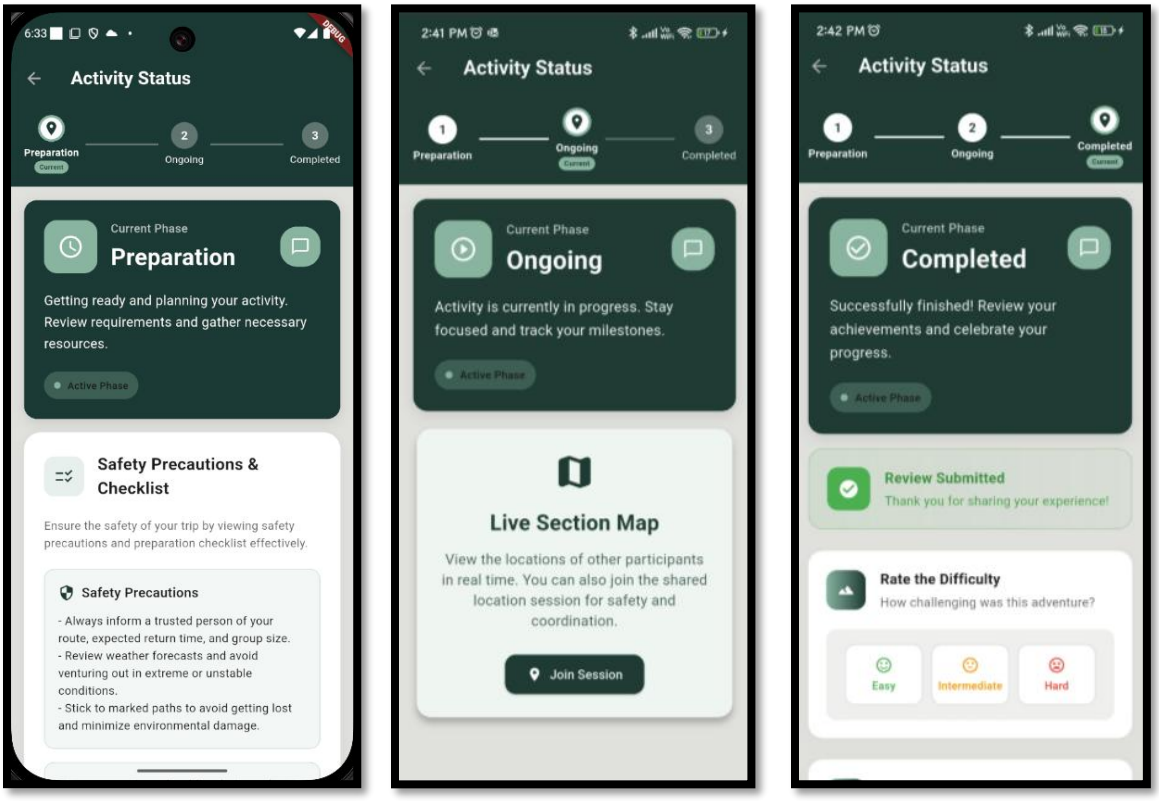
View Events

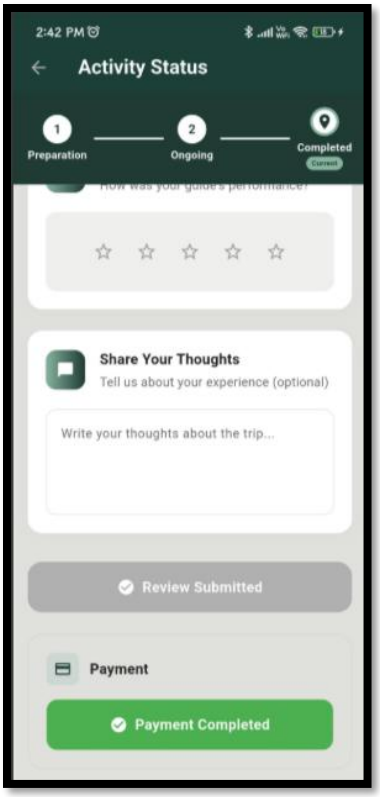


In the "Events" section, users can also filter by different statuses: All, Preparation, Ongoing, and Completed. When the user clicks on a specific activity card, they will be able to view the event details.

Figure 151

View Events Details





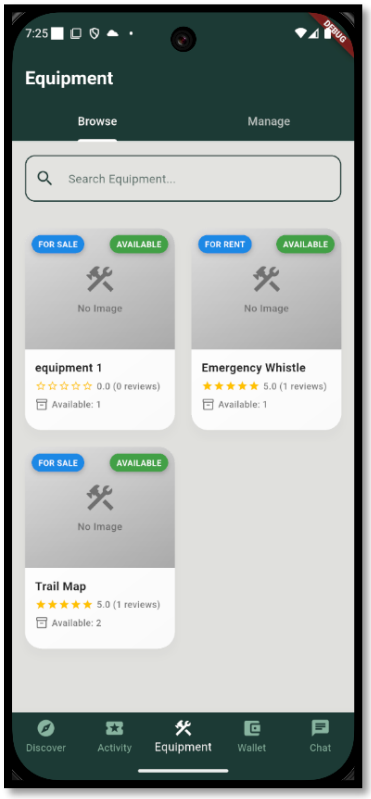
When the user clicks on the event details, they can view the activity status. The session will also be joined by the guide and other Explorers participating in the event.

In the "Preparation" stage, the user can view safety precautions and a checklist provided by the guide. In the next phase, "Ongoing", the user can join a live session map to view the real-time locations of all members. If an emergency occurs, the user can click on the emergency icon to notify all participants.

In the "Completed" stage, the user can provide feedback on the difficulty level, the guide, and their overall experience. Additionally, the user can make the payment by clicking on the "Make Payment" button.

Figure 152

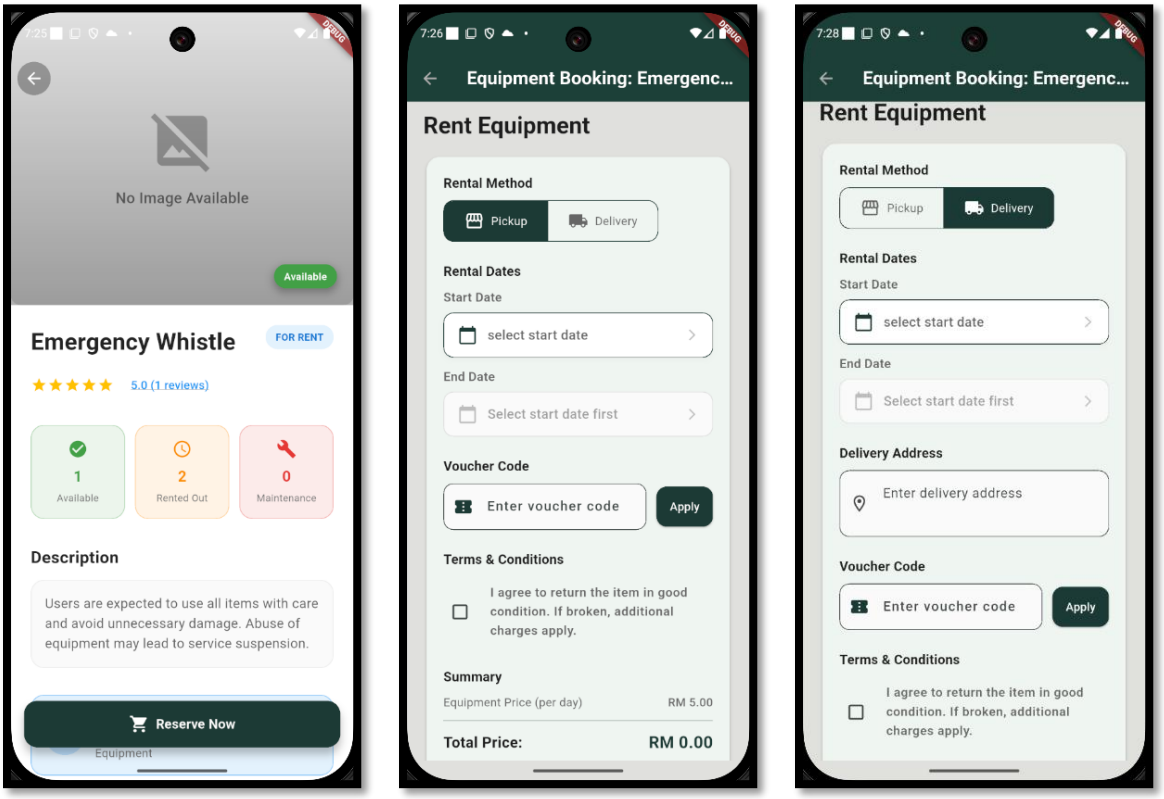
View Equipment



When the user clicks on the "Equipment" option in the bottom navigation bar, they can browse through the available equipment. If the user wants to search for specific equipment, a search bar is provided to help them find it easily.

Figure 153

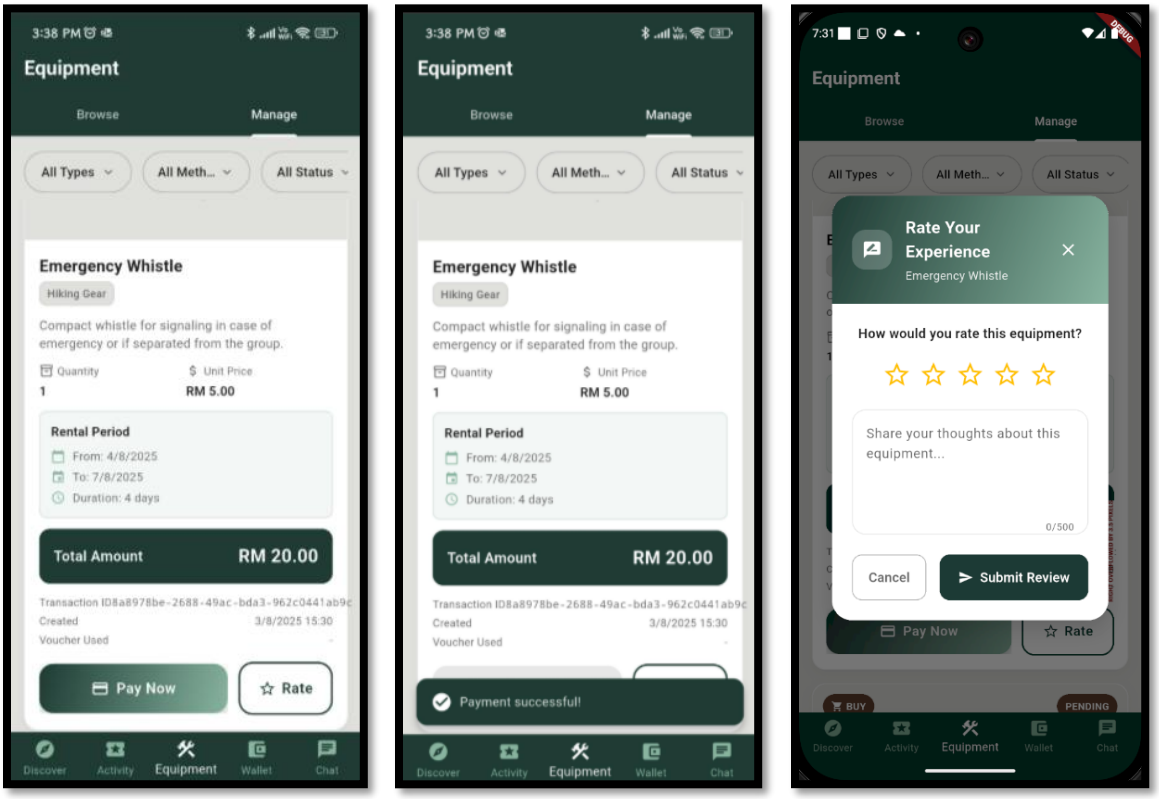
Purchase / Rent Equipment



When the user clicks on a specific piece of equipment, they can view its details, including the equipment name, description, ratings, and the number of units available. After clicking the "Reserve Now" button, the user can then select their preferred delivery method, either pickup or delivery. If the delivery option is chosen, the user will be required to provide a delivery address. The user can also apply a voucher code if they have one. After reviewing the total price and agreeing to the terms and conditions, the order will be sent to the equipment provider for processing.

Figure 154

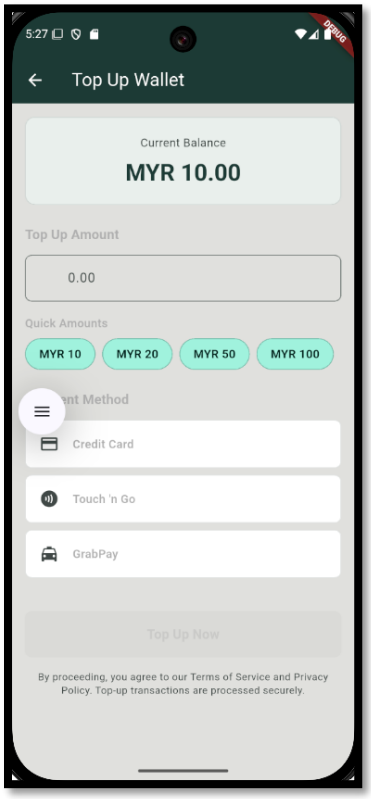
Manage Order



On the other hand, when the user clicks on the "Manage" option, they will be able to view all booking records. Users can sort these records based on transaction type, delivery method, and order status. For each order card, users can view the order details and payment status. They can also provide a rating by clicking the "Rate" button.

Figure 155

Topup Wallet

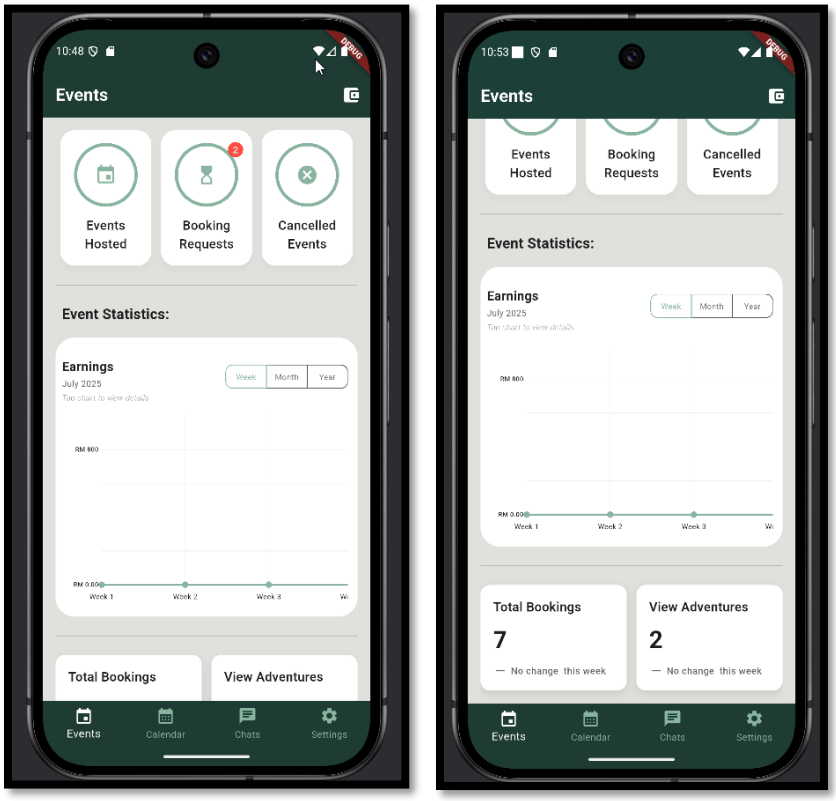


Unlike Guides and Equipment Providers, Explorers are not able to withdraw money. However, they can top up their balance using a credit card, Touch 'n Go, or GrabPay. After selecting the desired top-up amount, choosing a payment method, and clicking the "Top Up Now" button, the user's balance will be updated.

3.3 Guide Role

Figure 156

Main Dashboard



The main dashboard for guide below shows a summary of the guide's activities.

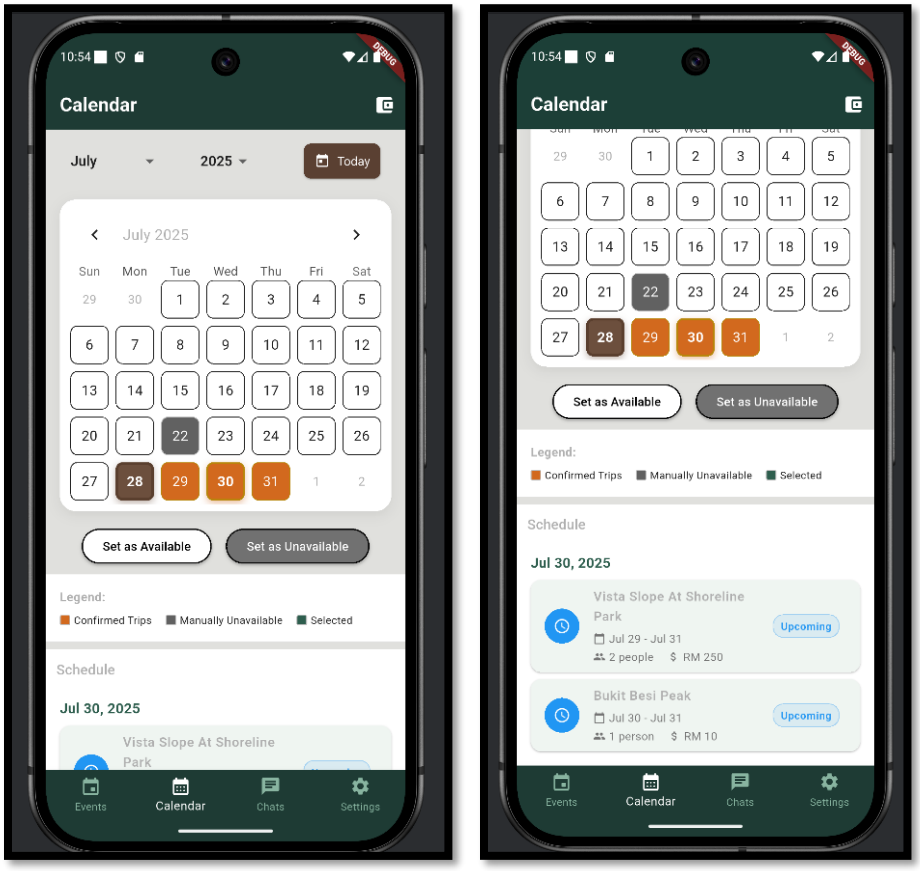
The top includes 3 cards on top showing several shortcuts to navigate to the adventures page and manage bookings page based on pending requests or cancellation requests. A red dot showing the count of new requests are displayed on top ensures guides are notified and up to date. After clicking into the card, the red dot disappears, signifying that the requests are read.

In the event statistics, an analytics chart of the guides earnings in week, month or year is provided. By clicking on the corresponding dot in the x axis, the exact amount earned for that week can be seen.

In the bottom 2 widgets, the total count for bookings and adventures are displayed. Changes of increasing or decreasing of bookings or adventures are calculated every week. In addition, clicking on the widgets also leads to the corresponding page for manage bookings and adventures page.

Figure 157

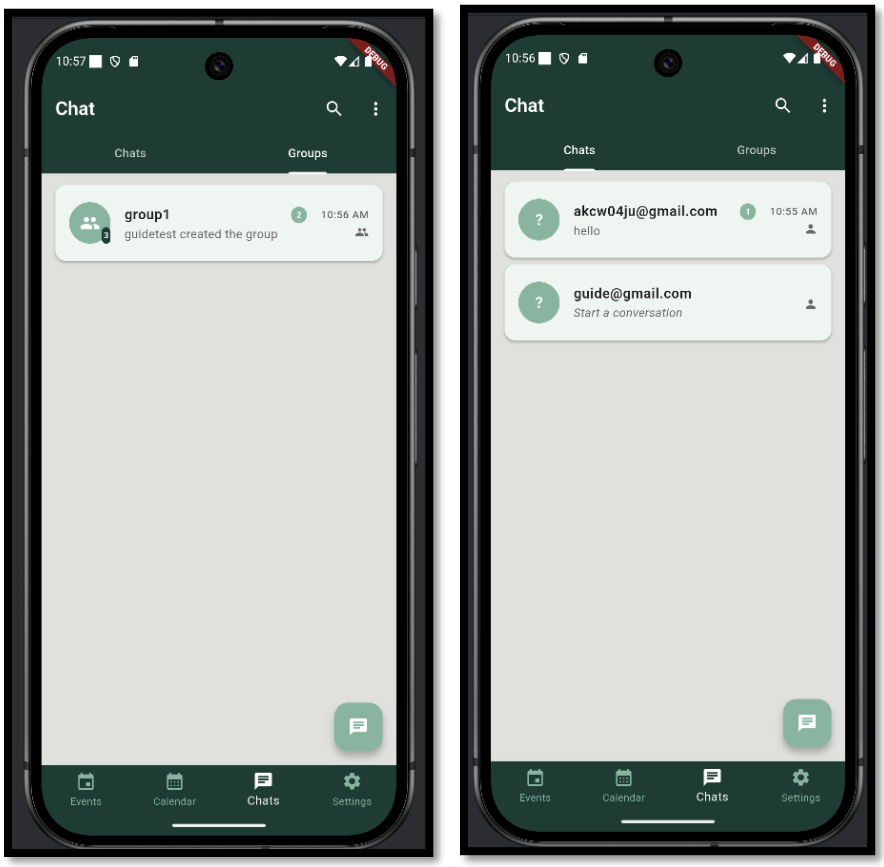
Calendar Page



Guides can use the calendar to manage their schedule by marking days as available (white) or unavailable (grey). This ensures bookings cannot be made during unavailable periods. Accepted trips are automatically set as orange and availability settings are disabled. By clicking on the dates, the schedule shows the trip details that are under the dates.

Figure 158

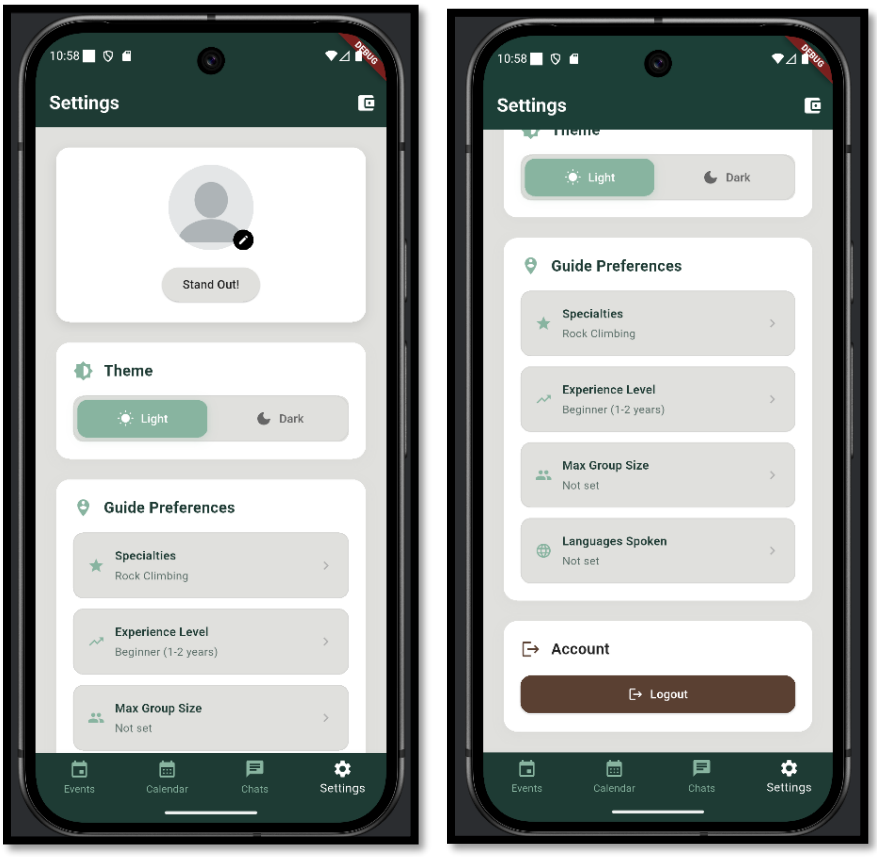
Chat Page

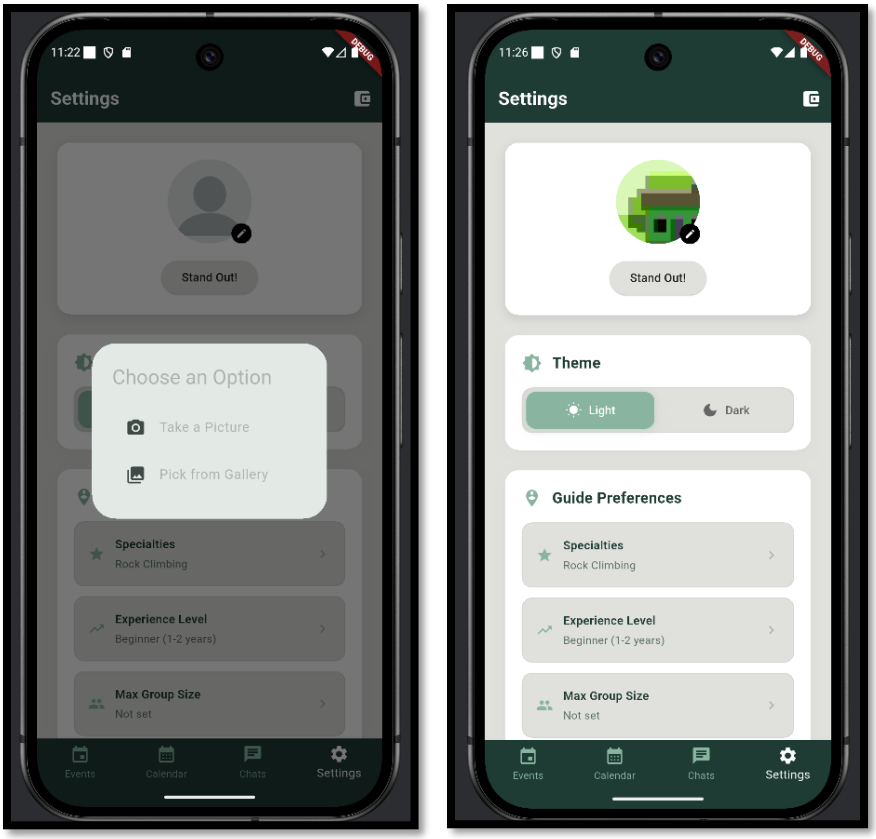


The chat page allows guides to communicate directly with explorers and participants, helping them plan trips, provide updates, help or answers questions. Guides can search for the particular email address to text to or create a group to chat in batch.

Figure 159

Settings Page



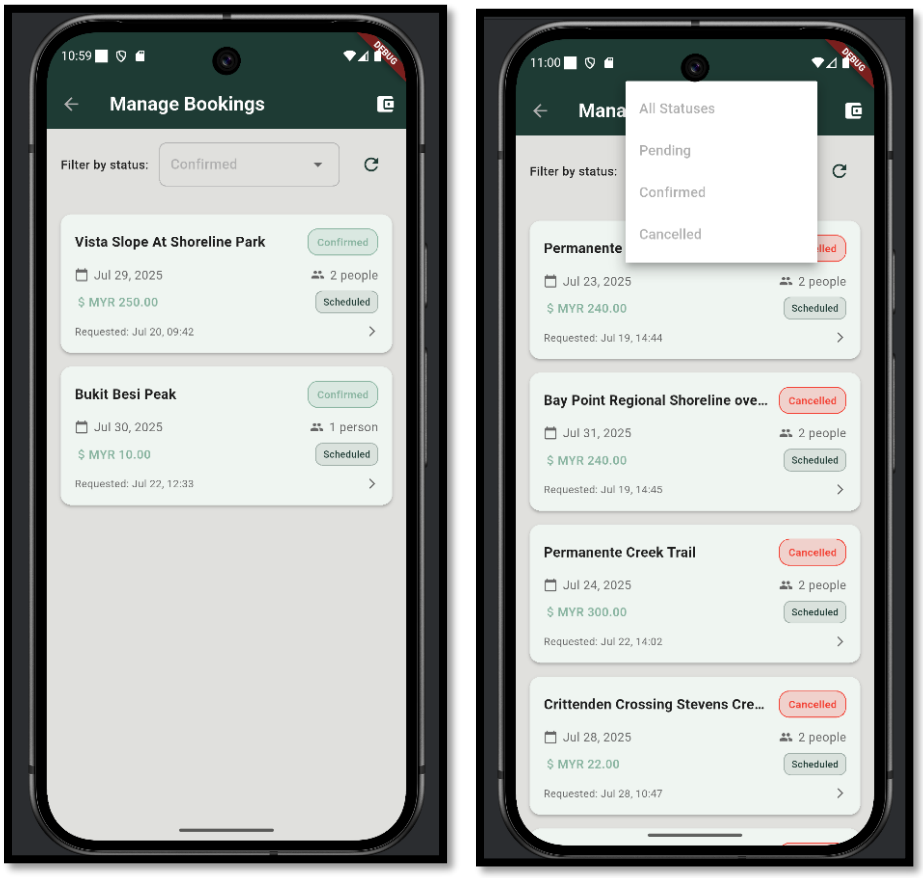


In the settings page, guides can adjust preferences and update their profile picture to maintain an up-to-date personal profile. The preferences chosen during guide sign up process can be changed here, including theme like light mode and dark mode.

Meanwhile, two choices are given for guides to select, uploading a picture or taking a photo with camera.

Figure 160

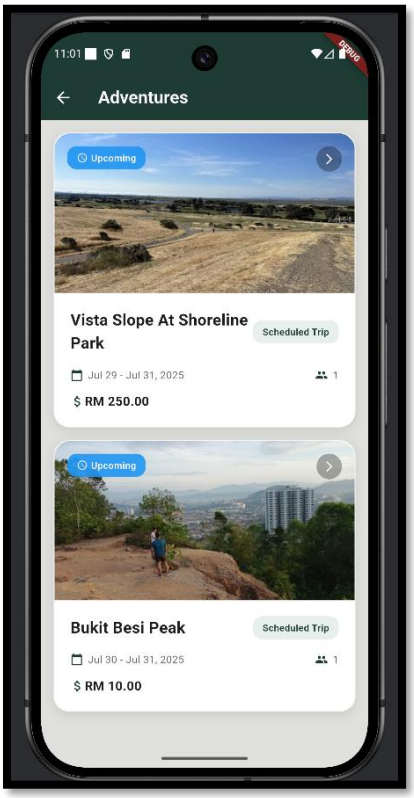
Manage Bookings Page



This page lists booking requests from explorers. Guides can filter these requests by status including all the statuses, pending, confirmed, or cancelled as needed.

Figure 161

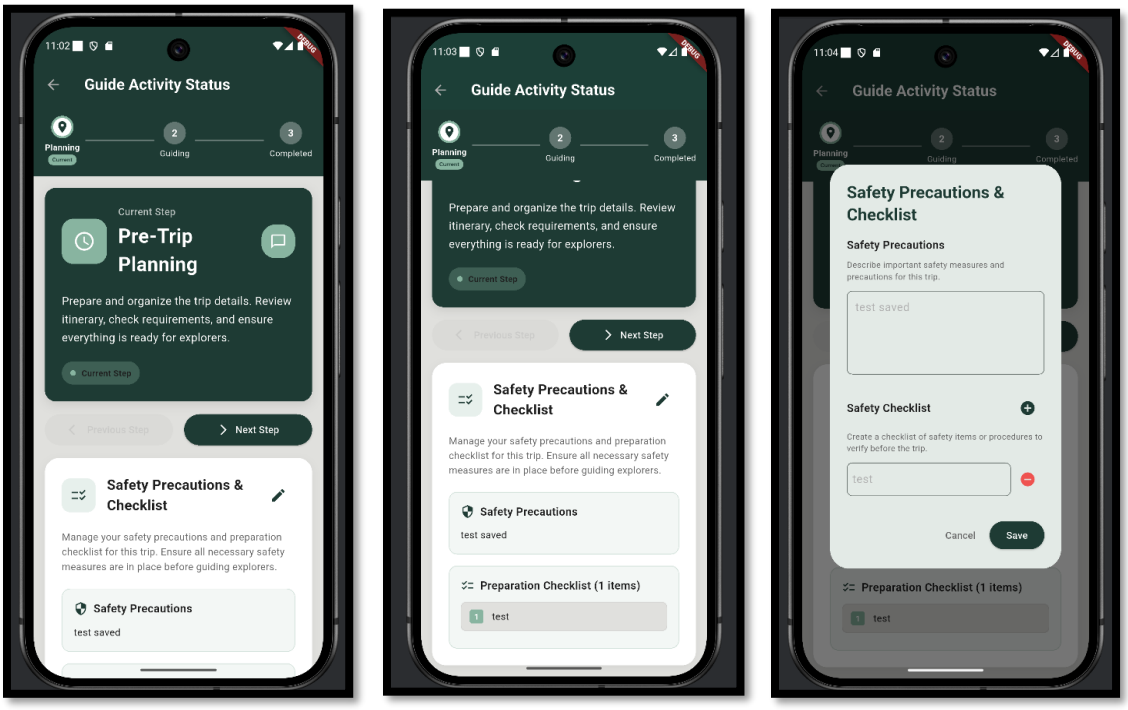
View Events / Adventure Page



This section shows all confirmed trips hosted by the guide in card views. The adventures are organized into different phases: planning, ongoing, and completed following the set dates of the trip, allowing guides to track progress easily.

Figure 162

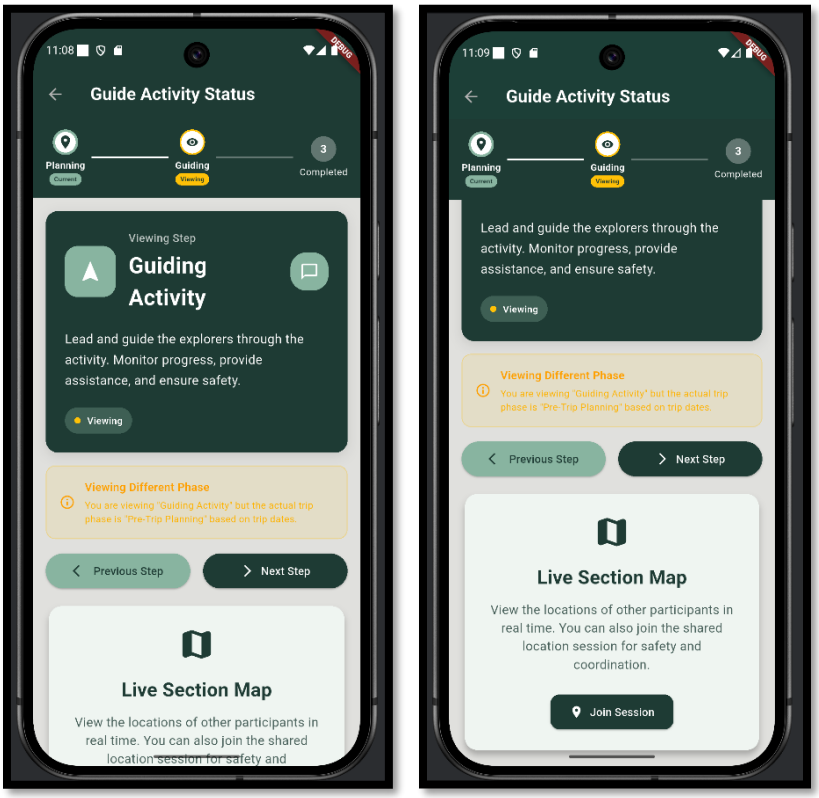
Trip Phase Page: Planning Phase

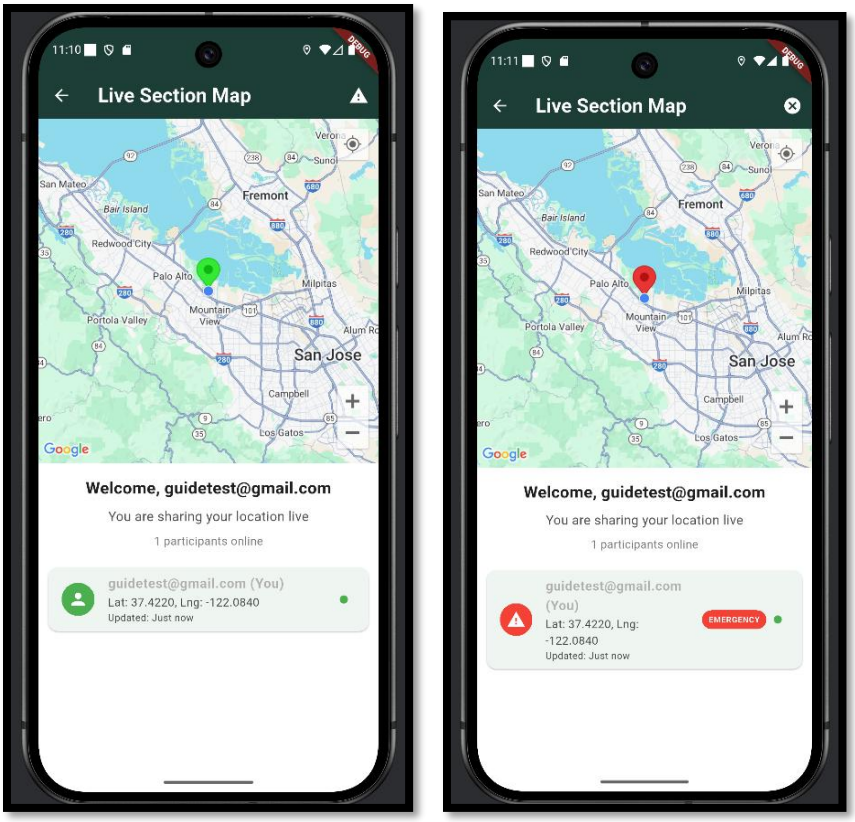


During the planning phase, guides can add or edit safety precautions and checklists. They can also enter the pre-trip planning chat to discuss details with explorers.

Figure 163

Trip Phase Page: Guiding Phase

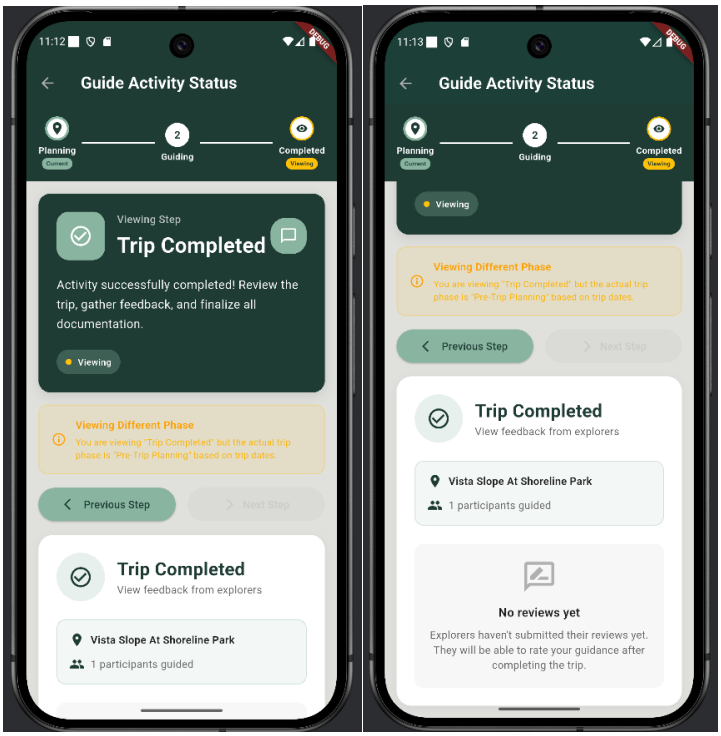




In the guiding phase, guides can join a live session map to track participant locations in real-time. An emergency alert feature is also available to notify participants instantly in case of issues.

Figure 164

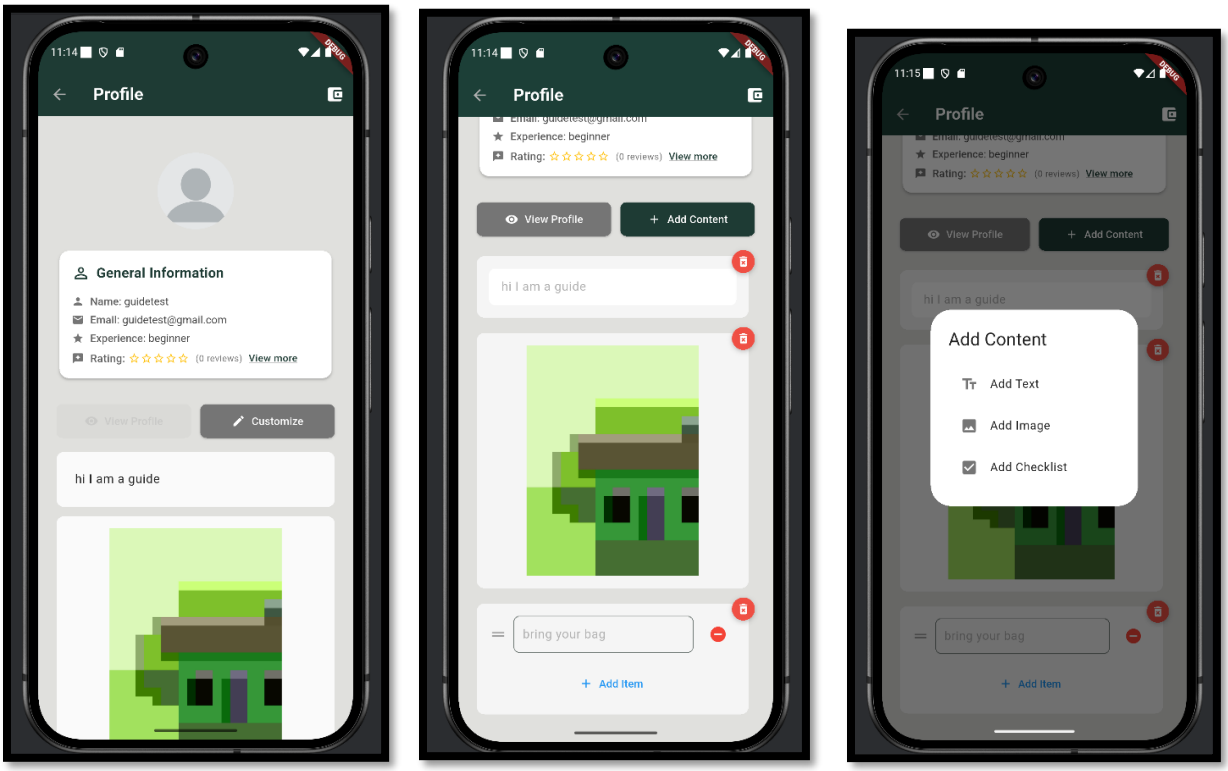
Trip Phase Page: Completed Phase



Once an event is completed, feedback and ratings from participants can be viewed in this phase.

Figure 165

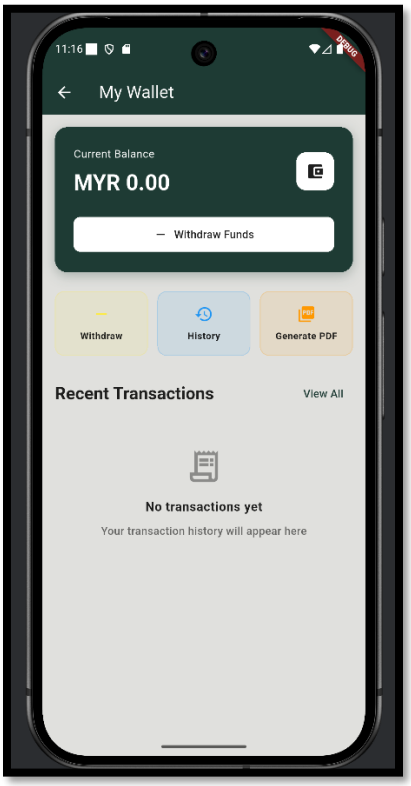
Edit Profile



This page is where guides can customize their personal and professional information so that they can stand out from other guides. Explorers can view this page when selecting guides to lead trips. Meanwhile, guides can choose to upload different formats like text based, images or checklists

Figure 166

Guide Wallet



This section is located under the wallet icon on the top right corner of every page, it allows guides to view their total earnings, withdraw funds, and review past transactions. A PDF transaction report can also be generated for record-keeping.

Figure 167

Wallet – Withdraw Funds

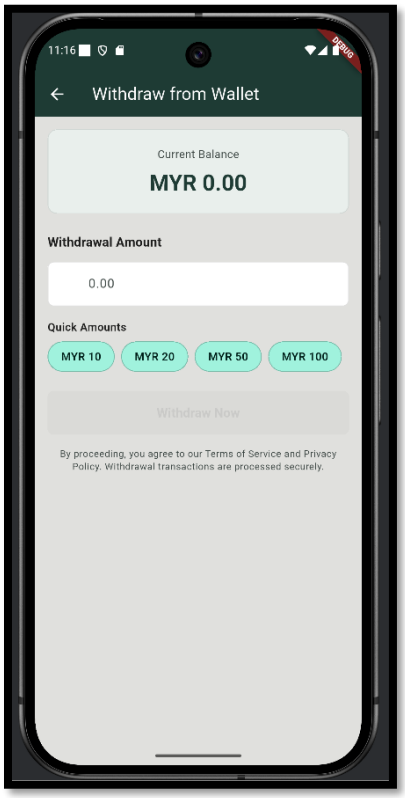


Figure 168

Wallet – View Transaction History

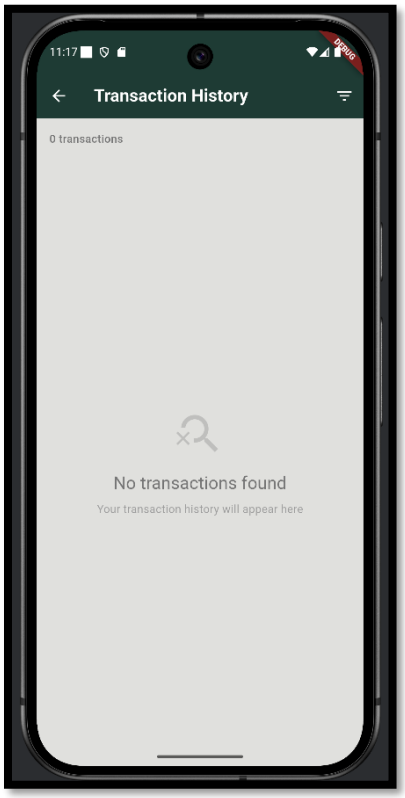
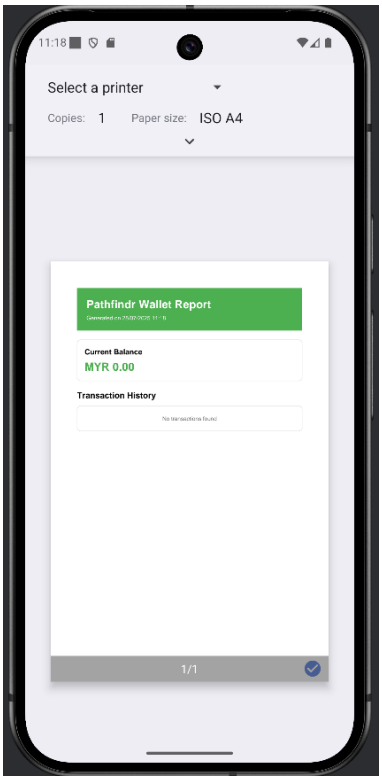


Figure 169

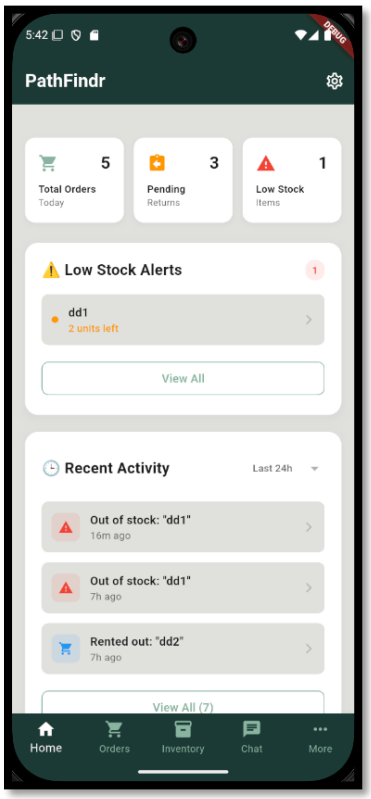
Wallet – Generate Transaction History in PDF format & Print Transaction History



3.4 Equipment Provider Role

Figure 170

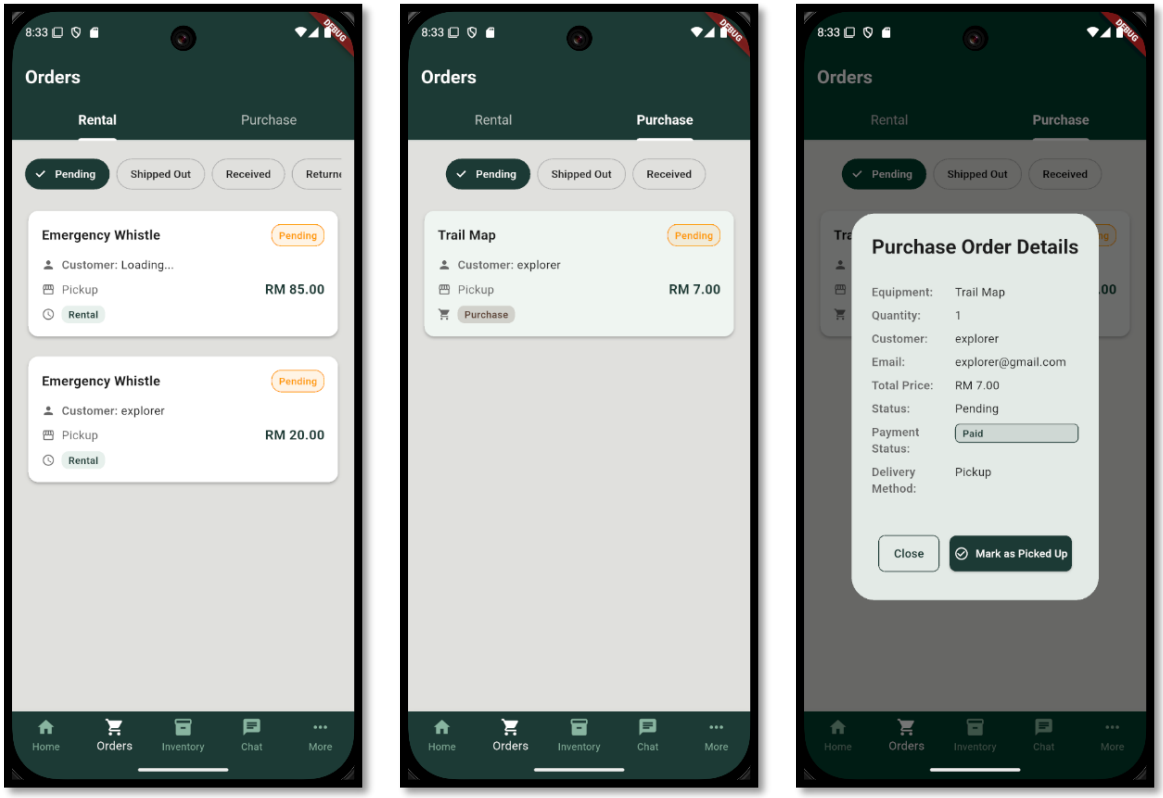
Main Dashboard



After the Equipment Provider logs in with their credentials, they will be directed to their main dashboard. On this page, the user can view statistical data, including the total orders for the current day, pending orders, and low stock items. In addition, the user can see specific items that have triggered low stock alerts. Recent activity is also displayed, allowing the Equipment Provider to manage their orders more efficiently.

Figure 171

Manage Rental Order

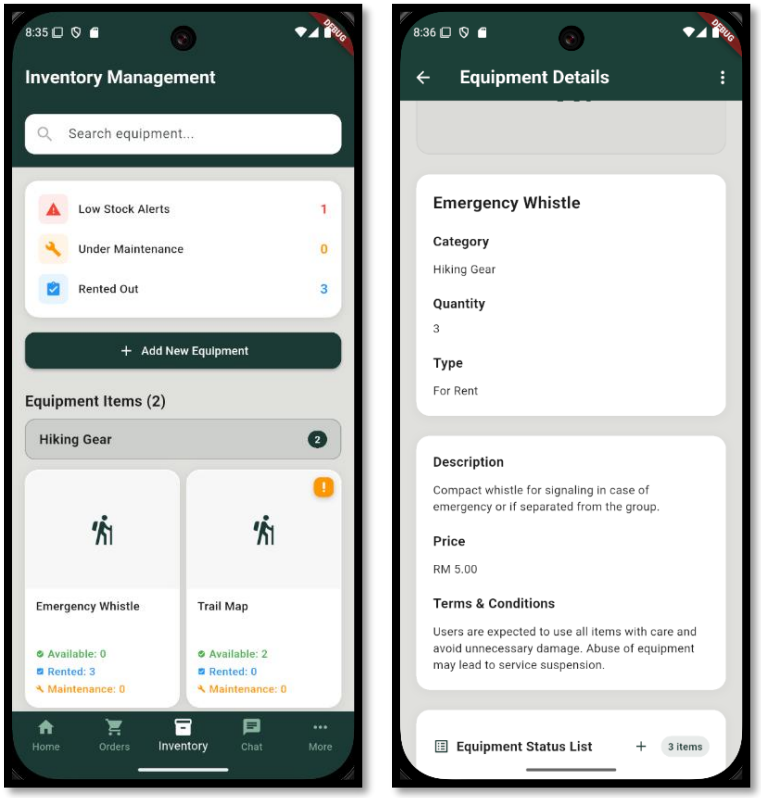


In the "Order" tab, Equipment Providers can manage customer orders, with two main categories provided: Rental and Purchase, for easier management.

Within the "Rental" category, there are five status filters: Pending, Shipped Out, Received, and Returned. Users can filter orders based on these statuses and view the corresponding order information. When clicking on a specific order, the user can also view detailed order information and update the order status accordingly.

Figure 172

Manage Inventory

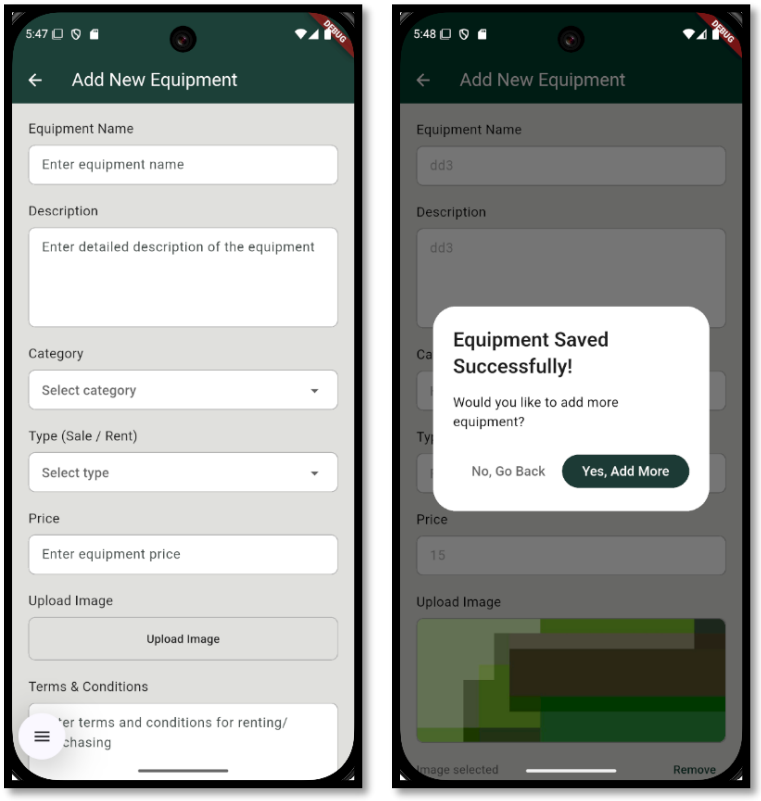


In the "Inventory" tab, users can manage all equipment. Initially, a static dashboard is displayed, showing the number of equipment items that have low stock alerts, are under maintenance, or are currently rented out. Additionally, all equipment is organized according to their respective categories.

When the user clicks on a specific equipment item, they can view its details, including the equipment name, image, description, category, quantity, type, price, and terms and conditions. A list of equipment stock codes is also displayed for users to view and manage. If the user wants to add new equipment, an "Add New Equipment" button is provided.

Figure 173

Add New Equipment

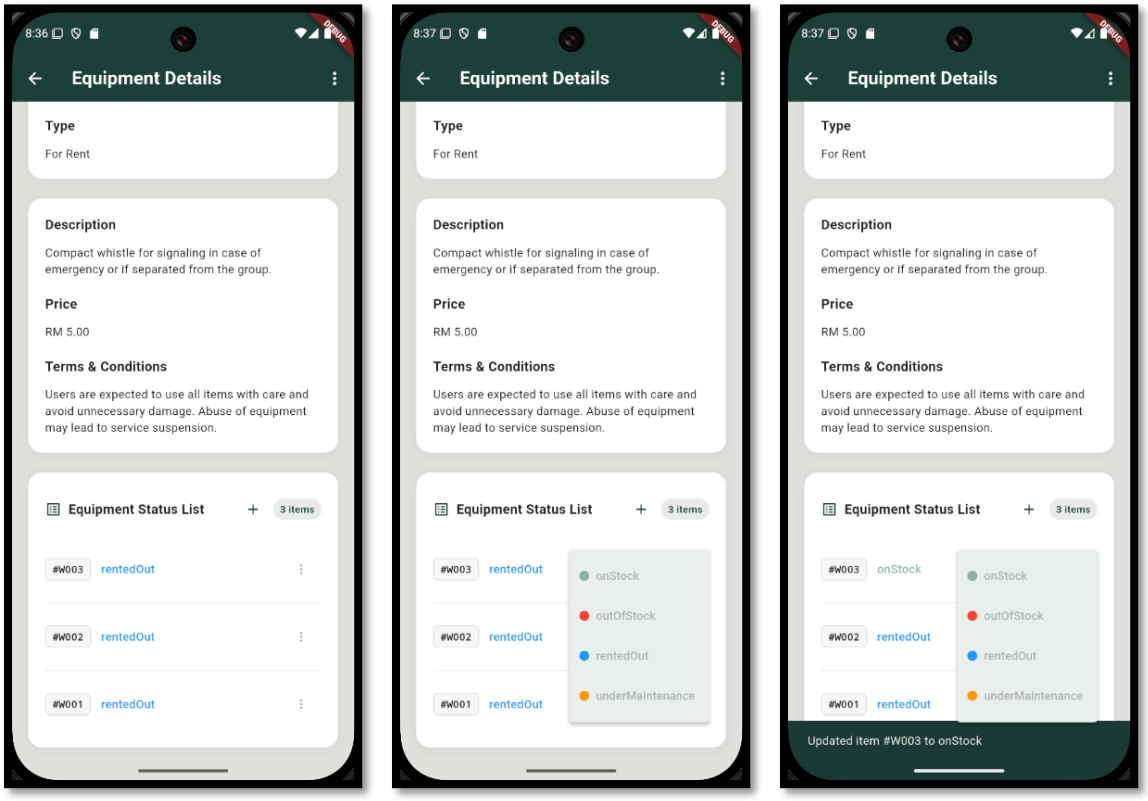


In the "Add New Equipment" section, users can enter the details of a new equipment item. Multiple fields are required, including the equipment name, description, category, order type (rent or purchase), price, image, and terms and conditions.

After the user adds the equipment, a pop-up box will appear asking whether they want to add another equipment item. If the user selects "Yes," they will be redirected back to the "Add New Equipment" page. If the user selects "No," they will be redirected to the "Inventory" tab.

Figure 174

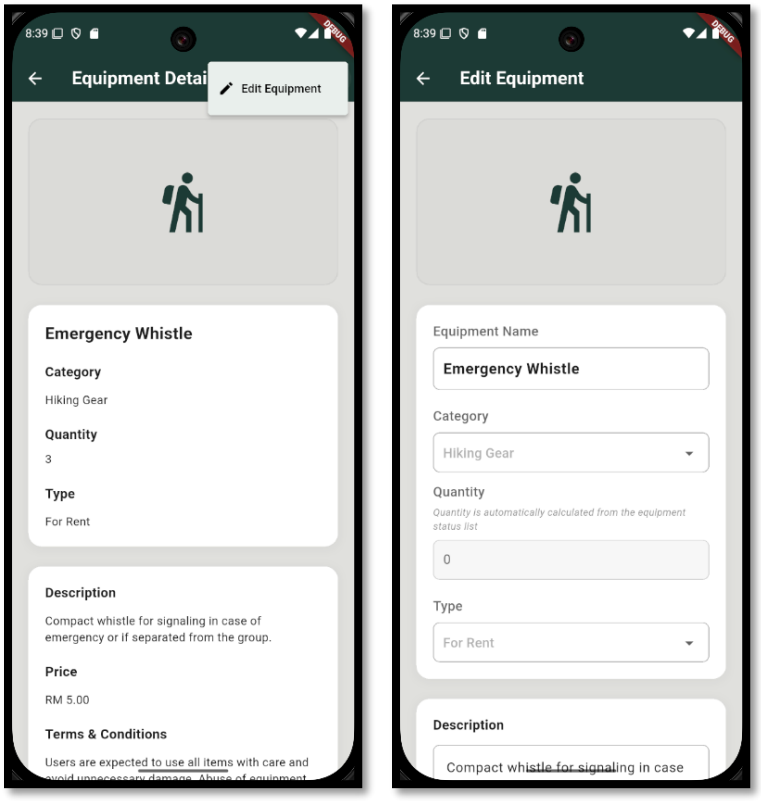
Manage New Equipment Stock



After a new equipment item is added, the user can add stock codes, as there may be multiple units available for purchase or rent under each category. The Equipment Provider can assign a stock code and status for each specific item. Additionally, the status of each item can be modified as needed.

Figure 175

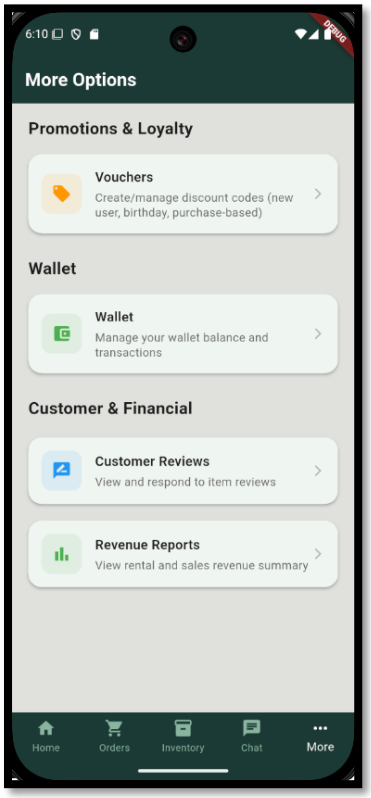
Manage Equipment



When the "Edit Equipment" button is clicked, it allows the user to modify the equipment details, such as the equipment name, description, category, type, and terms and conditions. After making the changes, the user can click the "Save" button to update the equipment information.

Figure 176

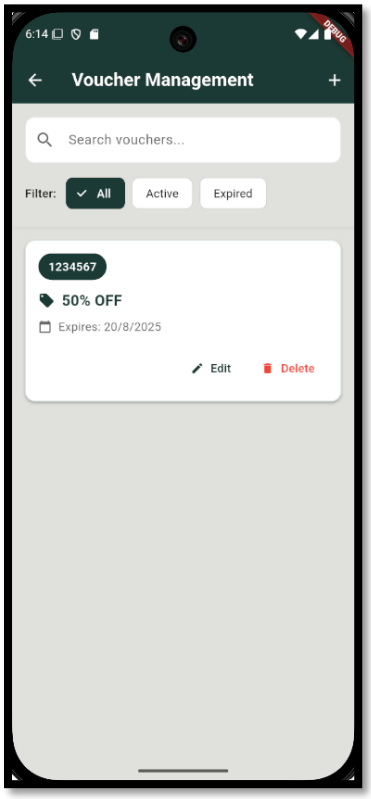
More Option



When the user accesses the "More" tab, they can access additional features such as vouchers, wallet, customer reviews, and revenue reports.

Figure 177

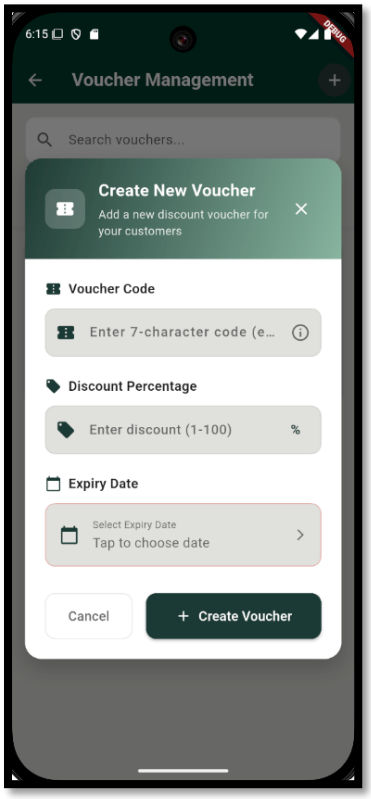
Voucher Management



The "Voucher" feature allows Equipment Providers to create vouchers for their customers. Three status filters are provided: All, Active, and Expired, enabling users to manage and filter vouchers easily. Additionally, a search bar is available for users to search for a specific voucher.

Figure 178

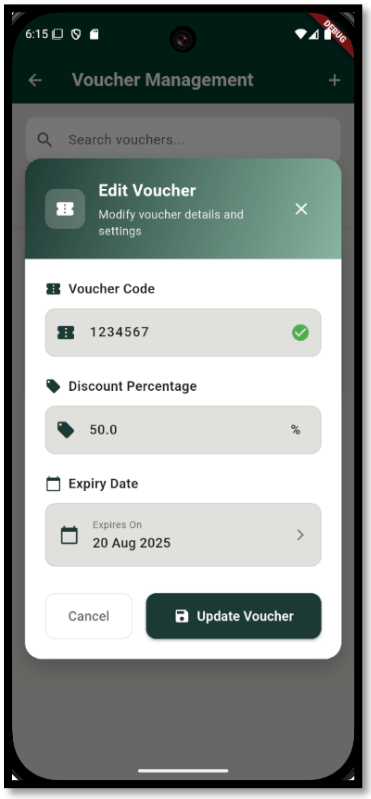
Add New Voucher



When the user clicks on the "+" icon at the top right corner, they can create a new voucher by entering details such as a 7 character voucher code, discount percentage, and expiry date. Once the voucher is created, it becomes available for Explorers to apply when purchasing or renting specific equipment.

Figure 179

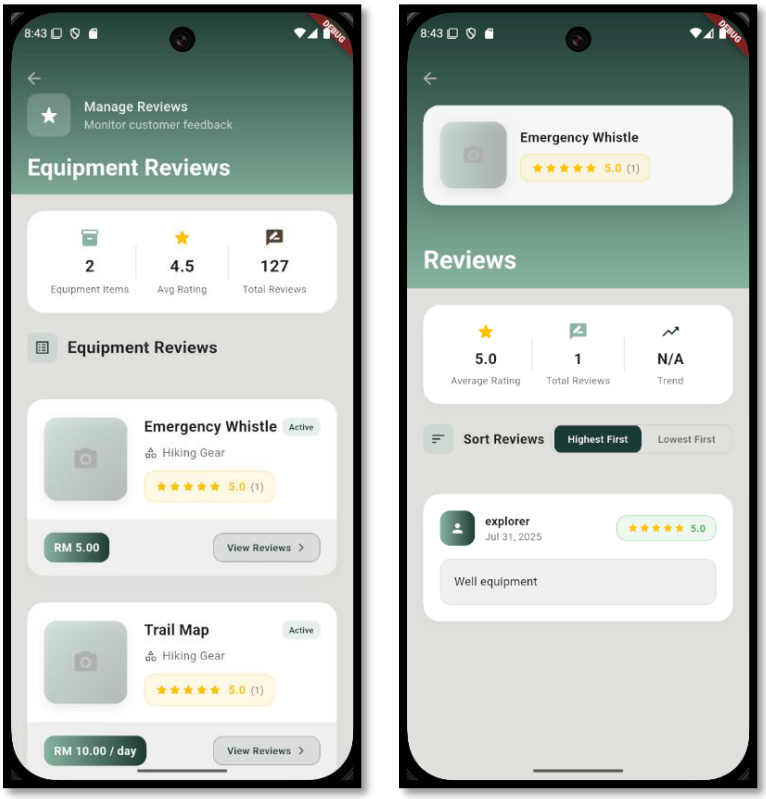
Edit / Delete Voucher



When the user needs to edit voucher information, an "Edit" button is provided to allow them to modify the voucher details. After making the changes, the user can click the "Update Voucher" button to save the updates. Additionally, the user can delete a voucher by clicking the "Delete" button.

Figure 180

Equipment Review

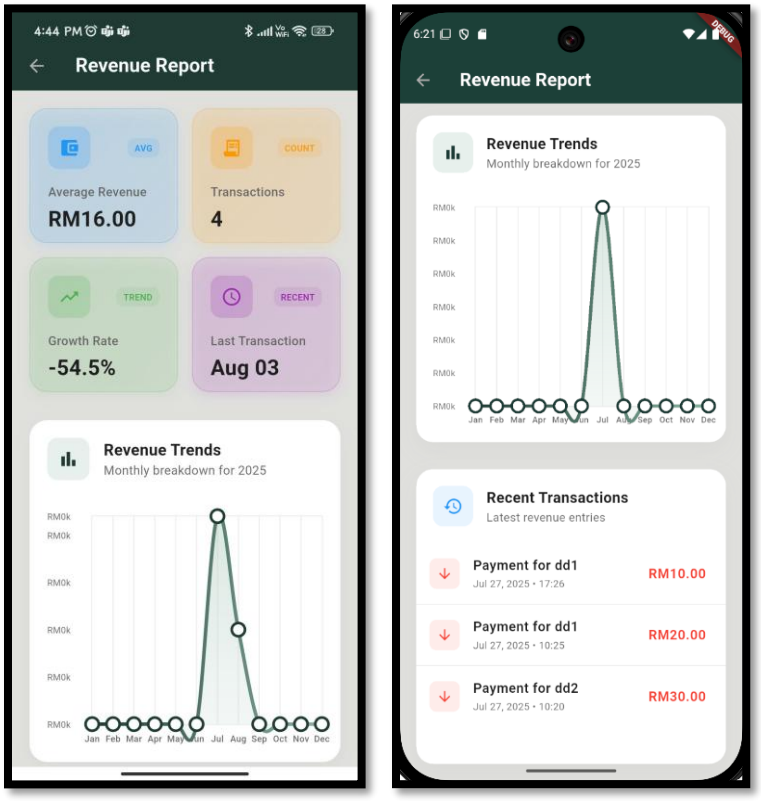


In the "Equipment Review" section, users can view all reviews submitted by Explorers after purchasing or renting equipment. The dashboard displays the total number of equipment items, average rating, and total number of reviews. Users can also view specific reviews for each equipment item.

By clicking on a particular equipment item, users can see detailed information such as the average rating, total reviews, rating trends, and individual review details for that item. Additionally, users can sort the reviews by highest or lowest rating.

Figure 181

Revenue Report



In the "Revenue Report" section, Equipment Providers can view their earnings. Key metrics such as average revenue, number of transactions, growth rate, and the date of the last transaction are displayed in a statistical format for easy reference. Additionally, revenue trends are presented in a line graph to help users analyze and adjust their business strategies more effectively. The transaction history is also displayed in a table format for detailed tracking.

4.0 Automated System Testing

4.1 Manual Testing

For manual testing, the project will focus on conducting unit testing. To facilitate this process, the test library will be utilized, as it provides the capability to easily test independent functions and services. Conducting unit testing is crucial, as it ensures that all parts of the application run smoothly and are free of bugs. This helps to verify that the user's input, and Firebase output are fully correct and error free.

4.1.1 Authentication Validation Unit Test

Authentication plays a critical role in ensuring the security of the user data and controlling user access within the application. Registration is also essential for maintaining data integrity by validating user input, especially email addresses and passwords, according to the defined requirements. The first part of the unit testing focuses on verifying the email and password validation logic.

4.1.1.1 Email Validation

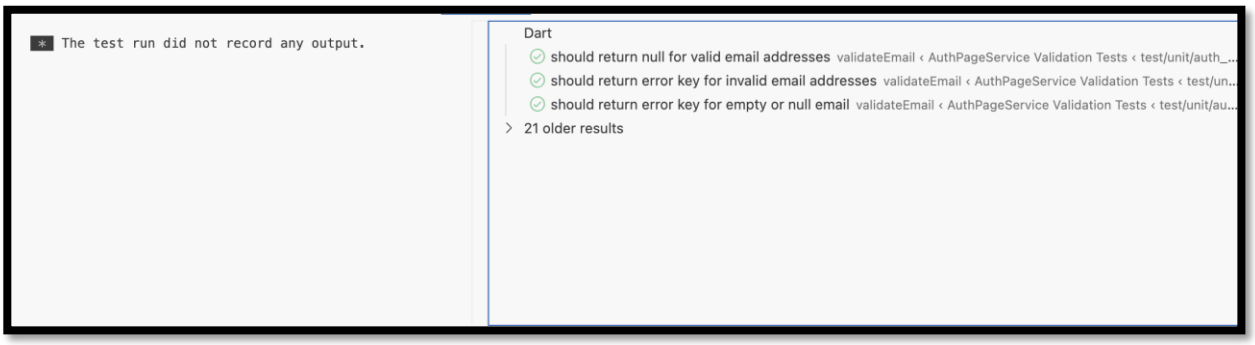
Figure 182

Code Snippet for Testing Email Validation

```

group('validateEmail', () {
  test('should return null for valid email addresses', () {
    // Arrange & Act & Assert
    expect(AuthPageService.validateEmail('test@example.com'), null);
    expect(AuthPageService.validateEmail('username@domain.co'), null);
    expect(AuthPageService.validateEmail('test123@gmail.com'), null);
    expect(AuthPageService.validateEmail('usertag@example.org'), null);
  });

  test('should return error key for invalid email addresses', () {
    // Arrange & Act & Assert
    expect(
      AuthPageService.validateEmail('invalid-email'),
      'auth.email.error.invalid',
    );
    expect(
      AuthPageService.validateEmail('test@'),
      'auth.email.error.invalid',
    );
    expect(
      AuthPageService.validateEmail('@domain.com'),
      'auth.email.error.invalid',
    );
    expect(
      AuthPageService.validateEmail('test@domain'),
      'auth.email.error.invalid',
    );
  });

  test('should return error key for empty or null email', () {
    // Arrange & Act & Assert
    expect(AuthPageService.validateEmail(''), 'auth.email.error.empty');
    expect(AuthPageService.validateEmail('  '), 'auth.email.error.empty');
  });
});
```

Figure 183

Code Snippet for Testing Results of Email Validation



The figure above shows that the AuthPageSerice.validateEmail() function successfully passed all the unit tests. This indicates that the service correctly identifies the invalid email formats and handles empty inputs by returning appropriate error messages. These results indicate that the validation logic is reliable and can prevent users from submitting improper email addresses during registration.

4.1.1.2 Password Validation

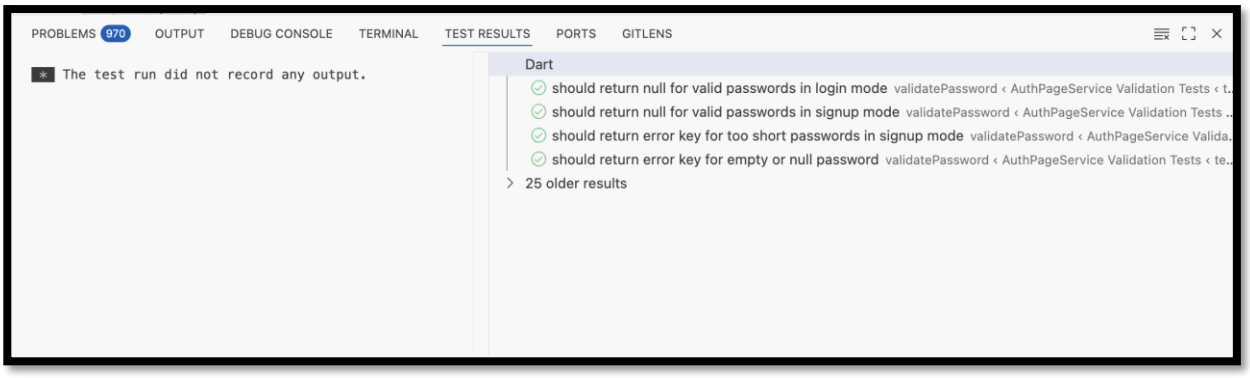
Figure 184

Code Snippet for Testing Password Validation

```
group('validatePassword', () {
  test('should return null for valid passwords in login mode', () {
    // Arrange & Act & Assert
    expect(
      AuthPageService.validatePassword('short', isSignup: false),
      null,
    );
    expect(
      AuthPageService.validatePassword('12345', isSignup: false),
      null,
    );
    expect(AuthPageService.validatePassword('a', isSignup: false), null);
  });

  test('should return null for valid passwords in signup mode', () {
    // Arrange & Act & Assert
    expect(
      AuthPageService.validatePassword('password123', isSignup: true),
      null,
    );
    expect(
      AuthPageService.validatePassword('mySecurePass!', isSignup: true),
      null,
    );
    expect(
      AuthPageService.validatePassword('123456', isSignup: true),
      null,
    );
  });

  test(
    'should return error key for too short passwords in signup mode',
    () {
      // Arrange & Act & Assert
      expect(
        AuthPageService.validatePassword('short', isSignup: true),
        'auth.password.error.too_short',
      );
      expect(
        AuthPageService.validatePassword('12345', isSignup: true),
        'auth.password.error.too_short',
      );
      expect(
        AuthPageService.validatePassword('a', isSignup: true),
        'auth.password.error.too_short',
      );
    },
  );

  test('should return error key for empty or null password', () {
    // Arrange & Act & Assert
    expect(
      AuthPageService.validatePassword(null),
      'auth.password.error.empty',
    );
    expect(
      AuthPageService.validatePassword(''),
      'auth.password.error.empty',
    );
    expect(
      AuthPageService.validatePassword(null, isSignup: true),
      'auth.password.error.empty',
    );
    expect(
      AuthPageService.validatePassword('', isSignup: true),
      'auth.password.error.empty',
    );
  });
});
```

Figure 185

Code Snippet for Testing Results of Password Validation



In terms of password validation, the validatePassword function also successfully passed all the tests, including those for both the login and sign-up pages. Moreover, it correctly returned appropriate error messages when users entered passwords that were fewer than 6 characters, as well as in case of empty or null passwords. This indicates that password validation is functioning correctly across both login and sign-up flows, ensuring that users cannot submit passwords that are too short or invalid.

4.1.2 Voucher Validation Unit Test

In the voucher section, there are many calculations and validations involved, and incorrect calculation or identification of a voucher might cause revenue loss as well as dissatisfaction from the equipment provider. Therefore, it is essential to conduct unit testing on this module. Since this section involves fetching data from Firebase, we will simulate the data in this scenario and apply the same logic used in the service to test the core functionality.

4.1.2.1 Voucher Validation Logic

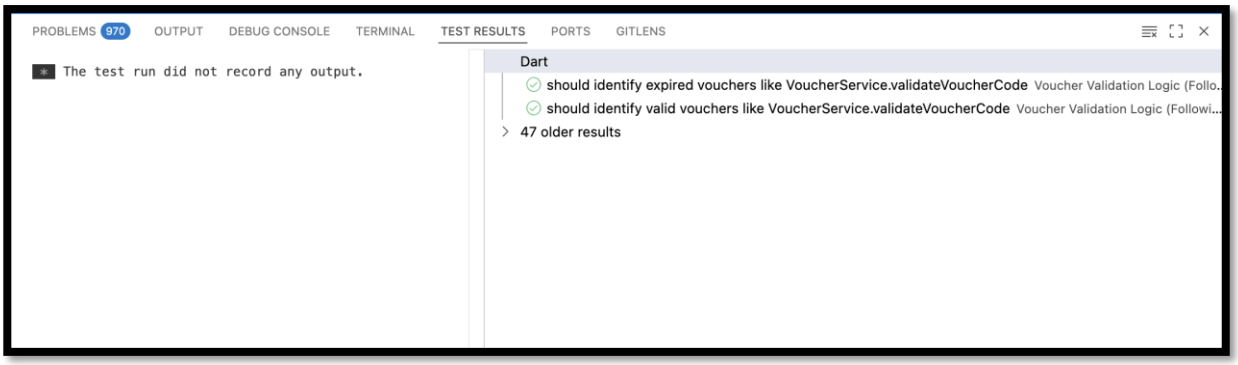
Figure 186

Code Snippet for Testing Voucher Validation

```dart
group('Voucher Validation Logic (Following VoucherService)', () {
  test(
    'should identify expired vouchers like VoucherService.validateVoucherCode',
    () {
      // Arrange - Simulate expired voucher
      final expiredDate = DateTime.now().subtract(const Duration(days: 1));
      final expiredVoucher = VoucherModel(
        id: 'expired_voucher',
        code: 'EXPIRED',
        discount: 10.0,
        expiryDate: expiredDate,
      );

      // Act - Simulate VoucherService validation logic
      final isExpired = expiredVoucher.expiryDate.isBefore(DateTime.now());

      // Assert
      expect(isExpired, true);
    },
  );

  test(
    'should identify valid vouchers like VoucherService.validateVoucherCode',
    () {
      // Arrange - Simulate valid voucher
      final futureDate = DateTime.now().add(const Duration(days: 30));
      final validVoucher = VoucherModel(
        id: 'valid_voucher',
        code: 'VALID30',
        discount: 30.0,
        expiryDate: futureDate,
      );

      // Act - Simulate VoucherService validation logic
      final isExpired = validVoucher.expiryDate.isBefore(DateTime.now());

      // Assert
      expect(isExpired, false);
    },
  );
});
```

Figure 187

Code Snippet for Testing Results of Voucher Validation

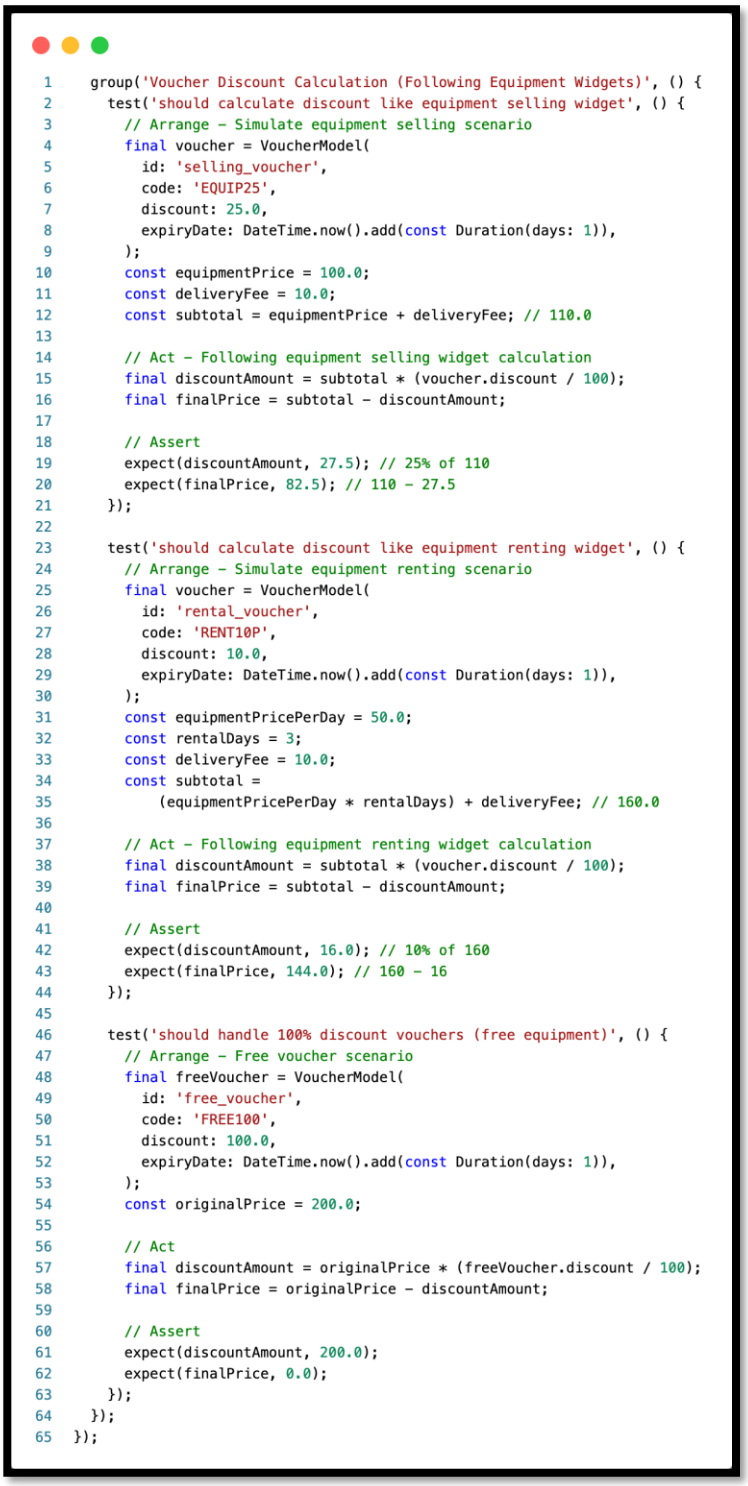


From the results, it is confirmed that the simulated validateVoucherCode function is able to correctly identify both expired and valid vouchers, returning accurate results. This ensures that the identification of valid vouchers retrieved from Firebase is reliable and helps prevent the use of invalid vouchers.

4.1.2.2 Voucher Discount Calculation

Figure 188

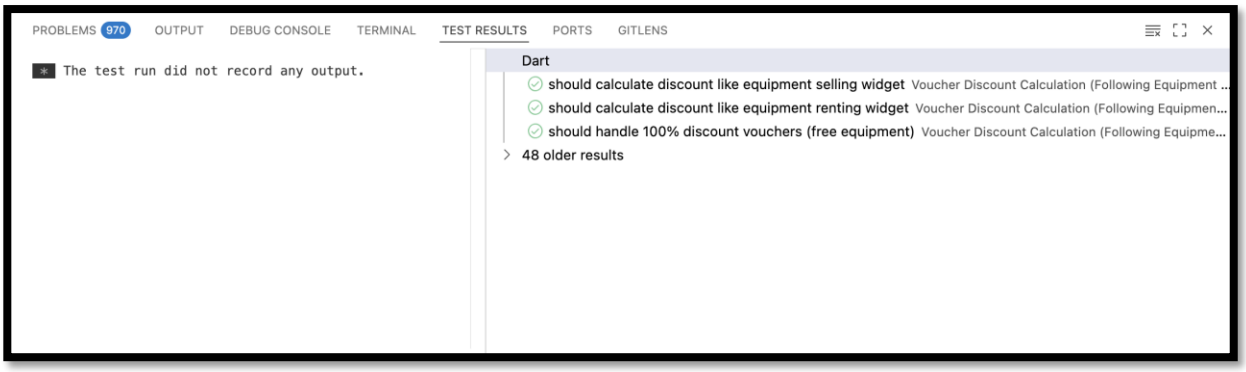
Code Snippet for Voucher Discount Calculation



```
group('Voucher Discount Calculation (Following Equipment Widgets)', () {
  test('should calculate discount like equipment selling widget', () {
    // Arrange - Simulate equipment selling scenario
    final voucher = VoucherModel(
      id: 'selling_voucher',
      code: 'EQUIP25',
      discount: 25.0,
      expiryDate: DateTime.now().add(const Duration(days: 1)),
    );
    const equipmentPrice = 100.0;
    const deliveryFee = 10.0;
    const subtotal = equipmentPrice + deliveryFee; // 110.0

    // Act - Following equipment selling widget calculation
    final discountAmount = subtotal * (voucher.discount / 100);
    final finalPrice = subtotal - discountAmount;

    // Assert
    expect(discountAmount, 27.5); // 25% of 110
    expect(finalPrice, 82.5); // 110 - 27.5
  });

  test('should calculate discount like equipment renting widget', () {
    // Arrange - Simulate equipment renting scenario
    final voucher = VoucherModel(
      id: 'rental_voucher',
      code: 'RENT10P',
      discount: 10.0,
      expiryDate: DateTime.now().add(const Duration(days: 1)),
    );
    const equipmentPricePerDay = 50.0;
    const rentalDays = 3;
    const deliveryFee = 10.0;
    const subtotal =
        (equipmentPricePerDay * rentalDays) + deliveryFee; // 160.0

    // Act - Following equipment renting widget calculation
    final discountAmount = subtotal * (voucher.discount / 100);
    final finalPrice = subtotal - discountAmount;

    // Assert
    expect(discountAmount, 16.0); // 10% of 160
    expect(finalPrice, 144.0); // 160 - 16
  });

  test('should handle 100% discount vouchers (free equipment)', () {
    // Arrange - Free voucher scenario
    final freeVoucher = VoucherModel(
      id: 'free_voucher',
      code: 'FREE100',
      discount: 100.0,
      expiryDate: DateTime.now().add(const Duration(days: 1)),
    );
    const originalPrice = 200.0;

    // Act
    final discountAmount = originalPrice * (freeVoucher.discount / 100);
    final finalPrice = originalPrice - discountAmount;

    // Assert
    expect(discountAmount, 200.0);
    expect(finalPrice, 0.0);
  });
 });
});
```

Figure 189

Code Snippet for Testing Results of Voucher Discount Calculation



Based on the testing results, it was revealed that the voucher calculation logic implemented in both the equipment selling and equipment renting widgets can handle various situations, including a 100% discount correctly. It also confirmed that the calculated price is accurate and as expected.

4.1.3 Equipment Validation Unit Testing

The equipment section contains multiple layers of logic, making it one of the more complex parts of the system. it involves the management of equipment logs, where each log represents an individual item. These logs must be updated independently to ensure data integrity, accurate tracking, and real-time reflection of equipment availability and condition.

4.1.3.1 Equipment Code Existence

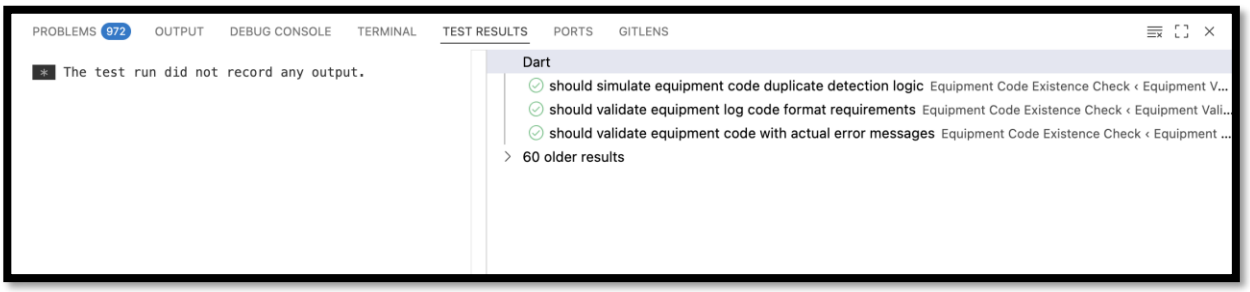
Figure 190

Code Snippet for Equipment Code Existence Validation

```dart
group('Equipment Code Existence Check', () {
  test('should simulate equipment code duplicate detection logic', () {
    // This simulates the actual logic from inventory_details.dart line 701
    // final existed = await EquipmentService.doesLogCodeExist(
    //   widget.equipment.id,
    //   code,
    // );
    // if (existed) {
    //   ScaffoldMessenger.of(context).showSnackBar(
    //     SnackBar(
    //       content: Text('Item #$code already exists.'),
    //       backgroundColor: Colors.orange,
    //     ),
    //   );
    //   return;
    // }

    // Simulate existing codes in equipment
    final existingCodes = ['T001', 'T002', 'T003', 'ABC123'];

    // Test duplicate detection
    expect(_doesCodeExist(existingCodes, 'T001'), true);
    expect(_doesCodeExist(existingCodes, 'T002'), true);
    expect(_doesCodeExist(existingCodes, 'ABC123'), true);

    // Test non-existing codes
    expect(_doesCodeExist(existingCodes, 'T004'), false);
    expect(_doesCodeExist(existingCodes, 'XYZ999'), false);
    expect(_doesCodeExist(existingCodes, 'NEW001'), false);
  });

  test('should validate equipment log code format requirements', () {
    // Based on ACTUAL validation from add_item_dialog.dart validator function:
    // if (value == null || value.trim().isEmpty) {
    //   return 'Please enter equipment code';
    // }
    // if (value.trim().length < 3) {
    //   return 'Code must be at least 3 characters';
    // }
    // if (!RegExp(r'^[A-Za-z0-9]+$').hasMatch(value.trim())) {
    //   return 'Use only letters and numbers (A-Z, 0-9)';
    // }

    // Valid formats
    expect(_isValidLogCode('T001'), true);
    expect(_isValidLogCode('T999'), true);
    expect(_isValidLogCode('ABC123'), true);
    expect(_isValidLogCode('EQP001'), true);

    // Invalid formats - empty/null
    expect(_isValidLogCode(''), false);
    expect(_isValidLogCode('   '), false); // Whitespace only

    // Invalid formats - too short
    expect(_isValidLogCode('T'), false);
    expect(_isValidLogCode('AB'), false);

    // Invalid formats - special characters
    expect(_isValidLogCode('T-001'), false); // Hyphen
    expect(_isValidLogCode('T 001'), false); // Space
    expect(_isValidLogCode('T@001'), false); // Special character
  });

  test('should validate equipment code with actual error messages', () {
    // This tests the actual validator function from add_item_dialog.dart

    // Valid cases
    expect(_validateEquipmentCode('T001'), null);
    expect(_validateEquipmentCode('ABC123'), null);
    expect(_validateEquipmentCode('EQP001'), null);

    // Invalid cases with exact error messages
    expect(_validateEquipmentCode(''), 'Please enter equipment code');
    expect(_validateEquipmentCode('   '), 'Please enter equipment code');
    expect(_validateEquipmentCode(null), 'Please enter equipment code');

    expect(
      _validateEquipmentCode('T'),
      'Code must be at least 3 characters',
    );
    expect(
      _validateEquipmentCode('AB'),
      'Code must be at least 3 characters',
    );

    expect(
      _validateEquipmentCode('T-001'),
      'Use only letters and numbers (A-Z, 0-9)',
    );
    expect(
      _validateEquipmentCode('T 001'),
      'Use only letters and numbers (A-Z, 0-9)',
    );
    expect(
      _validateEquipmentCode('T@001'),
      'Use only letters and numbers (A-Z, 0-9)',
    );
  });
});
```

Figure 191

Code Snippet for Testing Results of Equipment Code Existence Validation



The test focuses on input validation and verifying whether the voucher code already exists in Firebase before attempting to save it. However, no actual Firebase calls are made during testing; instead, mock data is used to simulate the logic. The results revealed that the code successfully detects duplicate entries, validate the input format, and displays the appropriate error messages based on the given input.

4.1.3.2 Equipment Log Management

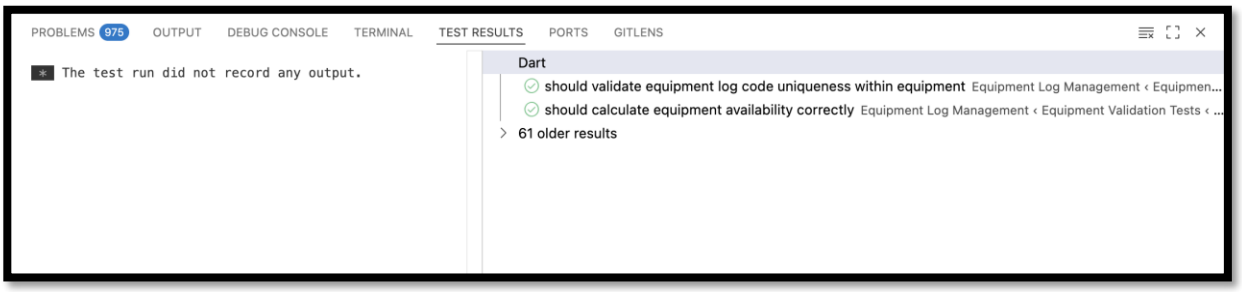
Figure 192

Code Snippet for Equipment Log Management

```dart
group('Equipment Log Management', () {
  test(
    'should validate equipment log code uniqueness within equipment',
    () {
      // This tests the duplicate checking logic for equipment logs

      final mockEquipmentLogs = [
        {'code': 'T001', 'status': 'onStock'},
        {'code': 'T002', 'status': 'rentedOut'},
        {'code': 'T003', 'status': 'underMaintenance'},
        {'code': 'ABC123', 'status': 'onStock'},
      ];

      // Test duplicate detection
      expect(_isLogCodeUnique(mockEquipmentLogs, 'T001'), false);
      expect(_isLogCodeUnique(mockEquipmentLogs, 'T002'), false);
      expect(_isLogCodeUnique(mockEquipmentLogs, 'ABC123'), false);

      // Test unique codes
      expect(_isLogCodeUnique(mockEquipmentLogs, 'T004'), true);
      expect(_isLogCodeUnique(mockEquipmentLogs, 'XYZ999'), true);
      expect(_isLogCodeUnique(mockEquipmentLogs, 'NEW001'), true);
    },
  );

  test('should calculate equipment availability correctly', () {
    // Test equipment counting logic for inventory management
    final mockLogs = [
      {'code': 'T001', 'status': 'onStock'},
      {'code': 'T002', 'status': 'onStock'},
      {'code': 'T003', 'status': 'rentedOut'},
      {'code': 'T004', 'status': 'rentedOut'},
      {'code': 'T005', 'status': 'underMaintenance'},
      {'code': 'T006', 'status': 'outOfStock'},
    ];

    expect(_countLogsByStatus(mockLogs, 'onStock'), 2);
    expect(_countLogsByStatus(mockLogs, 'rentedOut'), 2);
    expect(_countLogsByStatus(mockLogs, 'underMaintenance'), 1);
    expect(_countLogsByStatus(mockLogs, 'outOfStock'), 1);

    // Test availability calculation
    expect(_calculateAvailableItems(mockLogs), 2); // Only onStock items
    expect(_calculateTotalItems(mockLogs), 6); // All items
    expect(_calculateRentedItems(mockLogs), 2); // Only rented items
  });
});
```

Figure 193

Code Snippet for Testing Results of Equipment Log Management



These tests focus on verifying that each equipment log code is unique, ensuring that there are no duplicate entries that could make it difficult for equipment providers to track and analyse individual items. In addition, the tests also validate the logic responsible for calculating the total number of available products, rented products, items under maintenance, and out-of-stock products from a given list. Since this logic is implemented on the UI layer, it is crucial to ensure that the counts are accurate to prevent any misinterpretation or confusion for the equipment provider.

4.2 Automated Testing

In order to ensure the stability of PathFindr in various applications, conducting Robo testing is necessary. To facilitate this process, Test Lab in Firebase will be utilized, as it involves a series of Android devices for testing. This approach is very critical and essential, as it ensures that the PathFindr application can run stably and without crashes on various devices, ensuring a smooth user experience.

Moreover, the Test Lab also enables zero-effort setup, as the developers will only be required to upload the app to the Test Lab, and Firebase will return the results within a short period of time. Test Lab will also generate crash reports and visual logs automatically, enabling better enhancements and debugging to be conducted by the developers.

In this report, Robo testing will be conducted on four main sections of the application, and each part will be tested independently. These sections are the pre-login section, explorer section, guide section, and equipment provider section.

4.2.1 Pre-Login Section

In this section, the focus is mainly on whether the onboarding page and the main screen before login will crash on certain devices. It does not require any specific setup of the application, as it only involves the pre-login section. The figure below shows the results of the Robo testing for this section.

Figure 194

Screenshot Indicating Robo Testing Passed on Various Devices in the Pre-login Section

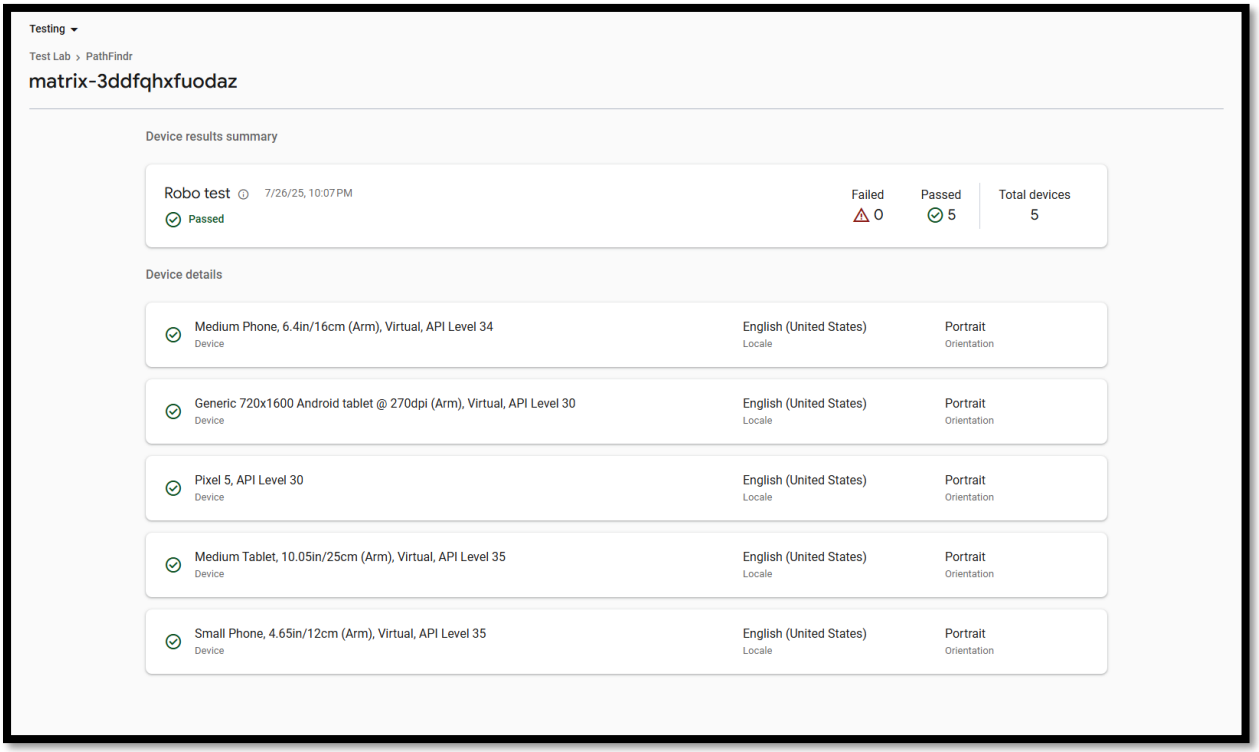


Figure 195

Screenshot Displaying Robo Testing Information for Specific Devices

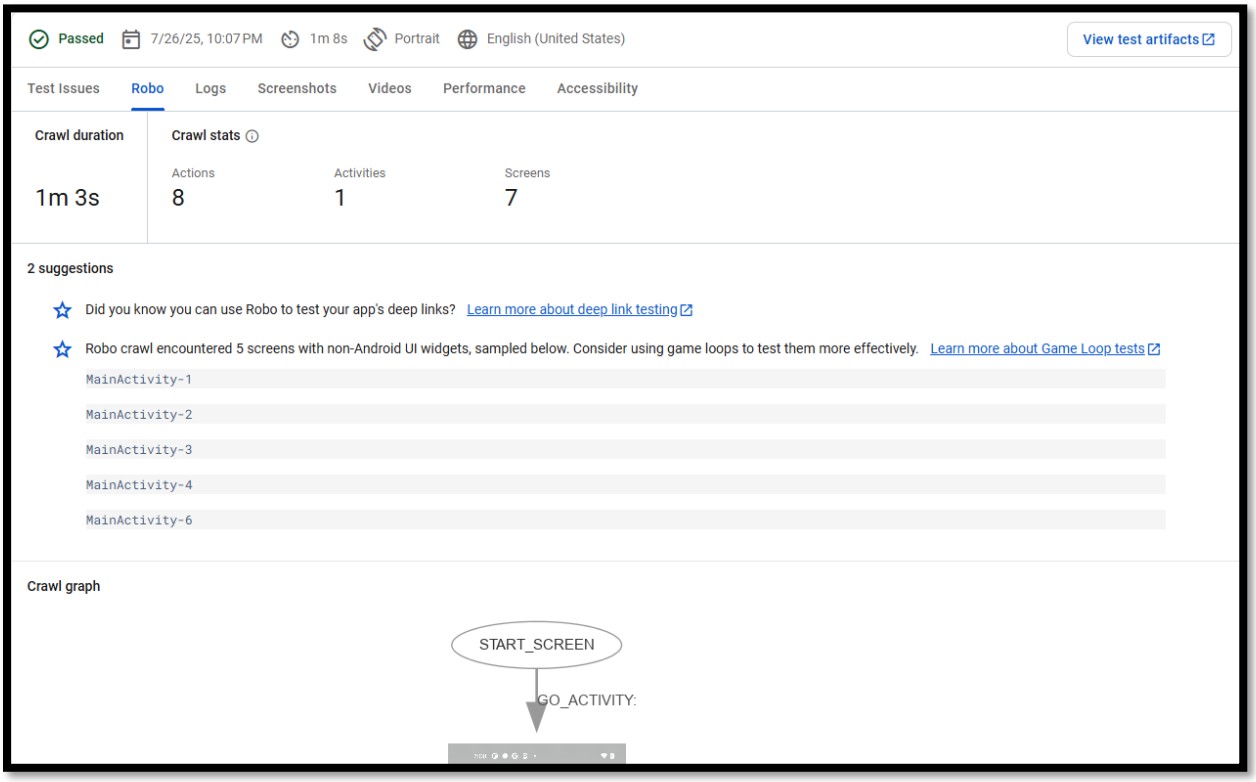
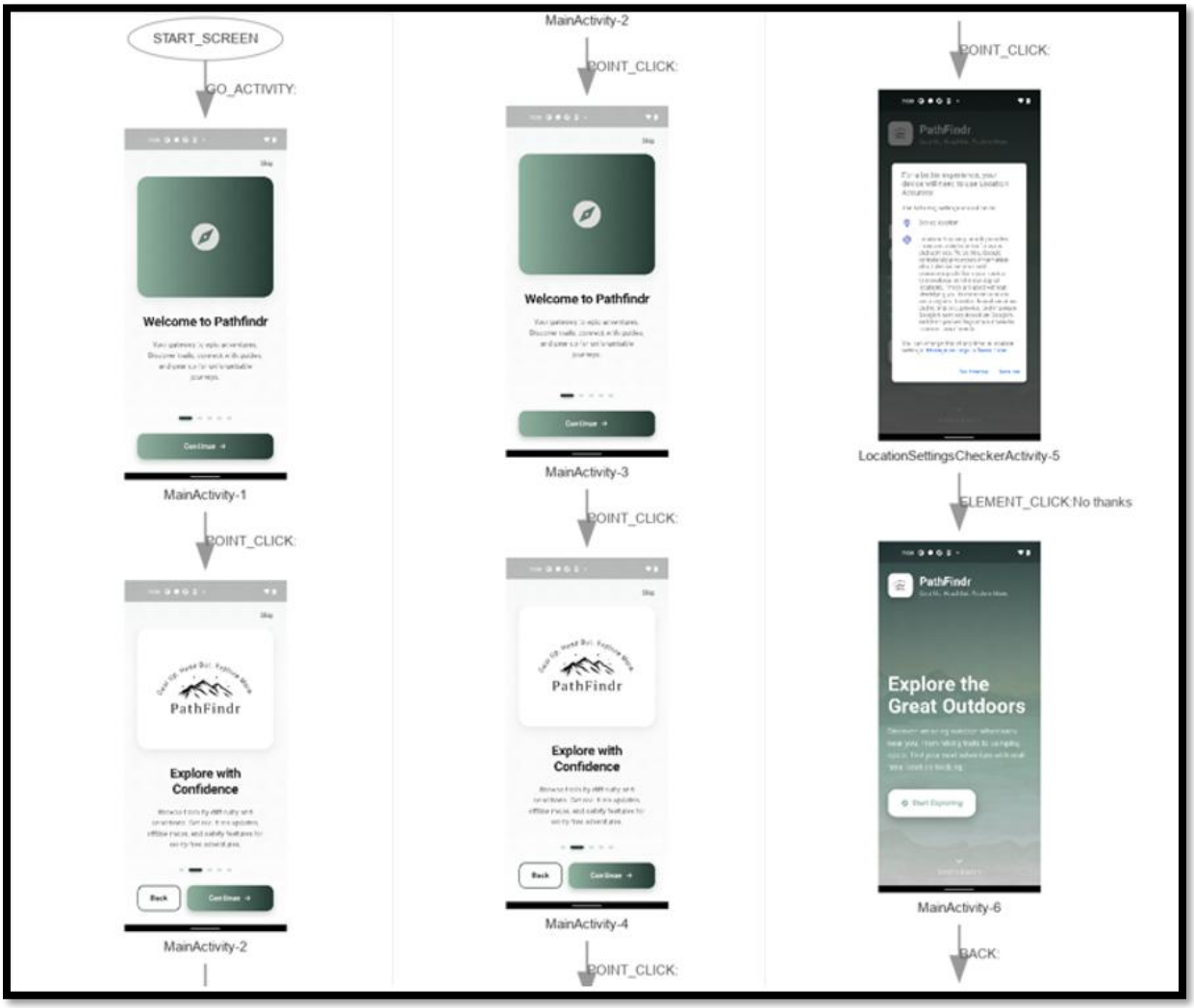


Figure 196

Screenshot Captured During the Robo Testing Flow of Pre-login Section



The screenshots above show that different devices have passed the test, and the app successfully boots on various devices. This ensures that the new users will not face issues such as the app crashing on first use, which could potentially lead to user frustration and dissatisfaction.

4.2.2 Explorer Section

For the explorer section, since this is a post-login page, Robo testing is not capable of testing it without credentials. Therefore, to facilitate this process, the app is hardcoded with the explorer's credentials and is set to automatically skip the onboarding and login pages, logging in directly to the explorer pages. The screenshot below shows the modified code used to enable automatic login.

Figure 197

Code Snippet to Enable Automatic Login to Explorer Account

```dart
class _AuthWrapperState extends State<AuthWrapper> {
  String? _lastUserId;
  bool _isLoadingProfile = false;

  @override
  void initState() {
    super.initState();
    _initializeUserProfile();
    _autoLoginForRoboTest();
  }

  Future<void> _autoLoginForRoboTest() async {
    final authService = Provider.of<AuthService>(context, listen: false);
    await authService.signInWithEmailAndPassword(
      email: 'explorer@gmail.com',
      password: '111111',
    );
  }
}
```

Figure 198

Screenshot Indicating Robo Testing Passed on Various Devices in the Explorer Section

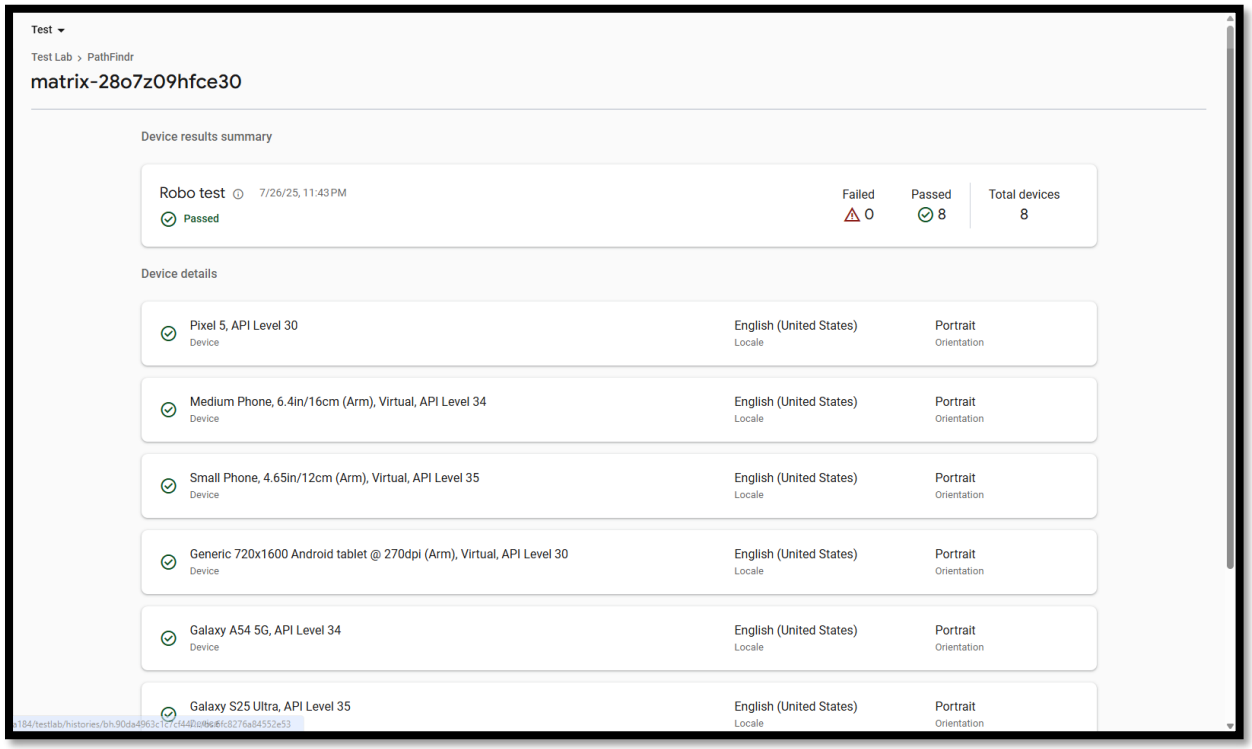


Figure 199

Screenshot Displaying Robo Testing Information for Specific Devices

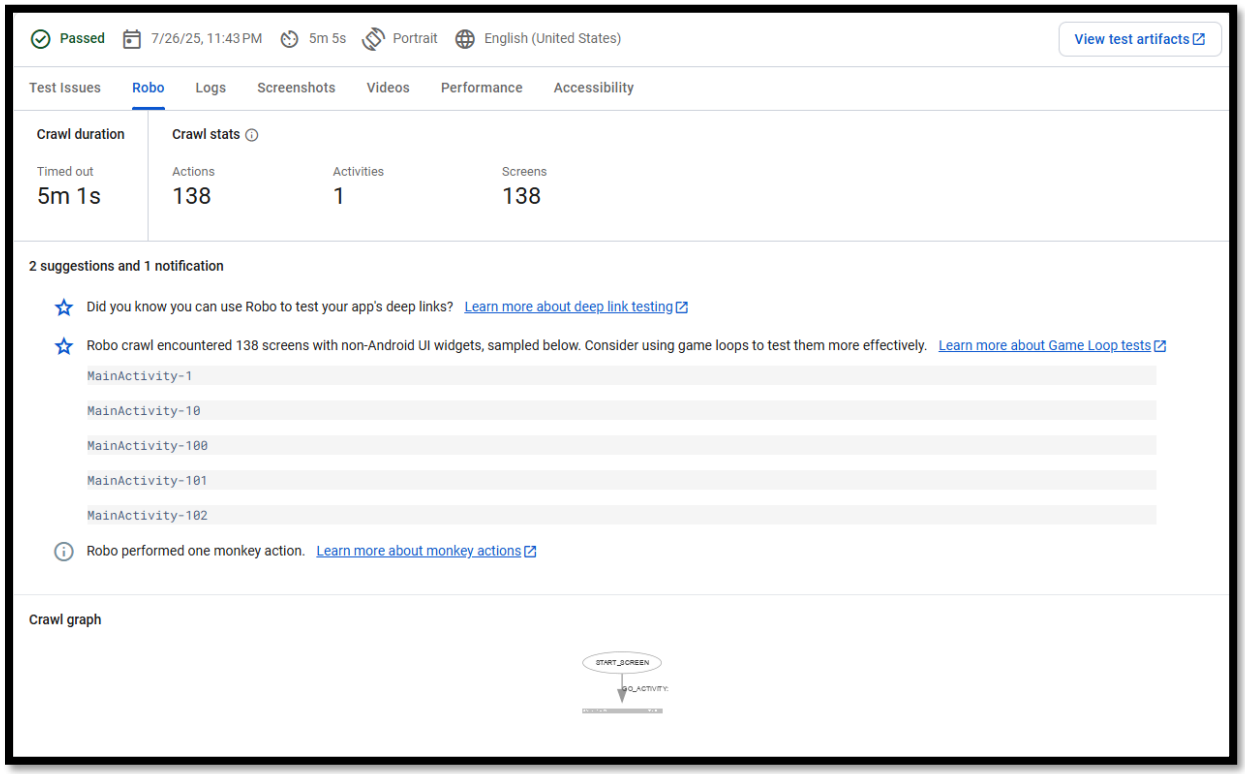
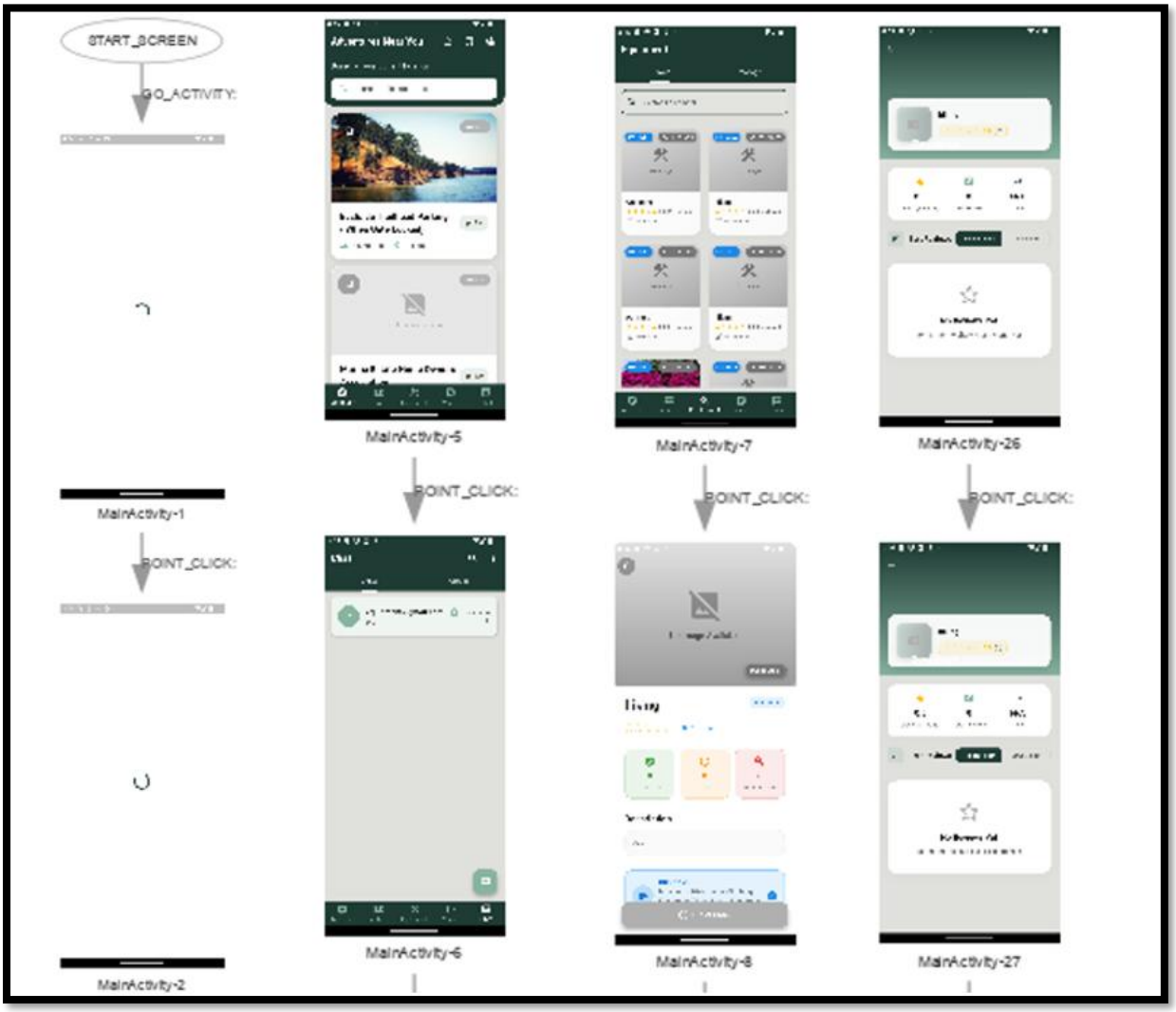


Figure 200

Screenshot Captured During the Robo Testing Flow of Explorer Section



The screenshots above indicate that the explorer section successfully runs on multiple devices. This Robo testing process does not include testing every individual page; instead, it allows the Robo to randomly navigate through different pages and check whether the app crashes. Thos helps ensure the general stability of the app across various navigation flows.

4.2.3 Guide Section

Similar to the explorer section, the guide section also requires hardcoded credentials for testing. The screenshot below shows the modified code that enables automatic login to the guide section.

Figure 201

Code Snippet to Enable Automatic Login to Guide Account

```dart
class _AuthWrapperState extends State<AuthWrapper> {
  String? _lastUserId;
  bool _isLoadingProfile = false;

  @override
  void initState() {
    super.initState();
    _initializeUserProfile();
    _autoLoginForRoboTest();
  }

  Future<void> _autoLoginForRoboTest() async {
    final authService = Provider.of<AuthService>(context, listen: false);
    await authService.signInWithEmailAndPassword(
      email: 'guide@gmail.com',
      password: '111111',
    );
  }
}
```

Figure 202

Screenshot Indicating Robo Testing Passed on Various Devices in the Guide Section

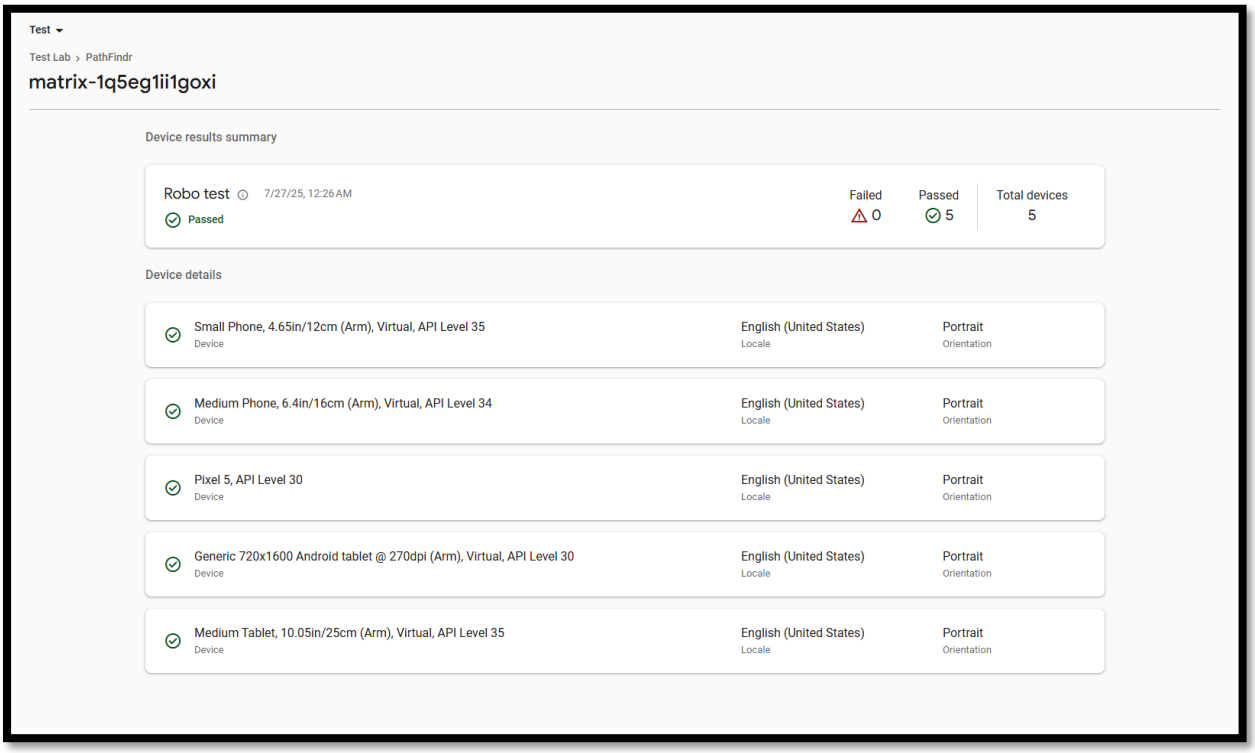


Figure 203

Screenshot Displaying Robo Testing Information for Specific Devices

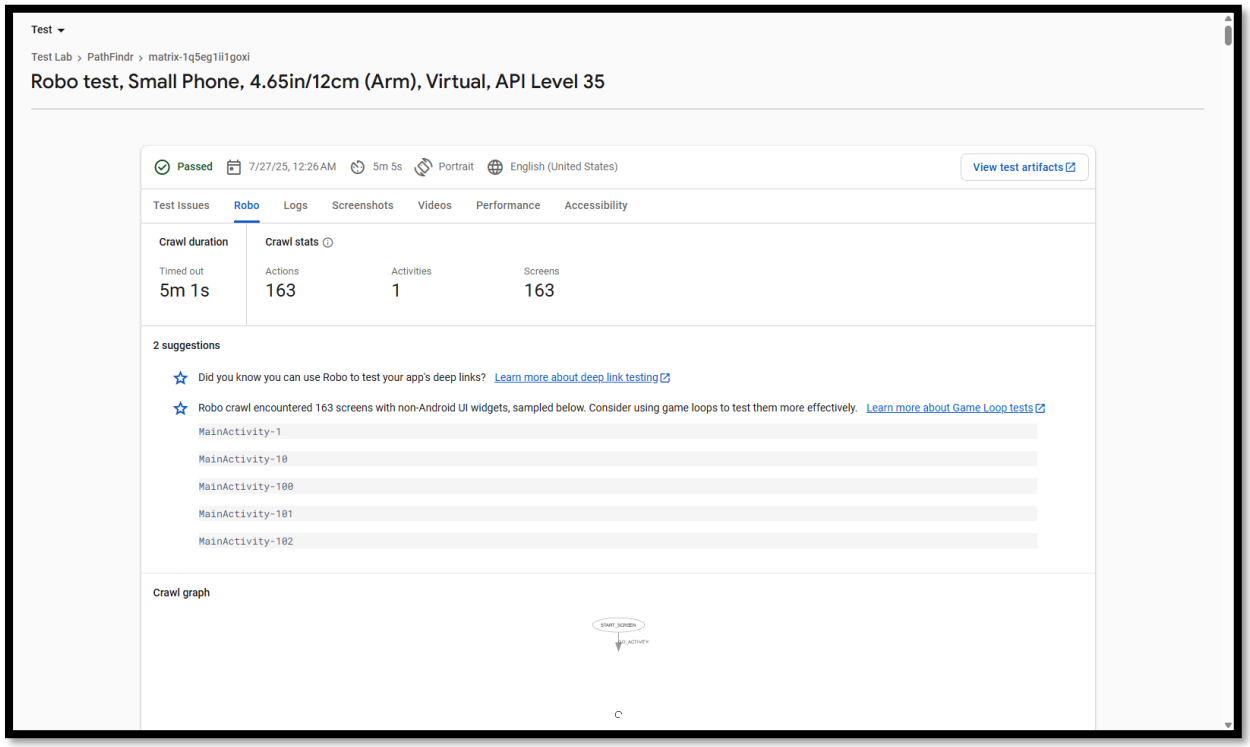
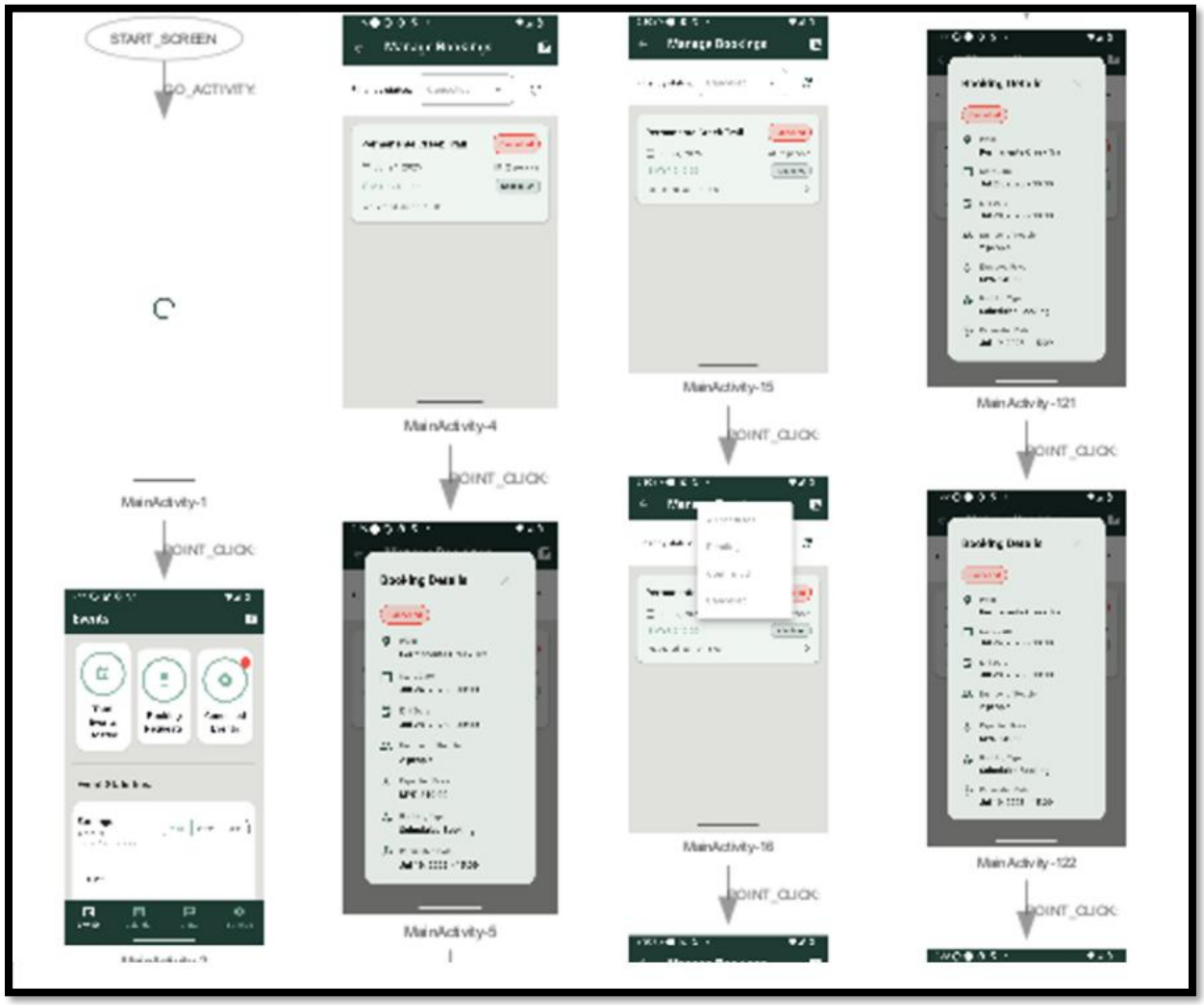


Figure 204

Screenshot Captured During the Robo Testing Flow of Guide Section



The ability of Robo testing to randomly navigate through different pages has proven the stability of the guide section and ensures that it does not crash across different brands of mobile devices.

4.2.4 Equipment Provider Section

For the Equipment Provider section, it requires the same setup as the Explorer and Guide sections. The code snippet and the screenshot showing the results are provided below.

Figure 205

Code Snippet to Enable Automatic Login to Equipment Provider Account

```
class _AuthWrapperState extends State<AuthWrapper> {
  String? _lastUserId;
  bool _isLoadingProfile = false;

  @override
  void initState() {
    super.initState();
    _initializeUserProfile();
    _autoLoginForRoboTest();
  }

  Future<void> _autoLoginForRoboTest() async {
    final authService = Provider.of<AuthService>(context, listen: false);
    await authService.signInWithEmailAndPassword(
      email: 'equipment@gmail.com',
      password: '111111',
    );
  }
}
```

Figure 206

Screenshot Indicating Robo Testing Passed on Various Devices in the Equipment Provider Section

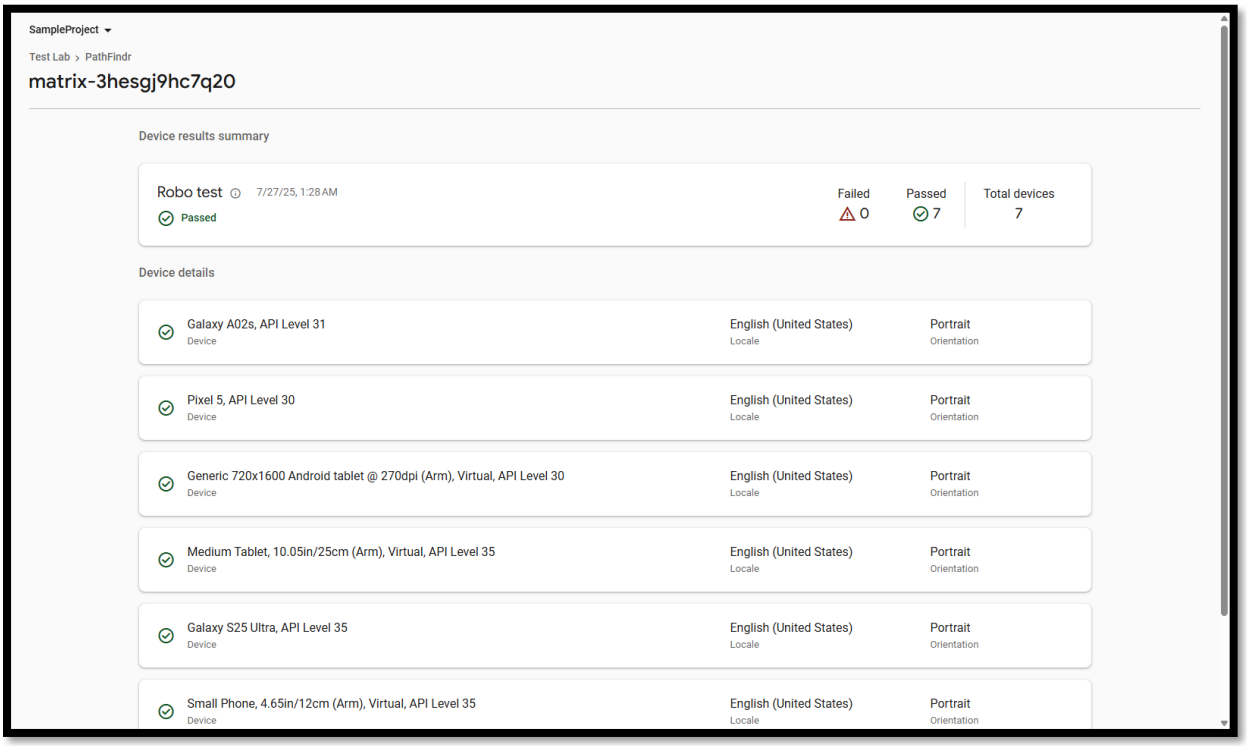


Figure 207

Screenshot Displaying Robo Testing Information for Specific Devices

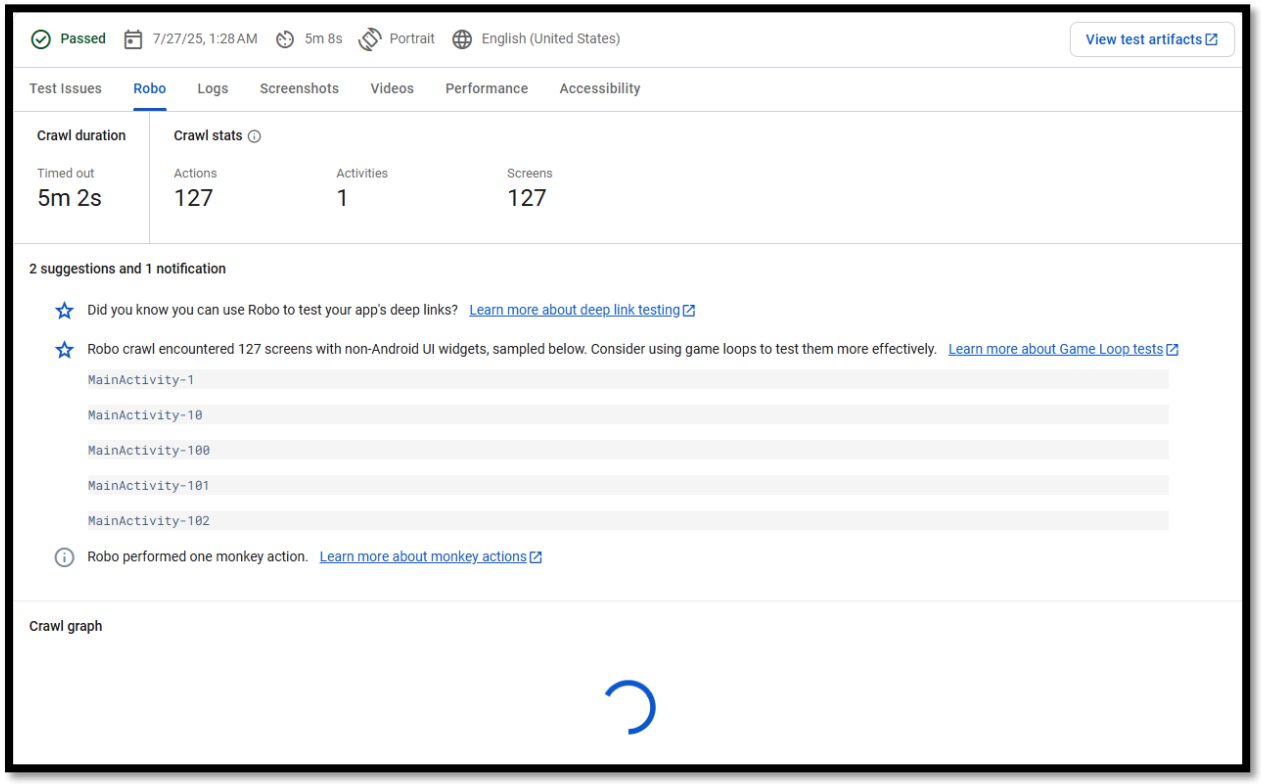


Figure 208

Screenshot Captured During the Robo Testing Flow of Equipment Provider Section

The screenshot above show that the Equipment Provider section has passed the tested across various devices without any crashing issues. This allows the development team to ensure the robustness and stability of the different sections before deploying the app to production.

5.0 Conclusion

In conclusion, PathFindr successfully delivers a robust, scalable solution for coordinating outdoor adventures, balancing user experience with maintainability and testability. By leveraging MVVM architecture and modern cloud services, the application achieves real-time data synchronization, lifecycle-aware state management, and modular code organization. The manual and automated testing phases confirmed system stability across diverse mobile platforms, with unit tests ensuring the correctness of critical functions such as authentication, voucher validation, and inventory management, as well as Firebase Test Lab's Robo tests demonstrating crash resilience. Moving forward, future enhancements could include advanced analytics for guide performance, expanded third-party integrations for trail data, and support for offline usage in remote areas. These developments will further strengthen PathFindr's position as a comprehensive platform for safe, well-managed outdoor experiences.

6.0 Reference

AllTrails. (n.d.). AllTrails: Hike, Run & Walk. Https://Apps.Apple.Com/My/App/Alltrails-Hike-Run-Walk/Id405075943.

Caminiti, K. (2025, June 11). Looking for a good hiking app? Meet AllTrails Peak, a new membership to help you explore. Https://Www.Usatoday.Com/Story/Shopping/2025/06/11/Alltrails-Peak-Membership/84136256007/.

Daniel, N. J., Patel, S. B., st. Marie, P., & Schoenfeld, E. M. (2021). Rethinking hiker preparedness: Association of carrying "10 essentials" with adverse events and satisfaction among day-hikers. The American Journal of Emergency Medicine, 49, 253–256. https://doi.org/10.1016/j.ajem.2021.06.017

Daoudi, A., ElBoussaidi, G., Moha, N., & Kpodjedo, S. (2019). An exploratory study of MVC-based architectural patterns in Android apps. Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing, 1711–1720. https://doi.org/10.1145/3297280.3297447

DiCicco-Bloom, B., & Crabtree, B. F. (2006). The qualitative research interview. In Medical Education (Vol. 40, Issue 4, pp. 314–321). https://doi.org/10.1111/j.1365-2929.2006.02418.x

Fuksa, M., Speth, S., & Becker, S. (2025). MVVM Revisited: Exploring Design Variants of the Model-View-ViewModel Pattern. https://doi.org/10.1007/978-3-031-78338-8_9

García, I., Albusac, J. A., Castro-Schez, J. J., Herrera, V., & Vallejo, D. (2023). GVIDI: A system to improve the safety of groups of hikers based on real-time monitoring and dataset generation. SoftwareX, 24, 101586. https://doi.org/10.1016/J.SOFTX.2023.101586

Gill, P., Stewart, K., Treasure, E., & Chadwick, B. (2008). Methods of data collection in qualitative research: interviews and focus groups. British Dental Journal, 204(6), 291–295. https://doi.org/10.1038/bdj.2008.192

Jamshed, S. (2014). Qualitative research method-interviewing and observation. Journal of Basic and Clinical Pharmacy, 5(4), 87. https://doi.org/10.4103/0976-0105.141942

Kallio, H., Pietilä, A., Johnson, M., & Kangasniemi, M. (2016). Systematic methodological review: developing a framework for a qualitative semi-structured interview guide. Journal of Advanced Nursing, 72(12), 2954–2965. https://doi.org/10.1111/jan.13031

Komoot. (n.d.-a). Komoot | Find, plan and share your adventures. Https://Www.Komoot.Com/.

Komoot. (n.d.-b). Komoot's Features | The Best Outdoor Tech. Https://Www.Komoot.Com/Features.

Komoot. (2023, October 11). 4 Ways to Use Komoot During your Adventure. Https://Www.Komoot.Com/Adventure-Hub/4MasE6DPzRyY4tHUgDqF6w/4-Ways-to-Use-Komoot-during-Your-Adventure.

Mohd Salleh Aman, Shariffah Mamat, Gunathevan Elumalai, Vellapandian Ponnusamy, Norhidayah Omar, & Shamshuritawati Shariff. (2024). Sport, exercise, and recreation participation in Malaysia 2023: The engagement pattern according to demographics, participant characteristics and social groups. Nanotechnology Perceptions, 354–360. https://doi.org/10.62441/nano-ntp.vi.3490

Mohd Sidi, M. A., Sulaiman, M. S., Mohd Said, O. F., & Johannis, N. J. (2020). Case Study Perception: Lack of Energy For New Hikers at Gunung Ledang, Taman Negara Johor. ATTARBAWIY: Malaysian Online Journal of Education, 4(2), 135–139. https://doi.org/10.53840/attarbawiy.v4i2.49

Nagar, T. (2025, June 11). 7 Steps To Build A Hiking App Like AllTrails In 2025. Https://Devtechnosys.Com/Insights/Build-a-Hiking-App-like-Alltrails/.

Rabionet, S. (2014). How I Learned to Design and Conduct Semi-structured Interviews: An Ongoing and Continuous Journey. The Qualitative Report. https://doi.org/10.46743/2160-3715/2011.1070

Searle, B. (2024, December 13). Komoot for beginners: Why I love it for our adventures. Https://Wildbeeoutdoors.Co.Uk/Komoot-for-Beginners-Why-i-Love-It-for-Our-Adventures/.

Sendy.io. (n.d.). The Alltrails App: Be informed, stay connected, share your experiences. Https://Sendy.Io/Blog/the-Alltrails-App-Be-Informed-Stay-Connected-Share-Your-Experiences.

von Aulock, I. (2024, October 28). Komoot App Review – Features, Pros, Cons & Best Use Cases - Baldy Basecamp. Https://Baldybasecamp.Com/Apps/Komoot.

Wilson, A., Wedyan, F., & Omari, S. (2022). An Empirical Evaluation and Comparison of the Impact of MVVM and MVC GUI Driven Application Architectures on Maintainability and Testability. 2022 International Conference on Intelligent Data Science Technologies and Applications (IDSTA), 101–108. https://doi.org/10.1109/IDSTA55301.2022.9923083

Yuen Mei Keng. (2023, September 10). INTERACTIVE: Do Malaysians get enough exercise? Find out here. Https://Apimaster.Thestar.Com.My/News/Nation/2023/09/10/Interactive-What-Are-People-in-Malaysia-Doing-to-Be-Physically-Active-Heres-a-Closer-Look